

Risk Analysis and Model Construction

FRE6871 & FRE7241, Fall 2025

Jerzy Pawlowski jp3900@nyu.edu

NYU Tandon School of Engineering

December 28, 2025



NYU

**TANDON SCHOOL
OF ENGINEERING**

Kernel Density of Asset Returns

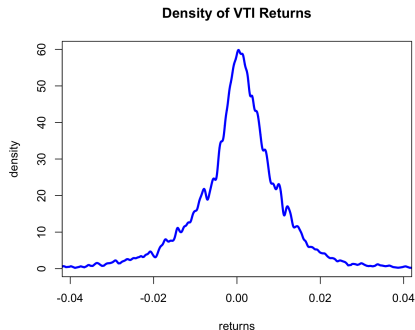
The kernel density is proportional to the number of data points close to a given point.

The kernel density is analogous to a histogram, but it provides more detailed information about the distribution of the data.

The smoothing kernel $K(x)$ is a symmetric function which decreases with the distance x .

The kernel density $d(r)$ at a point r is equal to the sum over the kernel function $K(x)$:

$$d(r) = \sum_{j=1}^n K(r - r_j)$$



```
> library(rutils) ## Load package rutils
> # Calculate VTI percentage returns
> retp <- rutils::etfenv$returns$VTI
> retp <- drop(coredata(na.omit(retp)))
> nrows <- NROW(retp)
> # Mean and standard deviation of returns
> c(mean(retp), sd(retp))
> # Calculate the smoothing bandwidth as the MAD of returns 10 poi
> retp <- sort(retp)
> bwidth <- 10*mad(rutils::diffit(retp, lagg=10))
> # Calculate the kernel density using a loop
> dens1 <- sapply(1:nrows, function(it) {
+   sum(dnorm(retp-retp[it], sd=bwidth))
+ })/nrows ## end supply
```

```
> # Plot the kernel density
> madv <- mad(retp)
> plot(retp, dens1, xlim=c(-5*madv, 5*madv),
+       t="l", col="blue", lwd=3,
+       xlab="returns", ylab="density",
+       main="Density of VTI Returns")
```

Kernel Density Using the Function density()

The function `density()` calculates a kernel estimate of the probability density for a sample of data.

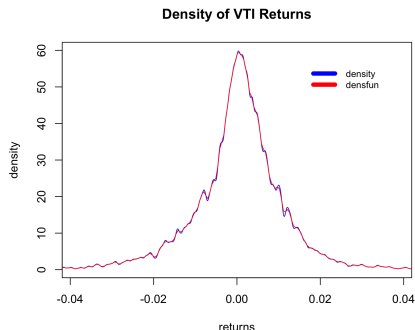
The parameter *smoothing bandwidth* is the standard deviation of the smoothing kernel $K(x)$.

The function `density()` returns a vector of densities at equally spaced points, not for the original data points.

The function `approx()` interpolates a vector of data into another vector.

The function `lines()` draws a line through specified points.

```
> # Calculate the kernel density using density()
> densv <- density(retp, bw=bandwidth)
> NROW(densv$y)
> plot(densv, xlim=c(-5*madv, 5*madv),
+      xlab="returns", ylab="density",
+      col="blue", lwd=3, main="Density of VTI Returns")
> # Interpolate the densv vector into returns
> densv <- approx(densv$x, densv$y, xout=retp)
> all.equal(densv$x, retp)
```



```
> # Plot the two density estimates
> plot(retp, dens1, xlim=c(-5*madv, 5*madv),
+      xlab="returns", ylab="density",
+      t="l", col="blue", lwd=1,
+      main="Density of VTI Returns")
> lines(retp, densv$y, col="red")
> # Add legend
> legend("topright", inset=0.05, cex=0.8, title=NULL,
+      leg=c("density", "densfun"), bty="n", y.intersp=0.4,
+      lwd=6, bg="white", col=c("blue", "red"))
```

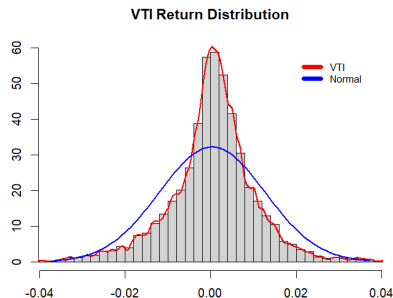
Distribution of Asset Returns

Asset returns are usually not normally distributed and they exhibit *leptokurtosis* (large kurtosis, or fat tails).

The function `hist()` calculates and plots a histogram, and returns its data *invisibly*.

The parameter `breaks` is the number of cells of the histogram.

The function `lines()` draws a line through specified points.



```
> # Plot histogram
> histp <- hist(retp, breaks=100, freq=FALSE,
+   xlim=c(-5*madv, 5*madv), xlab="", ylab="",
+   main="VTI Return Distribution")
> # Draw kernel density of histogram
> lines(densv, col="red", lwd=2)
> # Add density of normal distribution
> curve(expr=dnorm(x, mean=mean(retp), sd=sd(retp)),
+   add=TRUE, lwd=2, col="blue")
> # Add legend
> legend("topright", inset=0.05, cex=0.8, title=NULL,
+   leg=c("VTI", "Normal"), bty="n", y.intersp=0.4,
+   lwd=6, bg="white", col=c("red", "blue"))
```

depr: Distribution of Asset Returns

Asset returns are usually not normally distributed and they exhibit *leptokurtosis* (large kurtosis, or fat tails).

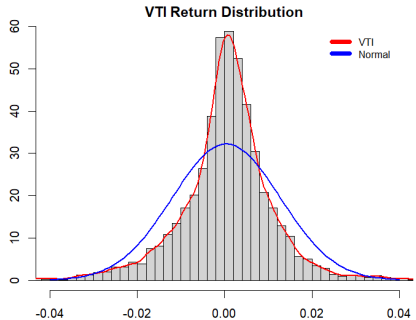
The function `hist()` calculates and plots a histogram, and returns its data *invisibly*.

The parameter `breaks` is the number of cells of the histogram.

The function `density()` calculates a kernel estimate of the probability density for a sample of data.

The function `lines()` draws a line through specified points.

```
> library(rutils) ## Load package rutils
> # Calculate VTI percentage returns
> retp <- na.omit(rutils::etfenv$returns$VTI)
> # Mean and standard deviation of returns
> c(mean(retp), sd(retp))
```



```
> # Plot histogram
> x11(width=6, height=5)
> par(mar=c(1, 1, 1, 1), oma=c(2, 2, 2, 0))
> madv <- mad(retp)
> histp <- hist(retp, breaks=100,
+   main="", xlim=c(-5*madv, 5*madv),
+   xlab="", ylab="", freq=FALSE)
> # Draw kernel density of histogram
> lines(density(retp), col="red", lwd=2)
> # Add density of normal distribution
> curve(expr=dnorm(x, mean=mean(retp), sd=sd(retp)),
+   add=TRUE, type="l", lwd=2, col="blue")
> title(main="VTI Return Distribution", line=0) ## Add title
> # Add legend
> legend("topright", inset=0.05, cex=0.8, title=NULL,
+   leg=c("VTI", "Normal"), bty="n", y.intersp=0.4,
```

The Quantile-Quantile Plot

A *Quantile-Quantile* (*Q-Q*) plot is a plot of points with the same *quantiles*, from two probability distributions.

If the two distributions are similar then all the points in the *Q-Q* plot lie along the diagonal.

The *VTI Q-Q* plot shows that the *VTI* return distribution has fat tails.

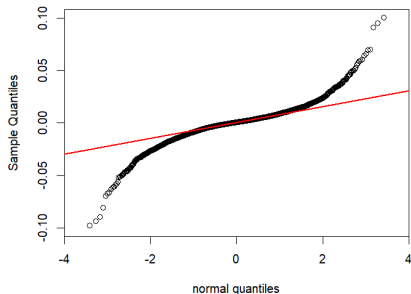
The *p*-value of the *Shapiro-Wilk* test is very close to zero, which shows that the *VTI* returns are very unlikely to be normal.

The function `shapiro.test()` performs the *Shapiro-Wilk* test of normality.

The function `qqnorm()` produces a normal *Q-Q* plot.

The function `qqline()` fits a line to the normal quantiles.

VTI Q-Q Plot



```
> # Create normal Q-Q plot
> qqnorm(retp, ylim=c(-0.1, 0.1), main="VTI Q-Q Plot",
+   xlab="Normal Quantiles")
> # Fit a line to the normal quantiles
> qqline(retp, col="red", lwd=2)
> # Perform Shapiro-Wilk test
> shapiro.test(retp)
```

Boxplots of Distributions of Values

Box-and-whisker plots (*boxplots*) are graphical representations of a distribution of values.

The bottom and top box edges (*hinges*) are equal to the first and third quartiles, and the *box* width is equal to the interquartile range (*IQR*).

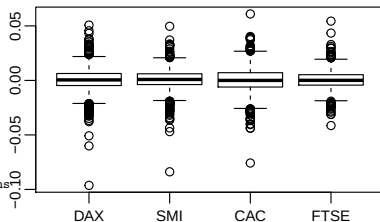
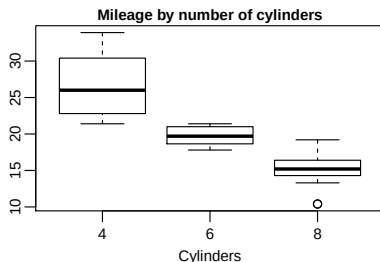
The nominal range is equal to 1.5 times the *IQR* above and below the box *hinges*.

The *whiskers* are dashed vertical lines representing values beyond the first and third quartiles, but within the nominal range.

The *whiskers* end at the last values within the nominal range, while the open circles represent outlier values beyond the nominal range.

The function `boxplot()` has two methods: one for formula objects (for categorical variables), and another for data frames.

```
> # Boxplot method for formula
> boxplot(formula=mpg ~ cyl, data=mtcars,
+   main="Mileage by number of cylinders",
+   xlab="Cylinders", ylab="Miles per gallon")
> # Boxplot method for data frame of EuStockMarkets percentage returns
> boxplot(x=diff(log(EuStockMarkets)))
```



Higher Moments of Asset Returns

The estimators of moments of a probability distribution are given by:

Sample mean: $\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$

Sample variance: $\hat{\sigma}^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2$

With their expected values equal to the population mean and standard deviation:

$\mathbb{E}[\bar{x}] = \mu$ and $\mathbb{E}[\hat{\sigma}] = \sigma$

The sample skewness (third moment):

$$\varsigma = \frac{n}{(n-1)(n-2)} \sum_{i=1}^n \left(\frac{x_i - \bar{x}}{\hat{\sigma}} \right)^3$$

The sample kurtosis (fourth moment):

$$\kappa = \frac{n(n+1)}{(n-1)(n-2)(n-3)} \sum_{i=1}^n \left(\frac{x_i - \bar{x}}{\hat{\sigma}} \right)^4$$

The normal distribution has skewness equal to 0 and kurtosis equal to 3.

Stock returns typically have negative skewness and kurtosis much greater than 3.

```
> # Calculate VTI percentage returns
> retp <- na.omit(rutils::etfenv$returns$VTI)
> # Number of observations
> nrow <- NROW(retp)
> # Mean of VTI returns
> retm <- mean(retp)
> # Standard deviation of VTI returns
> stdev <- sd(retp)
> # Skewness of VTI returns
> nrow/((nrow-1)*(nrow-2))*sum(((retp - retm)/stdev)^3)
> # Kurtosis of VTI returns
> nrow*(nrow+1)/((nrow-1)^3)*sum(((retp - retm)/stdev)^4)
> # Random normal returns
> retp <- rnorm(nrow, sd=stdev)
> # Mean and standard deviation of random normal returns
> retm <- mean(retp)
> stdev <- sd(retp)
> # Skewness of random normal returns
> nrow/((nrow-1)*(nrow-2))*sum(((retp - retm)/stdev)^3)
> # Kurtosis of random normal returns
> nrow*(nrow+1)/((nrow-1)^3)*sum(((retp - retm)/stdev)^4)
```

Functions for Calculating Skew and Kurtosis

R provides an easy way for users to write functions.

The function `calc_skew()` calculates the skew of returns, and `calc_kurt()` calculates the kurtosis.

Functions return the value of the last expression that is evaluated.

```
> # calc_skew() calculates skew of returns
> calc_skew <- function(retp) {
+   retp <- na.omit(retp)
+   sum(((retp - mean(retp))/sd(retp))^3)/NROW(retp)
+ } ## end calc_skew
> # calc_kurt() calculates kurtosis of returns
> calc_kurt <- function(retp) {
+   retp <- na.omit(retp)
+   sum(((retp - mean(retp))/sd(retp))^4)/NROW(retp)
+ } ## end calc_kurt
> # Calculate skew and kurtosis of VTI returns
> calc_skew(retp)
> calc_kurt(retp)
> # calc_mom() calculates the moments of returns
> calc_mom <- function(retp, moment=3) {
+   retp <- na.omit(retp)
+   sum(((retp - mean(retp))/sd(retp))^moment)/NROW(retp)
+ } ## end calc_mom
> # Calculate skew and kurtosis of VTI returns
> calc_mom(retp, moment=3)
> calc_mom(retp, moment=4)
```

Standard Errors of Estimators

Statistical estimators are functions of samples (which are random variables), and therefore are themselves *random variables*.

The *standard error* (SE) of an estimator is defined as its *standard deviation* (not to be confused with the *population standard deviation* of the underlying random variable).

For example, the *standard error* of the estimator of the mean is equal to:

$$\sigma_{\mu} = \frac{\sigma}{\sqrt{n}}$$

Where σ is the *population standard deviation* (which is usually unknown).

The *estimator* of this *standard error* is equal to:

$$SE_{\mu} = \frac{\hat{\sigma}}{\sqrt{n}}$$

where: $\hat{\sigma}^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2$ is the sample standard deviation (the estimator of the population standard deviation).

```
> # Initialize the random number generator
> set.seed(1121, "Mersenne-Twister", sample.kind="Rejection")
> # Sample from Standard Normal Distribution
> nrows <- 1000
> datav <- rnorm(nrows)
> # Sample mean
> mean(datav)
> # Sample standard deviation
> sd(datav)
> # Standard error of sample mean
> sd(datav)/sqrt(nrows)
```

Normal (Gaussian) Probability Distribution

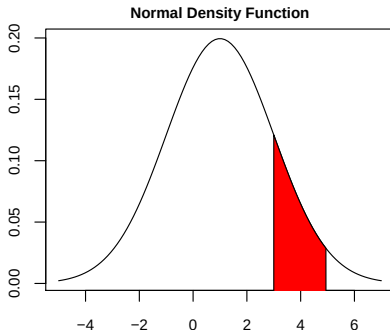
The *Normal (Gaussian)* probability density function is given by:

$$\phi(x, \mu, \sigma) = \frac{e^{-(x-\mu)^2/2\sigma^2}}{\sigma\sqrt{2\pi}}$$

The *Standard Normal* distribution $\phi(0, 1)$ is a special case of the *Normal* $\phi(\mu, \sigma)$ with $\mu = 0$ and $\sigma = 1$.

The function `dnorm()` calculates the *Normal* probability density.

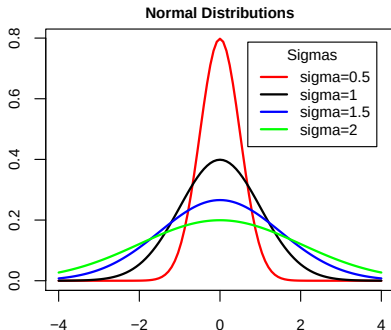
```
> xvar <- seq(-5, 7, length=100)
> yvar <- dnorm(xvar, mean=1.0, sd=2.0)
> plot(xvar, yvar, type="l", lty="solid", xlab="", ylab="")
> title(main="Normal Density Function", line=0.5)
> startp <- 3; endd <- 5 ## Set lower and upper bounds
> # Set polygon base
> subv <- ((xvar >= startp) & (xvar <= endd))
> polygon(c(startp, xvar[subv], endd), ## Draw polygon
+ c(-1, yvar[subv], -1), col="red")
```



Normal (Gaussian) Probability Distributions

Plots of several *Normal* distributions with different values of σ , using the function `curve()` for plotting functions given by their name.

```
> sigmavs <- c(0.5, 1, 1.5, 2) ## Sigma values
> # Create plot colors
> colorv <- c("red", "black", "blue", "green")
> # Create legend labels
> labelv <- paste("sigma", sigmavs, sep="")
> for (it in 1:4) { ## Plot four curves
+   curve(expr=dnorm(x, sd=sigmavs[it]),
+     xlim=c(-4, 4), xlab="", ylab="", lwd=2,
+     col=colorv[it], add=as.logical(it-1))
+ } ## end for
> # Add title
> title(main="Normal Distributions", line=0.5)
> # Add legend
> legend("topright", inset=0.05, title="Sigmas", y.intersp=0.4,
+   labelv, cex=0.8, lwd=2, lty=1, bty="n", col=colorv)
```



Student's t -distribution

Let $z_1, \dots, z_\nu$ be independent standard normal random variables, with sample mean: $\bar{z} = \frac{1}{\nu} \sum_{i=1}^{\nu} z_i$ ($\mathbb{E}[\bar{z}] = \mu$) and sample variance:

$$\hat{\sigma}^2 = \frac{1}{\nu-1} \sum_{i=1}^{\nu} (z_i - \bar{z})^2$$

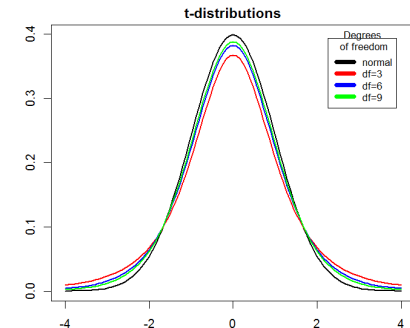
Then the random variable (t -ratio):

$$t = \frac{\bar{z} - \mu}{\hat{\sigma} / \sqrt{\nu}}$$

Follows the t -distribution with ν degrees of freedom, with the probability density function:

$$f(t) = \frac{\Gamma((\nu + 1)/2)}{\sqrt{\pi\nu} \Gamma(\nu/2)} (1 + t^2/\nu)^{-(\nu+1)/2}$$

```
> degf <- c(3, 6, 9) ## Df values
> colorv <- c("black", "red", "blue", "green")
> labelv <- c("normal", paste("df", degf, sep=" "))
> # Plot a Normal probability distribution
> curve(expr=dnorm, xlim=c(-4, 4), xlab="", ylab="", lwd=2)
> for (it in 1:3) { ## Plot three t-distributions
+   curve(expr=dt(x, df=degf[it]), xlab="", ylab="",
+   lwd=2, col=colorv[it+1], add=TRUE)
+ } ## end for
```



```
> # Add title
> title(main="t-distributions", line=0.5)
> # Add legend
> legend("topright", inset=0.05, bty="n", y.intersp=0.4,
+       title="Degrees\n of freedom", labelv,
+       cex=0.8, lwd=6, lty=1, col=colorv)
```

Mixture Models of Returns

Mixture models are produced by randomly sampling data from different distributions.

The mixture of two normal distributions with different variances produces a distribution with *leptokurtosis* (large kurtosis, or fat tails).

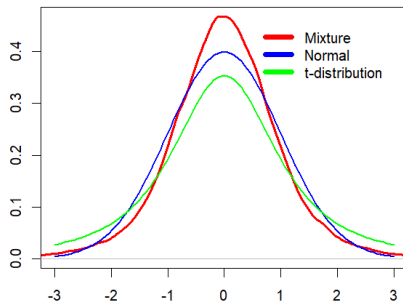
Student's *t-distribution* has fat tails because the sample variance in the denominator of the *t-ratio* is variable.

The time-dependent volatility of asset returns is referred to as *heteroskedasticity*.

Random processes with *heteroskedasticity* can be considered a type of mixture model.

The *heteroskedasticity* produces *leptokurtosis* (large kurtosis, or fat tails).

Mixture of Normal Returns



```
> # Mixture of two normal distributions with sd=1 and sd=2
> nrows <- 1e5
> retp <- c(rnorm(nrows/2), 2*rnorm(nrows/2))
> retp <- (retp-mean(retp))/sd(retp)
> # Kurtosis of normal
> calc_kurt(rnorm(nrows))
> # Kurtosis of mixture
> calc_kurt(retp)
> # Or
> nrows*sum(retp^4)/(nrows-1)^2
```

```
> # Plot the distributions
> plot(density(retp), xlab="", ylab="",
+      main="Mixture of Normal Returns",
+      xlim=c(-3, 3), type="l", lwd=3, col="red")
> curve(expr=dnorm, lwd=2, col="blue", add=TRUE)
> curve(expr=dt(x, df=3), lwd=2, col="green", add=TRUE)
> # Add legend
> legend("topright", inset=0.05, lty=1, lwd=6, bty="n",
+      legend=c("Mixture", "Normal", "t-distribution"), y.intersp=0.4,
+      col=c("red", "blue", "green"))
```

Non-standard Student's *t*-distribution

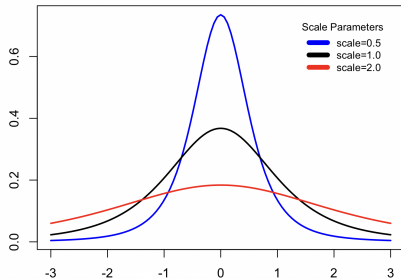
The non-standard Student's *t*-distribution has the probability density function:

$$f(t) = \frac{\Gamma((\nu + 1)/2)}{\sqrt{\pi\nu} \sigma \Gamma(\nu/2)} \left(1 + \left(\frac{t - \mu}{\sigma}\right)^2 / \nu\right)^{-(\nu+1)/2}$$

It has a non-zero mean equal to the location parameter μ , and a standard deviation proportional to the scale parameter σ .

```
> dev.new(width=6, height=5, noRStudioGD=TRUE)
> # x11(width=6, height=5)
> # Define density of non-standard t-distribution
> tdistr <- function(x, dfree, locv=0, scalev=1) {
+   dt((x-locv)/scalev, df=dfree)/scalev
+ } ## end tdistr
> # Or
> tdistr <- function(x, dfree, locv=0, scalev=1) {
+   gamma((dfree+1)/2)/(sqrt(pi*dfree)*gamma(dfree/2)*scalev)*
+   (1+((x-locv)/scalev)^2/dfree)^(-(dfree+1)/2)
+ } ## end tdistr
> # Calculate vector of scale values
> scalev <- c(0.5, 1.0, 2.0)
> colorv <- c("blue", "black", "red")
> labelv <- paste("scale", format(scalev, digits=2), sep="")
> # Plot three t-distributions
> for (it in 1:3) {
+   curve(expr=tdistr(x, dfree=3, scalev=scalev[it]), xlim=c(-3, 3),
+   xlab="", ylab="", lwd=2, col=colorv[it], add=(it>1))
+ } ## end for
```

t-distributions with Different Scale Parameters



```
> # Add title
> title(main="t-distributions with Different Scale Parameters", line=1)
> # Add legend
> legend("topright", inset=0.05, bty="n", title="Scale Parameters",
+       cex=0.8, lwd=6, lty=1, col=colorv, y.intersp=0.4)
```

The *Shapiro-Wilk* Test of Normality

The *Shapiro-Wilk* test is designed to test the *null hypothesis* that a sample: $\{x_1, \dots, x_n\}$ is from a normally distributed population.

The test statistic is equal to:

$$W = \frac{(\sum_{i=1}^n a_i x_{(i)})^2}{\sum_{i=1}^n (x_i - \bar{x})^2}$$

Where the: $\{a_1, \dots, a_n\}$ are proportional to the *order statistics* of random variables from the normal distribution.

$x_{(k)}$ is the *k*-th *order statistic*, and is equal to the *k*-th smallest value in the sample: $\{x_1, \dots, x_n\}$.

The *Shapiro-Wilk* statistic follows its own distribution, and is less than or equal to 1.

The *Shapiro-Wilk* statistic is close to 1 for samples from normal distributions.

The *p*-value for *VTI* returns is extremely small, and we conclude that the *null hypothesis* is FALSE, and the *VTI* returns are not from a normally distributed population.

The *Shapiro-Wilk* test is not reliable for large sample sizes, so it's limited to less than 5000 sample size.

```
> # Calculate VTI percentage returns
> library(rutils)
> retp <- as.numeric(na.omit(rutils::etfenv$returns$VTI))[1:499]
> # Reduce number of output digits
> ndigits <- options(digits=5)
> # Shapiro-Wilk test for normal distribution
> nrows <- NROW(retp)
> shapiro.test(rnorm(nrows))
```

Shapiro-Wilk normality test

```
data:  rnorm(nrows)
W = 0.997, p-value = 0.64
> # Shapiro-Wilk test for VTI returns
> shapiro.test(retp)
```

Shapiro-Wilk normality test

```
data:  retp
W = 0.991, p-value = 0.0029
> # Shapiro-Wilk test for uniform distribution
> shapiro.test(runif(nrows))
```

Shapiro-Wilk normality test

```
data:  runif(nrows)
W = 0.944, p-value = 9.9e-13
> # Restore output digits
> options(digits=ndigits$digits)
```

The Jarque-Bera Test of Normality

The *Jarque-Bera* test is designed to test the *null hypothesis* that a sample: $\{x_1, \dots, x_n\}$ is from a normally distributed population.

The test statistic is equal to:

$$JB = \frac{n}{6} \left(\varsigma^2 + \frac{1}{4} (\kappa - 3)^2 \right)$$

Where the *skewness* and *kurtosis* are defined as:

$$\varsigma = \frac{1}{n} \sum_{i=1}^n \left(\frac{x_i - \bar{x}}{\hat{\sigma}} \right)^3 \quad \kappa = \frac{1}{n} \sum_{i=1}^n \left(\frac{x_i - \bar{x}}{\hat{\sigma}} \right)^4$$

The *Jarque-Bera* statistic asymptotically follows the *chi-squared* distribution with 2 degrees of freedom.

The *Jarque-Bera* statistic is small for samples from normal distributions.

The *p*-value for *VTI* returns is extremely small, and we conclude that the *null hypothesis* is FALSE, and the *VTI* returns are not from a normally distributed population.

```
> library(tseries) ## Load package tseries
> # Jarque-Bera test for normal distribution
> jarque.bera.test(rnorm(nrows))
```

Jarque Bera Test

```
data:  rnorm(nrows)
X-squared = 0.3, df = 2, p-value = 0.8
> # Jarque-Bera test for VTI returns
> jarque.bera.test(retp)
```

Jarque Bera Test

```
data:  retp
X-squared = 22, df = 2, p-value = 2e-05
> # Jarque-Bera test for uniform distribution
> jarque.bera.test(runif(NROW(retp)))
```

Jarque Bera Test

```
data:  runif(NROW(retp))
X-squared = 25, df = 2, p-value = 5e-06
```

The Kolmogorov-Smirnov Test for Probability Distributions

The *Kolmogorov-Smirnov* test *null hypothesis* is that two samples: $\{x_1, \dots, x_n\}$ and $\{y_1, \dots, y_n\}$ were obtained from the same probability distribution.

The *Kolmogorov-Smirnov* statistic depends on the maximum difference between two empirical cumulative distribution functions (cumulative frequencies):

$$D = \sup_i |P(x_i) - P(y_i)|$$

The function `ks.test()` performs the *Kolmogorov-Smirnov* test and returns the statistic and its *p*-value *invisibly*.

The second argument to `ks.test()` can be either a numeric vector of data values, or a name of a cumulative distribution function.

The *Kolmogorov-Smirnov* test can be used as a *goodness of fit* test, to test if a set of observations fits a probability distribution.

```
> # KS test for normal distribution
> kstest <- ks.test(rnorm(100), pnorm)
> kstest$p.value
> # KS test for uniform distribution
> ks.test(runif(100), pnorm)
> # KS test for two shifted normal distributions
> ks.test(rnorm(100), rnorm(100, mean=0.1))
> ks.test(rnorm(100), rnorm(100, mean=1.0))
> # KS test for two different normal distributions
> ks.test(rnorm(100), rnorm(100, sd=2.0))
> # KS test for VTI returns vs normal distribution
> retp <- as.numeric(na.omit(rutils::etfenv$returns$VTI))
> retp <- (retp - mean(retp))/sd(retp)
> ks.test(retp, pnorm)
```

Chi-squared Distribution

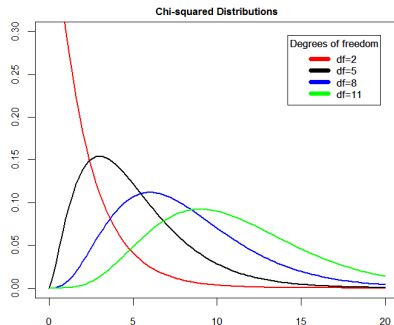
Let $z_1, \dots, z_k$ be independent standard *Normal* random variables.

Then the random variable $X = \sum_{i=1}^k z_i^2$ is distributed according to the *Chi-squared* distribution with k degrees of freedom: $X \sim \chi_k^2$, and its probability density function is given by:

$$f(x) = \frac{x^{k/2-1} e^{-x/2}}{2^{k/2} \Gamma(k/2)}$$

The *Chi-squared* distribution with k degrees of freedom has mean equal to k and variance equal to $2k$.

```
> # Degrees of freedom
> degf <- c(2, 5, 8, 11)
> # Plot four curves in loop
> colorv <- c("red", "black", "blue", "green")
> for (it in 1:4) {
+   curve(expr=dchisq(x, df=degf[it]),
+         xlim=c(0, 20), ylim=c(0, 0.3),
+         xlab="", ylab="", col=colorv[it],
+         lwd=2, add=as.logical(it-1))
+ } ## end for
```



```
> # Add title
> title(main="Chi-squared Distributions", line=0.5)
> # Add legend
> labelv <- paste("df", degf, sep="=")
> legend("topright", inset=0.05, bty="n", y.intersp=0.4,
+       title="Degrees of freedom", labelv,
+       cex=0.8, lwd=6, lty=1, col=colorv)
```

The *Chi-squared* Test for the Goodness of Fit

Goodness of Fit tests are designed to test if a set of observations fits an assumed theoretical probability distribution.

The *Chi-squared* test tests if a frequency of counts fits the specified distribution.

The *Chi-squared* statistic is the sum of squared differences between the observed frequencies o_i and the theoretical frequencies p_i :

$$\chi^2 = N \sum_{i=1}^n \frac{(o_i - p_i)^2}{p_i}$$

Where N is the total number of observations.

The *null hypothesis* is that the observed frequencies are consistent with the theoretical distribution.

The function `chisq.test()` performs the *Chi-squared* test and returns the statistic and its *p*-value *invisibly*.

The parameter `breaks` in the function `hist()` should be chosen large enough to capture the shape of the frequency distribution.

```
> # Observed frequencies from random normal data
> histp <- hist(rnorm(1e3, mean=0), breaks=100, plot=FALSE)
> countsn <- histp$counts
> # Theoretical frequencies
> countst <- rutils::diffit(pnorm(histp$breaks))
> # Perform Chi-squared test for normal data
> chisq.test(x=countsn, p=countst, rescale.p=TRUE, simulate.p.value=TRUE)
> # Return p-value
> chisqtest <- chisq.test(x=countsn, p=countst, rescale.p=TRUE, simulate.p.value=TRUE)
> chisqtest$p.value
> # Observed frequencies from shifted normal data
> histp <- hist(rnorm(1e3, mean=2), breaks=100, plot=FALSE)
> countsn <- histp$counts/sum(histp$counts)
> # Theoretical frequencies
> countst <- rutils::diffit(pnorm(histp$breaks))
> # Perform Chi-squared test for shifted normal data
> chisq.test(x=countsn, p=countst, rescale.p=TRUE, simulate.p.value=TRUE)
> # Calculate histogram of VTI returns
> histp <- hist(retp, breaks=100, plot=FALSE)
> countsn <- histp$counts
> # Calculate cumulative probabilities and then difference them
> countst <- pt((histp$breaks-locv)/scalev, df=2)
> countst <- rutils::diffit(countst)
> # Perform Chi-squared test for VTI returns
> chisq.test(x=countsn, p=countst, rescale.p=TRUE, simulate.p.value=TRUE)
```

The Likelihood Function of Student's *t*-distribution

The non-standard Student's *t*-distribution is:

$$f(t) = \frac{\Gamma((\nu + 1)/2)}{\sqrt{\pi\nu} \sigma \Gamma(\nu/2)} \left(1 + \left(\frac{t - \mu}{\sigma}\right)^2 / \nu\right)^{-(\nu+1)/2}$$

It has non-zero mean equal to the location parameter μ , and a standard deviation proportional to the scale parameter σ .

The negative logarithm of the probability density is equal to:

$$-\log(f(t|\mu, \sigma)) = -\log\left(\frac{\Gamma((\nu + 1)/2)}{\sqrt{\pi\nu} \Gamma(\nu/2)}\right) + \log(\sigma) + \frac{\nu + 1}{2} \log\left(1 + \left(\frac{t - \mu}{\sigma}\right)^2 / \nu\right)$$

The *likelihood* function $\mathcal{L}(\mu, \sigma|\mathbf{t})$ is the product of the individual probability density functions $f(t_i|\mu, \sigma)$:

$$\mathcal{L}(\mu, \sigma|\mathbf{t}) = \prod_{i=1}^n f(t_i|\mu, \sigma)$$

The *likelihood* $\mathcal{L}(\mu, \sigma|\mathbf{t})$ is a function of the model parameters μ and σ , given the vector of the observed values $\mathbf{t}$, under the model's probability distribution $f(t|\mu, \sigma)$:

```
> # Objective function from function dt()
> likefun <- function(par, dfree, datav) {
+   -sum(log(dt(x=(datav-par[1])/par[2], df=dfree)/par[2]))
+ } ## end likefun
> # Demonstrate equivalence with log(dt())
> likefun(c(1, 0.5), 2, 2:5)
> -sum(log(dt(x=2:5-1)/0.5, df=2)/0.5))
> # Objective function is negative log-likelihood
> likefun <- function(par, dfree, datav) {
+   sum(-log(gamma((dfree+1)/2)/(sqrt(pi*dfree)*gamma(dfree/2))) +
+     log(par[2]) + (dfree+1)*2*log(1+((datav-par[1])/par[2])^2/dfree))
+ } ## end likefun
```

The *likelihood* function measures how *likely* are the model parameters μ, σ , given the observed values $\mathbf{t}$.

The *maximum-likelihood* estimate (MLE) of the parameters $\theta = (\mu, \sigma)$ are those that maximize the *likelihood* function:

$$\theta_{MLE} = \arg \max_{\theta} \mathcal{L}(\theta|\mathbf{t})$$

In practice the logarithm of the *likelihood* $\log(\mathcal{L})$ is maximized, instead of the *likelihood* itself.

Fitting Asset Returns into Student's t -distribution

The function `fitdistr()` from package *MASS* fits a univariate distribution to a sample of data, by performing *maximum likelihood* optimization.

The function `fitdistr()` performs a *maximum likelihood* optimization to find the non-standardized Student's t -distribution location and scale parameters.

```
> # Calculate VTI percentage returns
> retp <- as.numeric(na.omit(rutils::etfenv$returns$VTI))
> # Fit VTI returns using MASS::fitdistr()
> fitobj <- MASS::fitdistr(retp, densfun="t", df=3)
> summary(fitobj)
> # Fitted parameters
> fitobj$estimate
> locv <- fitobj$estimate[1]
> scalev <- fitobj$estimate[2]
> locv; scalev
> # Standard errors of parameters
> fitobj$sed
> # Log-likelihood value
> fitobj$value
> # Fit distribution using optim()
> initp <- c(mean=0, scale=0.01) ## Initial parameters
> fitobj <- optim(par=initp,
+   fn=likefun, ## Log-likelihood function
+   datav=retp,
+   dfree=3, ## Degrees of freedom
+   method="L-BFGS-B", ## Quasi-Newton method
+   upper=c(1, 0.1), ## Upper constraint
+   lower=c(-1, 1e-7)) ## Lower constraint
> # Optimal parameters
> locv <- fitobj$par["mean"]
> scalev <- fitobj$par["scale"]
> locv; scalev
```

The Student's t -distribution Fitted to Asset Returns

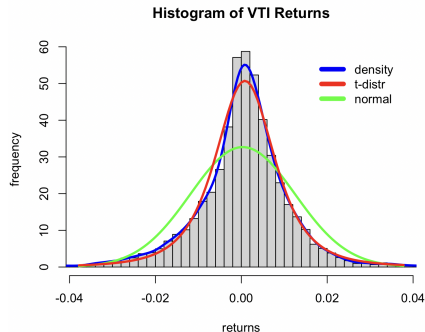
Asset returns typically exhibit *negative skewness* and *large kurtosis* (leptokurtosis), or fat tails.

Stock returns fit the non-standard t -distribution with 3 degrees of freedom quite well.

The function `hist()` calculates and plots a histogram, and returns its data *invisibly*.

The parameter `breaks` is the number of cells of the histogram.

```
> dev.new(width=6, height=5, noRStudioGD=TRUE)
> # x11(width=6, height=5)
> # Plot histogram of VTI returns
> madv <- mad(retp)
> histp <- hist(retp, col="lightgrey",
+   xlab="returns", breaks=100, xlim=c(-5*madv, 5*madv),
+   ylab="frequency", freq=FALSE, main="Histogram of VTI Returns")
> lines(density(retp, adjust=1.5), lwd=3, col="blue")
> # Plot the Normal probability distribution
> curve(expr=dnorm(x, mean=mean(retp),
+   sd=sd(retp)), add=TRUE, lwd=3, col="green")
> # Define non-standard t-distribution
> tdistr <- function(x, dfree, locv=0, scalev=1) {
+   dt((x-locv)/scalev, df=dfree)/scalev
+ } ## end tdistr
> # Plot t-distribution function
> curve(expr=tdistr(x, dfree=3, locv=locv, scalev=scalev), col="red", lwd=3, add=TRUE)
> # Add legend
> legend("topright", inset=0.05, bty="n", y.intersp=0.4,
+   leg=c("density", "t-distr", "normal"),
+   lwd=6, lty=1, col=c("blue", "red", "green"))
```



Goodness of Fit of Student's t -distribution Fitted to Asset Returns

The Q-Q plot illustrates the relative distributions of two samples of data.

The Q-Q plot shows that stock returns fit the non-standard t -distribution with 3 degrees of freedom quite well.

The function `qqplot()` produces a Q-Q plot for two samples of data.

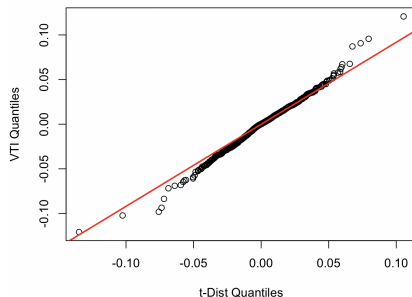
The function `ks.test()` performs the Kolmogorov-Smirnov test for the similarity of two distributions.

The null hypothesis of the Kolmogorov-Smirnov test is that the two samples were obtained from the same probability distribution.

The Kolmogorov-Smirnov test rejects the null hypothesis that stock returns follow closely the non-standard t -distribution with 3 degrees of freedom.

```
> # Calculate sample from non-standard t-distribution with df=3
> datat <- locv + scalev*rt(NROW(retp), df=3)
> # KS test for VTI returns vs t-distribution data
> ks.test(retp, datat)
```

Q-Q plot of VTI Returns vs Student's t -distribution



```
> # Q-Q plot of VTI Returns vs non-standard t-distribution
> qqplot(datat, retp, xlab="t-Dist Quantiles", ylab="VTI Quantiles",
+       main="Q-Q plot of VTI Returns vs Student's t-distribution")
> # Calculate quantiles of the distributions
> probs <- c(0.25, 0.75)
> qrets <- quantile(retp, probs)
> qtdata <- quantile(datat, probs)
> # Calculate slope and plot line connecting quantiles
> slope <- diff(qrets)/diff(qtdata)
> intercept <- qrets[1]-slope*qtdata[1]
> abline(intercept, slope, lwd=2, col="red")
```

Leptokurtosis Fat Tails of Asset Returns

The probability under the *normal* distribution decreases exponentially for large values of x :

$$\phi(x) \propto e^{-x^2/2\sigma^2} \quad (\text{as } |x| \rightarrow \infty)$$

This is because a normal variable can be thought of as the sum of a large number of independent binomial variables of equal size.

So large values are produced only when all the contributing binomial variables are of the same sign, which is very improbable, so it produces extremely low tail probabilities (thin tails),

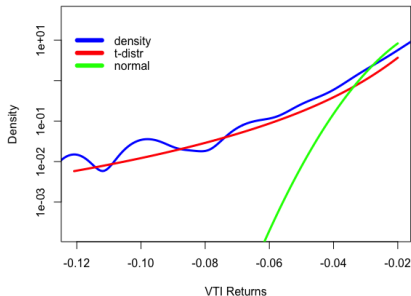
But in reality, the probability of large negative asset returns decreases much slower, as the negative power of the returns (fat tails).

The probability under Student's *t-distribution* decreases as a power for large values of x :

$$f(x) \propto |x|^{-(\nu+1)} \quad (\text{as } |x| \rightarrow \infty)$$

This is because a *t-variable* can be thought of as the sum of normal variables with different volatilities (different sizes).

Fat Left Tail of VTI Returns (density in log scale)



```
> # Plot log density of VTI returns
> plot(density(retp, adjust=4), xlab="VTI Returns", ylab="Density",
+      main="Fat Left Tail of VTI Returns (density in log scale)",
+      type="l", lwd=3, col="blue", xlim=c(min(retp), -0.02), log="y")
> # Plot t-distribution function
> curve(expr=dt((x-locv)/scalev, df=3)/scalev, lwd=3, col="red", add=TRUE)
> # Plot the Normal probability distribution
> curve(expr=dnorm(x, mean=mean(retp), sd=sd(retp)), lwd=3, col="green", add=TRUE)
> # Add legend
> legend("topleft", inset=0.01, bty="n", y.intersp=c(0.25, 0.25, 0.25),
+       legend=c("density", "t-distr", "normal"), y.intersp=0.4,
+       lwd=6, lty=1, col=c("blue", "red", "green"))
```

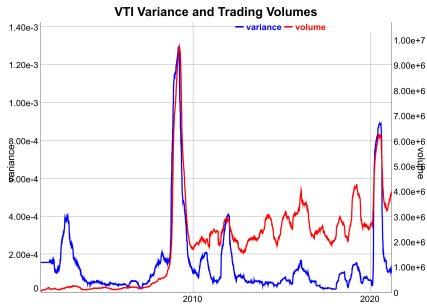
Trading Volumes

The average trading volumes have increased significantly since the 2008 crisis, mostly because of high frequency trading (HFT).

Higher levels of volatility coincide with higher *trading volumes*.

The time-dependent volatility of asset returns (*heteroskedasticity*) produces their fat tails (*leptokurtosis*).

```
> # Calculate VTI returns and trading volumes
> ohlc <- rutils::etfenv$VTI
> closep <- drop(coredata(quantmod::Cl(ohlc)))
> retp <- rutils::diffit(log(closep))
> volumv <- coredata(quantmod::Vo(ohlc))
> # Calculate trailing variance
> lookb <- 121
> varv <- HighFreq::roll_var_ohlc(log(ohlc), method="close", lookb=lookb, scale=FALSE)
> varv[1:lookb, ] <- varv[lookb+1, ]
> # Calculate trailing average volume
> volumr <- HighFreq::roll_sum(volumv, lookb=lookb)/lookb
> # dygraph plot of VTI variance and trading volumes
> datav <- xts::xts(cbind(varv, volumr), zoo::index(ohlc))
> colv <- c("variance", "volume")
> colnames(datav) <- colv
> dygraphs::dygraph(datav, main="VTI Variance and Trading Volumes") %>%
+   dyAxis("y", label=colv[1], independentTicks=TRUE) %>%
+   dyAxis("y2", label=colv[2], independentTicks=TRUE) %>%
+   dySeries(name=colv[1], strokeWidth=2, axis="y", col="blue") %>%
+   dySeries(name=colv[2], strokeWidth=2, axis="y2", col="red") %>%
+   dyLegend(show="always", width=500)
```



Asset Returns in Trading Time

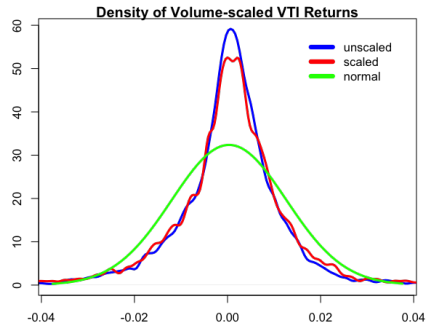
The time-dependent volatility of asset returns (*heteroskedasticity*) produces their fat tails (*leptokurtosis*).

If asset returns were measured at fixed intervals of *trading volumes* (*trading time* instead of clock time), then the volatility would be lower and less time-dependent.

The asset returns can be adjusted to *trading time* by dividing them by the *square root of the trading volumes*, to obtain scaled returns over equal trading volumes.

The scaled returns have a more positive *skewness* and a smaller *kurtosis* than unscaled returns.

```
> # Scale the returns using volume clock to trading time
> retsc <- ifelse(volumv > 0, sqrt(volumv)*retp/sqrt(volumv), 0)
> retsc <- sd(retp)*retsc/sd(retsc)
> # retsc <- ifelse(volumv > 1e4, retp/volumv, 0)
> # Calculate moments of scaled returns
> nrows <- NROW(retp)
> sapply(list(retp=retp, retsc=retsc),
+   function(rets) {sapply(c(skew=3, kurt=4),
+     function(x) sum((rets/sd(rets))^x)/nrows)
+ }) ## end sapply
```



```
> # x11(width=6, height=5)
> dev.new(width=6, height=5, noRStudioGD=TRUE)
> par(mar=c(3, 3, 2, 1), oma=c(1, 1, 1, 1))
> # Plot densities of SPY returns
> madv <- mad(retp)
> # bwidh <- mad(rutils::diffit(retp))
> plot(density(retp, bw=madv/10), xlim=c(-5*madv, 5*madv),
+   lwd=3, mgp=c(2, 1, 0), col="blue",
+   xlab="returns (standardized)", ylab="frequency",
+   main="Density of Volume-scaled VTI Returns")
> lines(density(retsc, bw=madv/10), lwd=3, col="red")
> curve(expr=dnorm(x, mean=mean(retp), sd=sd(retp)),
+   add=TRUE, lwd=3, col="green")
> # Add legend
> legend("topright", inset=0.05, bty="n", y.intersp=0.4
```

draft: Central Limit Theorem

Let $x_1, \dots, x_n$ be independent and identically distributed (i.i.d.) random variables with expected value μ and variance σ^2 , and let $\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$ be their mean.

The random variables x_i don't have to be normally distributed, they only need a finite second moment σ .

The *Central Limit Theorem* states that as $n \rightarrow \infty$, then in the limit, the random variable z :

$$z = \frac{\bar{x} - \mu}{\sigma/\sqrt{n}}$$

Follows the *standard normal* distribution $\phi(0, 1)$.

The *normal* distribution is the limiting distribution of sums of random variables which have a finite second moment.

For example, the sums of random variables with fat tails, which decrease as a power for large values of x :

$$f(x) \propto |x|^{-(\nu+1)} \quad (\text{with } \nu > 1)$$

Tend to the *standard normal* distribution $\phi(0, 1)$.

Package *PerformanceAnalytics* for Risk and Performance Analysis

The package *PerformanceAnalytics* contains functions for calculating risk and performance statistics, such as the variance, skewness, kurtosis, beta, alpha, etc.

The function `data()` loads external data or listv data sets in a package.

`managers` is an *xts* time series containing monthly percentage returns of six asset managers (HAM1 through HAM6), the EDHEC Long-Short Equity hedge fund index, the S&P 500, and US Treasury 10-year bond and 3-month bill total returns.

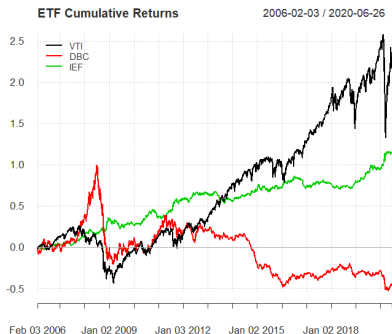
```
> # Load package PerformanceAnalytics
> library(PerformanceAnalytics)
> # Get documentation for package PerformanceAnalytics
> # Get short description
> packageDescription("PerformanceAnalytics")
> # Load help page
> help(package="PerformanceAnalytics")
> # List all objects in PerformanceAnalytics
> ls("package:PerformanceAnalytics")
> # List all datasets in PerformanceAnalytics
> data(package="PerformanceAnalytics")
> # Remove PerformanceAnalytics from search path
> detach("package:PerformanceAnalytics")
```

```
> perfstats <- unclass(data(
+   package="PerformanceAnalytics"))$results[, -(1:2)]
> apply(perfstats, 1, paste, collapse=" - ")
> # Load "managers" data set
> data(managers)
> class(managers)
> dim(managers)
> head(managers, 3)
```

Plots of Cumulative Returns

The function `chart.CumReturns()` from package *PerformanceAnalytics* plots the cumulative returns of a time series of returns.

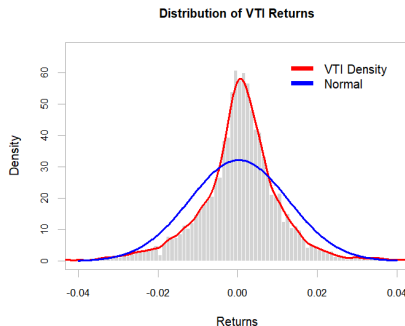
```
> # Load package "PerformanceAnalytics"
> library(PerformanceAnalytics)
> # Calculate ETF returns
> retp <- rutils::etfenv$returns[, c("VTI", "DBC", "IEF")]
> retp <- na.omit(retp)
> # Plot cumulative ETF returns
> x11(width=6, height=5)
> chart.CumReturns(retp, lwd=2, ylab="",
+   legend.loc="topleft", main="ETF Cumulative Returns")
```



The Distribution of Asset Returns

The function `chart.Histogram()` from package *PerformanceAnalytics* plots the histogram (frequency distribution) and the density of returns.

```
> retp <- na.omit(rutils::etfenv$returns$VTI)
> chart.Histogram(retp, xlim=c(-0.04, 0.04),
+   colorset = c("lightgray", "red", "blue"), lwd=3,
+   main=paste("Distribution of", colnames(retp), "Returns"),
+   methods = c("add.density", "add.normal"))
> legend("topright", inset=0.05, bty="n", y.intersp=0.4,
+   leg=c("VTI Density", "Normal"),
+   lwd=6, lty=1, col=c("red", "blue"))
```



Boxplots of Returns

The function `chart.Boxplot()` from package *PerformanceAnalytics* plots a box-and-whisker plot for a distribution of returns.

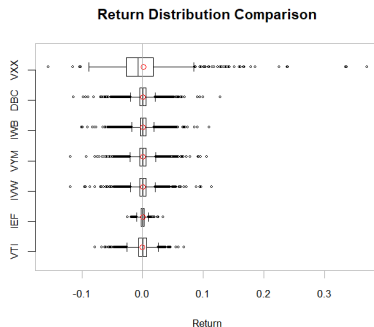
The function `chart.Boxplot()` is a wrapper and calls the function `graphics::boxplot()` to plot the box plots.

A *box plot* (box-and-whisker plot) is a graphical display of a distribution of data:

The *box* represents the upper and lower quartiles,
The vertical lines (whiskers) represent values beyond the quartiles,

Open circles represent values beyond the nominal range (outliers).

```
> retp <- rutils::etfenv$returns[,
+   c("VTI", "IEF", "IVW", "VYM", "IWB", "DBC", "VXX")]
> x11(width=6, height=5)
> chart.Boxplot(names=FALSE, retp)
> par(cex.lab=0.8, cex.axis=0.8)
> axis(side=2, at=(1:NCOL(retp))/7.5-0.05, labels=colnames(retp))
```



The Median Absolute Deviation Estimator of Dispersion

The *Median Absolute Deviation (MAD)* is a nonparametric measure of dispersion (variability), defined using the median instead of the mean:

$$MAD = \text{median}(\text{abs}(x_i - \text{median}(x)))$$

The advantage of *MAD* is that it's always well defined, even for data that has infinite variance.

The *MAD* for normally distributed data is equal to $\Phi^{-1}(0.75) \cdot \hat{\sigma} = 0.6745 \cdot \hat{\sigma}$.

The function `mad()` calculates the *MAD* and divides it by $\Phi^{-1}(0.75)$ to make it comparable to the standard deviation.

For normally distributed data the *MAD* has a larger standard error than the standard deviation.

```
> # Simulate normally distributed data
> nrows <- 1000
> datav <- rnorm(nrows)
> sd(datav)
> mad(datav)
> median(abs(datav - median(datav)))
> median(abs(datav - median(datav)))/qnorm(0.75)
> # Bootstrap of sd and mad estimators
> bootd <- supply(1:10000, function(x) {
+   samplev <- datav[sample.int(nrows, replace=TRUE)]
+   c(sd=sd(samplev), mad=mad(samplev))
+ }) ## end supply
> bootd <- t(bootd)
> # Analyze bootstrapped variance
> head(bootd)
> sum(is.na(bootd))
> # Means and standard errors from bootstrap
> apply(bootd, MARGIN=2, function(x)
+   c(mean=mean(x), stdererror=sd(x)))
> # Parallel bootstrap under Windows
> library(parallel) ## Load package parallel
> ncores <- detectCores() - 1 ## Number of cores
> compclust <- makeCluster(ncores) ## Initialize compute cluster
> bootd <- parLapply(compclust, 1:10000,
+   function(x, datav) {
+     samplev <- datav[sample.int(nrows, replace=TRUE)]
+     c(sd=sd(samplev), mad=mad(samplev))
+   }, datav=datav) ## end parLapply
> # Parallel bootstrap under Mac-OSX or Linux
> bootd <- mclapply(1:10000, function(x) {
+   samplev <- datav[sample.int(nrows, replace=TRUE)]
+   c(sd=sd(samplev), mad=mad(samplev))
+ }, mc.cores=ncores) ## end mclapply
> stopCluster(compclust) ## Stop R processes over cluster
> bootd <- rutils::do_call(rbind, bootd)
> # Means and standard errors from bootstrap
> apply(bootd, MARGIN=2, function(x)
+   c(mean=mean(x), stdererror=sd(x)))
```

The Median Absolute Deviation of Asset Returns

For normally distributed data the *MAD* has a larger standard error than the standard deviation.

But for distributions with fat tails (like asset returns), the standard deviation has a larger standard error than the *MAD*.

The *bootstrap* procedure performs a loop, which naturally lends itself to parallel computing.

The function `makeCluster()` starts running R processes on several CPU cores under *Windows*.

The function `parLapply()` is similar to `lapply()`, and performs loops under *Windows* using parallel computing on several CPU cores.

The R processes started by `makeCluster()` don't inherit any data from the parent R process.

Therefore the required data must be either passed into `parLapply()` via the dots `"..."` argument, or by calling the function `clusterExport()`.

The function `mclapply()` performs loops using parallel computing on several CPU cores under *Mac-OSX* or *Linux*.

The function `stopCluster()` stops the R processes running on several CPU cores.

```
> # Calculate VTI returns
> retp <- na.omit(rutils::etfenv$returns$VTI)
> nrows <- NROW(retp)
> sd(retp)
> mad(retp)
> # Bootstrap of sd and mad estimators
> bootd <- sapply(1:10000, function(x) {
+   samplev <- retp[sample.int(nrows, replace=TRUE)]
+   c(sd=sd(samplev), mad=mad(samplev))
+ }) ## end sapply
> bootd <- t(bootd)
> # Means and standard errors from bootstrap
> 100*apply(bootd, MARGIN=2, function(x)
+   c(mean=mean(x), stdev=sd(x)))
> # Parallel bootstrap under Windows
> library(parallel) ## Load package parallel
> ncores <- detectCores() - 1 ## Number of cores
> compclust <- makeCluster(ncores) ## Initialize compute cluster
> clusterExport(compclust, c("nrows", "returns"))
> bootd <- parLapply(compclust, 1:10000,
+   function(x) {
+     samplev <- retp[sample.int(nrows, replace=TRUE)]
+     c(sd=sd(samplev), mad=mad(samplev))
+   }) ## end parLapply
> # Parallel bootstrap under Mac-OSX or Linux
> bootd <- mclapply(1:10000, function(x) {
+   samplev <- retp[sample.int(nrows, replace=TRUE)]
+   c(sd=sd(samplev), mad=mad(samplev))
+ }, mc.cores=ncores) ## end mclapply
> stopCluster(compclust) ## Stop R processes over cluster
> bootd <- rutils::do_call(rbind, bootd)
> # Means and standard errors from bootstrap
> apply(bootd, MARGIN=2, function(x)
+   c(mean=mean(x), stdev=sd(x)))
```

The Downside Deviation of Asset Returns

Some investors argue that positive returns don't represent risk, only those returns that are less than the target rate of return r_t .

The downside deviation (DD , or semi-deviation) σ_d is equal to the standard deviation of the returns minus the target rate of return r_t .

There are two methods of calculating the DD .

The first method is to sum over the full time series, while setting the positive values to zero:

$$\sigma_d^2 = \frac{1}{n} \sum_{i=1}^n \min(r_i - r_t, 0)^2$$

But this can underestimate the volatility when the excess returns are positive, because it ignores the positive values.

The second method is to sum only over the negative values:

$$\sigma_d^2 = \frac{1}{k} \sum_{(r_i - r_t) < 0}^n (r_i - r_t)^2$$

But this can overestimate the volatility of streaming (live) returns, when the excess returns are positive, because it ignores the positive values.

```
> library(PerformanceAnalytics)
> # Define target rate of return of 50 bps
> targetr <- 0.005
> # Calculate the full downside returns
> retsub <- (retp - targetr)
> retsub <- ifelse(retsub < 0, retsub, 0)
> nrows <- NROW(retsub)
> # Calculate the downside deviation
> all.equal(sqrt(sum(retsub^2)/nrows),
+   drop(DownsideDeviation(retp, MAR=targetr, method="full")))
> # Calculate the subset downside returns
> retsub <- (retp - targetr)
> retsub <- retsub[retsub < 0]
> nrows <- NROW(retsub)
> # Calculate the downside deviation
> all.equal(sqrt(sum(retsub^2)/nrows),
+   drop(DownsideDeviation(retp, MAR=targetr, method="subset")))
```

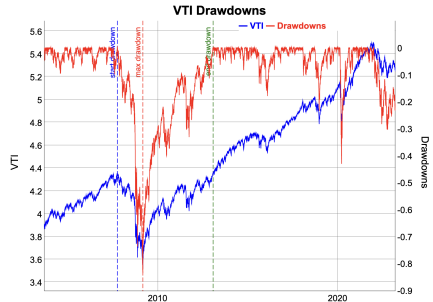
The function `DownsideDeviation()` from package *PerformanceAnalytics* calculates the downside deviation, for either the full time series (`method="full"`) or only for the subseries less than the target rate of return r_t (`method="subset"`).

Drawdown Risk

A *drawdown* is the drop in prices from their historical peak, and is equal to the difference between the prices minus the cumulative maximum of the prices.

Drawdown risk determines the risk of liquidation due to stop loss limits.

```
> # Calculate time series of VTI drawdowns
> closep <- log(quantmod::Cl(rutils::etfenv$VTI))
> drawdns <- (closep - cummax(closep))
> # Extract the date index from the time series closep
> datev <- zoo::index(closep)
> # Calculate the drawdown trough date
> indexm <- which.min(drawdns)
> datem <- datev[indexm]
> # Calculate the drawdown start and end dates
> startd <- max(datev[(datev < datem) & (drawdns == 0)])
> startd <- datev[which(startd==datev)+1] ## Shift ahead by one day
> endd <- min(datev[(datev > datem) & (drawdns == 0)])
> # Calculate the drawdown depth
> maxdd <- drawdns[datem]
> # dygraph plot of VTI drawdowns
> datav <- cbind(closep, drawdns)
> colv <- c("VTI", "Drawdowns")
> colnames(datav) <- colv
> dygraphs::dygraph(datav, main="VTI Drawdowns") %>%
+   dyAxis("y", label=colv[1], independentTicks=TRUE) %>%
+   dyAxis("y2", label=colv[2],
+     valueRange=(1.2*range(drawdns)+0.1), independentTicks=TRUE) %>%
+   dySeries(name=colv[1], axis="y", col="blue") %>%
+   dySeries(name=colv[2], axis="y2", col="red") %>%
+   dyEvent(startd, "start drawdown", col="blue") %>%
+   dyEvent(datem, "max drawdown", col="red") %>%
+   dyEvent(endd, "end drawdown", col="green")
```



```
> # Plot VTI drawdowns using package quantmod
> themev <- chart_theme()
> themev$col$line.col <- c("blue")
> x11(width=6, height=5)
> quantmod::chart_Series(x=closep, name="VTI Drawdowns", theme=themev)
> xval <- match(startd, datev)
> yval <- max(closep)
> abline(v=xval, col="blue")
> text(x=xval, y=0.95*yval, "start drawdown", col="blue", cex=0.9)
> xval <- match(datem, datev)
> abline(v=xval, col="red")
> text(x=xval, y=0.9*yval, "max drawdown", col="red", cex=0.9)
> xval <- match(endd, datev)
> abline(v=xval, col="green")
> text(x=xval, y=0.85*yval, "end drawdown", col="green", cex=0.9)
```

Drawdown Risk Using PerformanceAnalytics::table.Drawdowns()

The function `table.Drawdowns()` from package *PerformanceAnalytics* calculates a data frame of drawdowns.

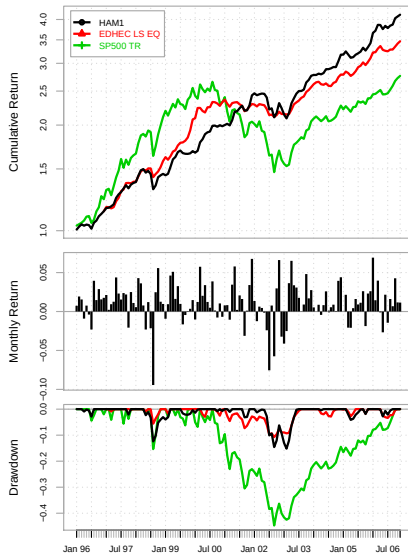
```
> library(xtable)
> library(PerformanceAnalytics)
> closep <- log(quantmod::Cl(rutils::etfenv$VTI))
> retp <- rutils::diffit(closep)
> # Calculate table of VTI drawdowns
> tablev <- PerformanceAnalytics::table.Drawdowns(retp, geometric=FALSE)
> # Convert dates to strings
> tablev <- cbind(sapply(tablev[, 1:3], as.character), tablev[, 4:7])
> # Print table of VTI drawdowns
> print(xtable(tablev), comment=FALSE, size="tiny", include.rownames=FALSE)
```

From	Trough	To	Depth	Length	To Trough	Recovery
2007-10-10	2009-03-09	2012-03-13	-0.57	1115.00	355.00	760.00
2001-06-06	2002-10-09	2004-11-04	-0.45	858.00	336.00	522.00
2020-02-20	2020-03-23	2020-08-12	-0.18	122.00	23.00	99.00
2022-01-04	2022-10-12	2023-12-18	-0.10	492.00	195.00	297.00
2018-09-21	2018-12-24	2019-04-23	-0.10	146.00	65.00	81.00

PerformanceSummary Plots

The function `charts.PerformanceSummary()` from package *PerformanceAnalytics* plots three charts: cumulative returns, return bars, and drawdowns, for time series of returns.

```
> data(managers)
> charts.PerformanceSummary(ham1,
+   main="", lwd=2, ylog=TRUE)
```



The Loss Distribution of Asset Returns

The distribution of returns has a long left tail of negative returns representing the risk of loss.

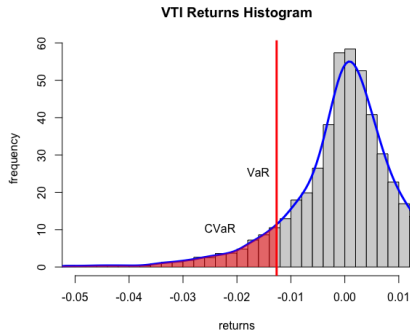
The *Value at Risk* (VaR) is equal to the quantile of returns corresponding to a given confidence level α .

The *Conditional Value at Risk* (CVaR) is equal to the average of negative returns less than the VaR.

The function `hist()` calculates and plots a histogram, and returns its data *invisibly*.

The function `density()` calculates a kernel estimate of the probability density for a sample of data.

```
> # Calculate VTI percentage returns
> retp <- na.omit(rutils::etfenv$returns$VTI)
> confl <- 0.1
> varisk <- quantile(retp, confl)
> cvar <- mean(retp[retp <= varisk])
> # Plot histogram of VTI returns
> x11(width=6, height=5)
> par(mar=c(3, 2, 1, 0), oma=c(0, 0, 0, 0))
> histp <- hist(retp, col="lightgrey",
+   xlab="returns", ylab="frequency", breaks=100,
+   xlim=c(-0.05, 0.01), freq=FALSE, main="VTI Returns Histogram")
> # Calculate density
> densv <- density(retp, adjust=1.5)
```



```
> # Plot density
> lines(densv, lwd=3, col="blue")
> # Plot line for VaR
> abline(v=varisk, col="red", lwd=3)
> text(x=varisk, y=25, labels="VaR", lwd=2, pos=2)
> # Plot polygon shading for CVaR
> text(x=1.5*varisk, y=10, labels="CVaR", lwd=2, pos=2)
> varmax <- -0.06
> rangev <- (densv$x < varisk) & (densv$x > varmax)
> polygon(c(varmax, densv$x[rangev], varisk),
+   c(0, densv$y[rangev], 0), col=rgb(1, 0, 0, 0.5), border=NA)
```

Value at Risk (VaR)

The *Value at Risk* (VaR) is equal to the quantile of returns corresponding to a given confidence level α :

$$\alpha = \int_{-\infty}^{\text{VaR}(\alpha)} f(r) dr$$

Where $f(r)$ is the probability density (distribution) of returns.

At a high confidence level, the value of VaR is subject to estimation error, and various numerical methods are used to approximate it.

The function `quantile()` calculates the sample quantiles. It uses interpolation to improve the accuracy. Information about the different interpolation methods can be found by typing `?quantile`.

A simpler but less accurate way of calculating the quantile is by sorting and selecting the data closest to the quantile.

The function `VaR()` from package *PerformanceAnalytics* calculates the *Value at Risk* using several different methods.

```
> # Calculate VTI percentage returns
> retp <- na.omit(rutils::etfenv$returns$VTI)
> nrow <- NROW(retp)
> confl <- 0.05
> # Calculate VaR approximately by sorting
> sortv <- sort(as.numeric(retp))
> cutoff <- round(confl*nrow)
> varisk <- sortv[cutoff]
> # Calculate VaR as quantile
> varisk <- quantile(retp, probs=confl)
> # PerformanceAnalytics VaR
> PerformanceAnalytics::VaR(retp, p=(1-confl), method="historical")
> all.equal(unname(varisk),
+   as.numeric(PerformanceAnalytics::VaR(retp,
+     p=(1-confl), method="historical")))
```

Conditional Value at Risk (CVaR)

The *Conditional Value at Risk* (CVaR) is equal to the average of negative returns less than the VaR:

$$\text{CVaR} = \frac{1}{\alpha} \int_0^{\alpha} \text{VaR}(p) dp$$

The *Conditional Value at Risk* is also called the *Expected Shortfall* (ES), or the *Expected Tail Loss* (ETL).

The function `ETL()` from package *PerformanceAnalytics* calculates the *Conditional Value at Risk* using several different methods.

```
> # Calculate VaR as quantile
> varisk <- quantile(retp, confl)
> # Calculate CVaR as expected loss
> cvar <- mean(retp[retp <= varisk])
> # PerformanceAnalytics VaR
> PerformanceAnalytics::ETL(retp, p=(1-confl), method="historical")
> all.equal(unname(cvar),
+   as.numeric(PerformanceAnalytics::ETL(retp,
+     p=(1-confl), method="historical")))
```

Risk and Return Statistics

The function `table.Stats()` from package *PerformanceAnalytics* calculates a data frame of risk and return statistics of the return distributions.

```
> # Calculate the risk-return statistics
> riskstats <-
+   PerformanceAnalytics::table.Stats(rutils::etfenv$returns)
> class(riskstats)
> # Transpose the data frame
> riskstats <- as.data.frame(t(riskstats))
> # Add Name column
> riskstats$Name <- rownames(riskstats)
> # Add Sharpe ratio column
> riskstats$"Arithmetic Mean" <-
+   sapply(rutils::etfenv$returns, mean, na.rm=TRUE)
> riskstats$Sharpe <-
+   sqrt(252)*riskstats$"Arithmetic Mean"/riskstats$Stdev
> # Sort on Sharpe ratio
> riskstats <- riskstats[order(riskstats$Sharpe, decreasing=TRUE), ]
```

	Sharpe	Skewness	Kurtosis
USMV	0.827	-0.861	21.41
QUAL	0.727	-0.410	13.00
MTUM	0.720	-0.541	11.36
AIEQ	0.701	0.386	14.76
GLD	0.590	-0.300	5.79
SPY	0.540	-0.248	11.38
VLUE	0.538	-0.878	16.69
IEF	0.510	0.048	2.46
VTI	0.465	-0.330	11.08
VTV	0.459	-0.659	14.15
VYM	0.453	-0.665	14.93
XLV	0.441	0.048	10.05
XLP	0.418	-0.125	8.66
IWB	0.414	-0.356	10.26
XLI	0.414	-0.359	7.76
IVW	0.414	-0.245	8.59
XLY	0.412	-0.311	6.66
XLU	0.397	-0.020	11.61
IWD	0.378	-0.482	12.88
IWF	0.370	-0.584	29.45
QQQ	0.366	-0.010	6.67
IVE	0.364	-0.470	10.43
XLK	0.358	0.088	6.82
EEM	0.327	0.022	16.04
XLB	0.323	-0.352	5.52
TLT	0.266	-0.021	3.38
XLE	0.259	-0.553	12.57
VNQ	0.250	-0.536	18.61
VEU	0.211	-0.503	12.16
XLF	0.202	-0.132	14.56
SVXY	0.152	-18.317	695.40
DBC	0.029	-0.488	3.34
USO	-0.290	-1.106	13.89
VXX	-1.101	1.119	6.07

Investor Risk and Return Preferences

Investors typically prefer larger *odd moments* of the return distribution (mean, skewness), and smaller *even moments* (variance, kurtosis).

But positive skewness is often associated with lower returns, which can be observed in the *VIX* volatility ETFs, *VXX* and *SVXY*.

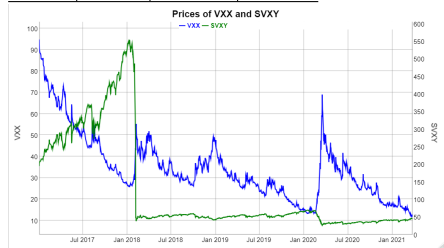
The *VXX* ETF is long the *VIX* index (effectively long an option), so it has positive skewness and small kurtosis, but negative returns (it's short market risk).

Since the *VXX* is effectively long an option, it pays option premiums so it has negative returns most of the time, with isolated periods of positive returns when markets drop.

The *SVXY* ETF is short the *VIX* index, so it has negative skewness and large kurtosis, but positive returns (it's long market risk).

Since the *SVXY* is effectively short an option, it earns option premiums so it has positive returns most of the time, but it suffers sharp losses when markets drop.

	Sharpe	Skewness	Kurtosis
VXX	-1.101	1.12	6.07
SVXY	0.152	-18.32	695.40



```
> # dygraph plot of VXX versus SVXY
> pricev <- na.omit(rutils::etfenv$prices[, c("VXX", "SVXY")])
> pricev <- pricev["2017/"]
> colv <- c("VXX", "SVXY")
> colnames(pricev) <- colv
> dygraphs::dygraph(pricev, main="Prices of VXX and SVXY") %>%
+   dyAxis("y", label=colv[1], independentTicks=TRUE) %>%
+   dyAxis("y2", label=colv[2], independentTicks=TRUE) %>%
+   dySeries(name=colv[1], axis="y", strokeWidth=2, col="blue") %>%
+   dySeries(name=colv[2], axis="y2", strokeWidth=2, col="green") %>%
+   dyLegend(show="always", width=300) %>% dyLegend(show="always", width=300)
+   dyLegend(show="always", width=300)
```

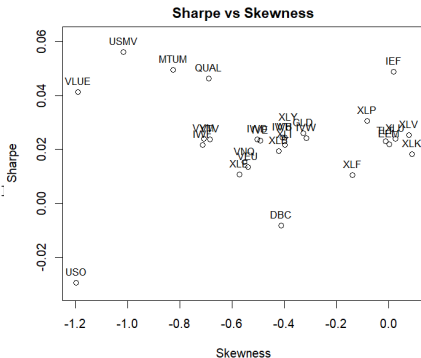
Skewness and Return Tradeoff

Similarly to the VXX and SVXY, for most other ETFs positive skewness is often associated with lower returns.

Some of the exceptions are bond ETFs (like *IEF*), which have both non-negative skewness and positive returns.

Another exception are commodity ETFs (like *USO* oil), which have both negative skewness and negative returns.

```
> # Remove VIX volatility ETF data
> riskstats <- riskstats[~match(c("VIX", "SVXY"), riskstats$Name), ]
> # Plot scatterplot of Sharpe vs Skewness
> plot(Sharpe ~ Skewness, data=riskstats,
+      ylim=1.1*range(riskstats$Sharpe),
+      main="Sharpe vs Skewness")
> # Add labels
> text(x=riskstats$Skewness, y=riskstats$Sharpe,
+      labels=riskstats$Name, pos=3, cex=0.8)
> # Plot scatterplot of Kurtosis vs Skewness
> x11(width=6, height=5)
> par(mar=c(4, 4, 2, 1), oma=c(0, 0, 0, 0))
> plot(Kurtosis ~ Skewness, data=riskstats,
+      ylim=c(1, max(riskstats$Kurtosis)),
+      main="Kurtosis vs Skewness")
> # Add labels
> text(x=riskstats$Skewness, y=riskstats$Kurtosis,
+      labels=riskstats$Name, pos=1, cex=0.5)
```



draft: Skewness and Return Tradeoff for ETFs and Stocks

The ETFs or stocks can be sorted on their skewness to create high_skew and low_skew cohorts.

But the high_skew cohort has better returns than the low_skew cohort - contrary to the thesis that assets with positive skewness produce lower returns than those with a negative skewness.

The low and high volatility cohorts have very similar returns, contrary to expectations. So do the low and high kurtosis cohorts.

```
> ### Below is for ETFs
> # Sort on Sharpe ratio
> riskstats <- riskstats[order(riskstats$Skewness, decreasing=TRUE),]
> # Select high skew and low skew ETFs
> cutoff <- (NROW(riskstats) %/% 2)
> high_skew <- riskstats$Name[1:cutoff]
> low_skew <- riskstats$Name[(cutoff+1):NROW(riskstats)]
> # Calculate returns and log prices
> retp <- rutils::etfenv$returns
> retp <- zoo::na.locf(retp, na.rm=FALSE)
> retp[is.na(retp)] <- 0
> sum(is.na(retp))
> high_skew <- rowMeans(retp[, high_skew])
> low_skew <- rowMeans(retp[, low_skew])
> wealthv <- cbind(high_skew, low_skew)
> wealthv <- xts::xts(wealthv, zoo::index(retp))
> wealthv <- cumsum(wealthv)
> # dygraph plot of high skew and low skew ETFs
> colv <- colnames(wealthv)
> dygraphs::dygraph(wealthv, main="Log Wealth of Low and High Skew ETFs")
+   dyAxis("y", label=colv[1], independentTicks=TRUE) %>%
+   dyAxis("y2", label=colv[2], independentTicks=TRUE) %>%
+   dySeries(name=colv[1], axis="y", strokeWidth=2, col="blue") %>%
+   dySeries(name=colv[2], axis="y2", strokeWidth=2, col="green") %>%
+   dyLegend(show="always", width=300)
>
> ### Below is for S&P500 constituent stocks
> # calc_mom() calculates the moments of returns
> calc_mom <- function(retp, moment=3) {
+   retp <- na.omit(retp)
+   sum((((retp - mean(retp))/sd(retp))~moment)/NROW(retp)
+ } ## end calc_mom
> # Calculate skew and kurtosis of VTI returns
> calc_mom(retp, moment=3)
> calc_mom(retp, moment=4)
> # Load the S&P500 constituent stock returns
```

Risk-adjusted Return Measures

The *Sharpe ratio* S_r is equal to the excess returns (in excess of the risk-free rate r_f) divided by the standard deviation σ of the returns:

$$S_r = \frac{E[r - r_f]}{\sigma}$$

The *Sortino ratio* S_{Or} is equal to the excess returns divided by the *downside deviation* σ_d (standard deviation of returns that are less than a target rate of return r_t):

$$S_{Or} = \frac{E[r - r_t]}{\sigma_d}$$

The *Calmar ratio* C_r is equal to the excess returns divided by the *maximum drawdown* DD of the returns:

$$C_r = \frac{E[r - r_f]}{DD}$$

The *Dowd ratio* D_r is equal to the excess returns divided by the *Value at Risk* (VaR) of the returns:

$$D_r = \frac{E[r - r_f]}{VaR}$$

The *Conditional Dowd ratio* D_{c_r} is equal to the excess returns divided by the *Conditional Value at Risk* (CVaR) of the returns:

$$D_{c_r} = \frac{E[r - r_f]}{CVaR}$$

```
> library(PerformanceAnalytics)
> retp <- rutils::etfenv$returns[, c("VTI", "IEF")]
> retp <- na.omit(retp)
> # Calculate the Sharpe ratio
> confl <- 0.05
> PerformanceAnalytics::SharpeRatio(retp, p=(1-confl),
+   method="historical")
> # Calculate the Sortino ratio
> PerformanceAnalytics::SortinoRatio(retp)
> # Calculate the Calmar ratio
> PerformanceAnalytics::CalmarRatio(retp)
> # Calculate the Dowd ratio
> PerformanceAnalytics::SharpeRatio(retp, FUN="VaR",
+   p=(1-confl), method="historical")
> # Calculate the Dowd ratio from scratch
> varisk <- sapply(retp, quantile, probs=confl)
> ~sapply(retp, mean)/varisk
> # Calculate the Conditional Dowd ratio
> PerformanceAnalytics::SharpeRatio(retp, FUN="ES",
+   p=(1-confl), method="historical")
> # Calculate the Conditional Dowd ratio from scratch
> cvar <- sapply(retp, function(x) {
+   mean(x[x < quantile(x, confl)])
+ })
> ~sapply(retp, mean)/cvar
```

Risk and Return of Stocks Over Longer Holding Periods

Stocks held over longer holding periods often have higher risk-adjusted returns than over shorter holding periods - provided the long-term stock returns are positive.

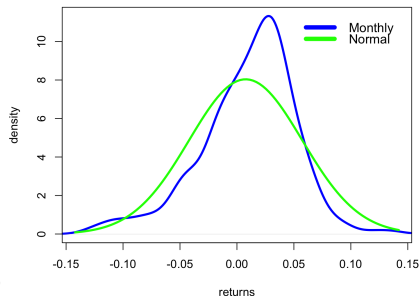
This is because returns are proportional to the holding period, while risk is proportional to the square root of the holding period.

Therefore investors with longer holding periods may choose to own a higher percentage of stocks than bonds.

The skewness of monthly returns is larger than for daily returns, but their kurtosis and tail risks are lower.

```
> # Calculate VTI daily log returns
> pricev <- log(drop(coredata(na.omit(rutils::etfenv$prices$VTI))))
> retp <- rutils::diffit(pricev)
> nrow <- NROW(retp)
> # Calculate VTI monthly log returns
> holdp <- 22 ## Holding period in days
> pricecm <- pricev[rutils::calc_endpoints(pricev, holdp)]
> retm <- rutils::diffit(pricecm)
> retm <- retm[-1] ## Drop the first zero return
> # Calculate the mean, standard deviation, skewness, and kurtosis
> datav <- list(retp, retm)
> names(datav) <- c("Daily", "Monthly")
> do.call(cbind, lapply(datav, function(x) {
+   ## Standardize the returns
+   meanv <- mean(x); stdev <- sd(x); x <- (x - meanv)/stdev
+   c(mean=meanv, stdev=stdev, skew=mean(x^3), kurt=mean(x^4))
+ }))) ## end lapply
```

Distribution of Monthly VTI Returns



```
> # Calculate the Sharpe and Dowd ratios
> do.call(cbind, lapply(datav, function(x) {
+   meanv <- mean(x); stdev <- sd(x)
+   varisk <- unname(quantile(x, probs=0.02))
+   cvar <- mean(x[x < varisk])
+   ## Annualize the ratios
+   sqrt(252*NROW(x)/nrow)*mean(x)/c(Sharpe=stdev, Dowd=-varisk, D
+ }))) ## end lapply
> # Plot the density of monthly returns
> plot(density(retm), t="l", lwd=3, col="blue",
+   xlab="returns", ylab="density", xlim=c(-4*mad(retm), 4*mad(re
+   main="Distribution of Monthly VTI Returns")
> curve(expr=dnorm(x, mean=mean(retm), sd=sd(retm)), col="green", l
> legend("topright", legend=c("Monthly", "Normal"), y.intersp=0.4, c
+ inset=0.0, bg="white", lty=1, lwd=6, col=c("blue", "green"), bty
```

draft: Feature Engineering

Feature engineering derives predictive data elements (features) from a large input data set.

Feature engineering reduces the size of the input data set to a smaller set of features with the highest predictive power.

The predictive features are then used as inputs into machine learning models.

Out-of-sample features only depend on past data, while *in-sample* features depend both on past and future data.

A *trailing* data filter is an example of an *out-of-sample* feature.

A *centered* data filter is an example of an *in-sample* feature.

Out-of-sample features are used in forecasting and scrubbing real-time (live) data.

In-sample features are used in data labeling and scrubbing historical data.

Principal Component Analysis (PCA) is a *dimension reduction* technique used in multivariate feature engineering.

Feature engineering can be developed using *domain knowledge* and analytical techniques.

Some features indicate trend, for example the moving

```
> # Number of flights from each airport
> dtable[, .N, by=origin]
> # Same, but add names to output
> dtable[, .(flights=.N), by=(airport=origin)]
> # Number of AA flights from each airport
> dtable[carrier=="AA", .(flights=.N),
+   by=(airport=origin)]
> # Number of flights from each airport and airline
> dtable[, .(flights=.N),
+   by=(airport=origin, airline=carrier)]
> # Average aircraft_delay
> dtable[, mean(aircraft_delay)]
> # Average aircraft_delay from JFK
> dtable[origin=="JFK", mean(aircraft_delay)]
> # Average aircraft_delay from each airport
> dtable[, .(delay=mean(aircraft_delay)),
+   by=(airport=origin)]
> # Average and max delays from each airport and month
> dtable[, .(mean_delay=mean(aircraft_delay), max_delay=max(aircraft_delay)),
+   by=(airport=origin, month=month)]
> # Average and max delays from each airport and month
> dtable[, .(mean_delay=mean(aircraft_delay), max_delay=max(aircraft_delay)),
+   keyby=(airport=origin, month=month)]
```

Convolution Filtering of Time Series

The function `filter()` applies a trailing linear filter to time series, vectors, and matrices, and returns a time series of class "ts".

The function `filter()` with the argument `method="convolution"` calculates the *convolution* of the vector r_t with the filter φ_i :

$$f_t = \varphi_1 r_{t-1} + \varphi_2 r_{t-2} + \dots + \varphi_p r_{t-p}$$

Where f_t is the filtered output vector, and φ_i are the filter coefficients.

`filter()` is very fast because it calculates the filter by calling compiled C++ functions.

`filter()` with `method="convolution"` calls the function `stats::C_cfilter()` to calculate the *convolution*.

Convolution filtering can be performed even faster by directly calling the compiled function `stats::C_cfilter()`.

The function `HighFreq::roll_conv()` calculates the *weighted* trailing sum (convolution) even faster than `stats::C_cfilter()`.

```
> # Extract log VTI prices
> ohlc <- log(rutils::etfenv$VTI)
> closep <- quantmod::Cl(ohlc)
> colnames(closep) <- "VTI"
> nrow <- NROW(closep)
> # Inspect the R code of the function filter()
> filter
> # Calculate EMA weights
> lookb <- 21
> weightv <- exp(-0.1*1:lookb)
> weightv <- weightv/sum(weightv)
> # Calculate convolution using filter()
> pricef <- filter(closep, filter=weightv, method="convolution", sides=1)
> # filter() returns time series of class "ts"
> class(pricef)
> # Get information about C_cfilter()
> getAnywhere(C_cfilter)
> # Filter using C_cfilter() over past values (sides=1).
> priceff <- .Call(stats::C_cfilter, closep, filter=weightv,
+               sides=1, circular=FALSE)
> all.equal(as.numeric(pricef), priceff, check.attributes=FALSE)
> # Calculate EMA prices using HighFreq::roll_conv()
> pricecpp <- HighFreq::roll_conv(closep, weightv=weightv)
> all.equal(priceff[-(1:lookb)], as.numeric(pricecpp)[-(1:lookb)])
> # Benchmark speed of trailing calculations
> library(microbenchmark)
> summary(microbenchmark(
+   filter=filter(closep, filter=weightv, method="convolution", sides=1),
+   priceff=.Call(stats::C_cfilter, closep, filter=weightv, sides=1),
+   rcpp=HighFreq::roll_conv(closep, weightv=weightv),
+   times=10)[, c(1, 4, 5)])
```

Recursive Filtering of Time Series

The function `filter()` with `method="recursive"` calls the function `stats::C_rfilter()` to calculate the *recursive filter* as follows:

$$r_t = \varphi_1 r_{t-1} + \varphi_2 r_{t-2} + \dots + \varphi_p r_{t-p} + \xi_t$$

Where r_t is the filtered output vector, φ_i are the filter coefficients, and ξ_t are standard normal innovations.

The *recursive* filter describes an $AR(p)$ process, which is a special case of an *ARIMA* process.

The function `HighFreq::sim_arima()` is very fast because it's written using the C++ *Armadillo* numerical library.

```
> # Simulate AR process using filter()
> nrow <- NROW(closep)
> # Calculate AR coefficients and innovations
> coeff <- matrix(weightv)/4
> ncoeff <- NROW(coeff)
> innov <- matrix(rnorm(nrow))
> arimav <- filter(x=innov, filter=coeff, method="recursive")
> # Get information about C_rfilter()
> getAnywhere(C_rfilter)
> # Filter using C_rfilter() compiled C++ function directly
> arimafast <- .Call(stats::C_rfilter, innov, coeff,
+                   double(ncoeff + nrow))
> all.equal(as.numeric(arimav), arimafast[-(1:ncoeff)],
+           check.attributes=FALSE)
> # Filter using C++ code
> arimacpp <- HighFreq::sim_ar(coeff, innov)
> all.equal(arimafast[-(1:ncoeff)], drop(arimacpp))
> # Benchmark speed of the three methods
> summary(microbenchmark(
+   filter=filter(x=innov, filter=coeff, method="recursive"),
+   priceff=.Call(stats::C_rfilter, innov, coeff, double(ncoeff + nrow)),
+   Rcpp=HighFreq::sim_ar(coeff, innov)
+ ), times=10)[, c(1, 4, 5)]
```

Data Smoothing and The Bias-Variance Tradeoff

Filtering through an averaging filter produces data *smoothing*.

Smoothing real-time data with a trailing filter reduces its *variance* but it increases its *bias* because it introduces a time lag.

Smoothing historical data with a centered filter reduces its *variance* but it introduces *data snooping*.

In engineering, smoothing is called a *low-pass filter*, since it eliminates high frequency signals, and it passes through low frequency signals.

```
> # Extract log VTI prices
> closep <- log(na.omit(rutils::etfenv$prices$VTI))
> retp <- rutils::diffit(closep)
> nrow <- NROW(closep)
> # Calculate EMA prices using HighFreq::run_mean()
> pricema <- HighFreq::run_mean(closep, lambda=0.9)
> # Combine prices with EMA prices
> pricev <- cbind(closep, pricema)
> colnames(pricev)[2] <- "VTI EMA"
> # Calculate standard deviations of returns
> sapply(rutils::diffit(pricev), sd)
```



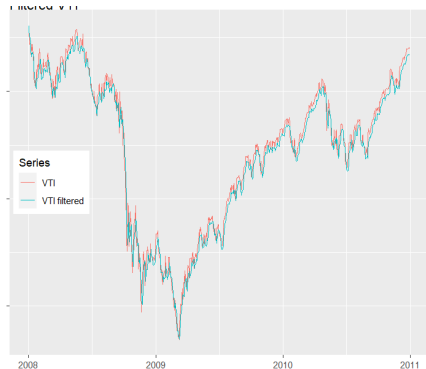
```
> # Plot dygraph
> dygraphs::dygraph(pricev["2009"], main="VTI Prices and EMA Prices")
+ dyOptions(colors=c("blue", "red"), strokeWidth=2) %>%
+ dyLegend(show="always", width=300)
```

depr: Plotting Filtered Time Series

```

> library(rutils) ## Load package rutils
> library(ggplot2) ## Load ggplot2
> library(gridExtra) ## Load gridExtra
> # Coerce to zoo and merge the time series
> pricef <- cbind(closep, pricef)
> colnames(pricef) <- c("VTI", "VTI filtered")
> # Plot ggplot2
> autoplot(pricef["2008/2010"],
+   main="Filtered VTI", facets=NULL) + ## end autoplot
+ xlab("") + ylab("") +
+ theme( ## Modify plot theme
+   legend.position=c(0.1, 0.5),
+   plot.title=element_text(vjust=-2.0),
+   plot.margin=unit(c(-0.5, 0.0, -0.5, 0.0), "cm"),
+   plot.background=element_blank(),
+   axis.text.y=element_blank()
+ ) ## end theme
> # end ggplot2

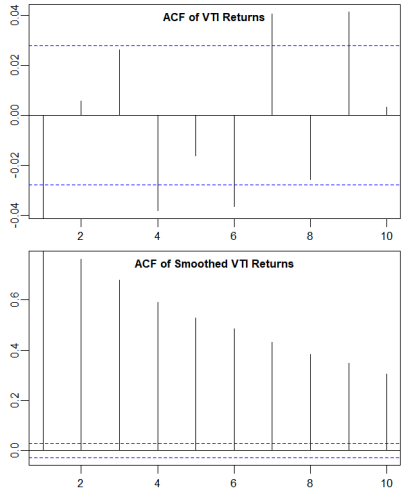
```



Autocorrelations of Smoothed Time Series

Smoothing a time series of prices produces autocorrelations of their returns.

```
> # Calculate VTI log returns
> retf <- rutils::diffit(closef)
> # Open plot window
> x11(width=6, height=7)
> # Set plot parameters
> par(oma=c(1, 1, 0, 1), mar=c(1, 1, 1, 1), mgp=c(0, 0.5, 0),
+     cex.lab=0.8, cex.axis=0.8, cex.main=0.8, cex.sub=0.5)
> # Set two plot panels
> par(mfrow=c(2,1))
> # Plot ACF of VTI returns
> rutils::plot_acf(retf[, 1], lag=10, xlab="")
> title(main="ACF of VTI Returns", line=-1)
> # Plot ACF of smoothed VTI returns
> rutils::plot_acf(retf[, 2], lag=10, xlab="")
> title(main="ACF of Smoothed VTI Returns", line=-1)
```



RSI Technical Indicator

The *Relative Strength Indicator (RSI)* is the ratio of the average *EMA* gains divided by the sum of the *EMA* gains plus the *EMA* losses:

$$RSI_t = \frac{100 * gain_t}{gain_t + loss_t}$$

The *RSI* oscillates between the values of 0 and 100. If the past gains are small then the *RSI* is close to 0. If the past gains are large then the *RSI* is close to 100.

The *EMA* gains and losses are calculated recursively using the decay factor λ as follows:

$$gain_t = \lambda gain_{t-1} + (1 - \lambda) r_t^+$$

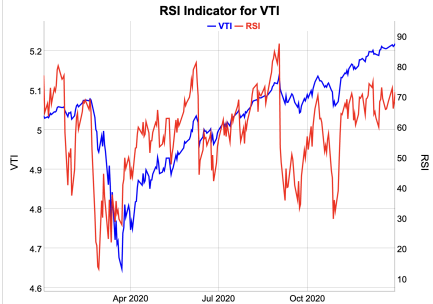
$$loss_t = \lambda loss_{t-1} + (1 - \lambda) r_t^-$$

Where r_t^+ is the gain at time t and r_t^- is the loss.

The gains and losses are non-negative: $r_t^+ = \max(r_t, 0)$ and $r_t^- = \max(-r_t, 0)$.

The *RSI* is often used as a rich or cheap price indicator, depending on its value relative to some threshold levels.

For example, if $RSI > 80$ then the prices may be considered to be overbought (rich - too high). And if $RSI < 20$ then they may be considered to be oversold (cheap - too low).



```
> # Calculate the EMA gains and losses
> lambdaf <- 0.9
> gainm <- HighFreq::run_mean(ifelse(retp > 0, retp, 0), lambdaf)
> lossm <- HighFreq::run_mean(ifelse(retp < 0, -retp, 0), lambdaf)
> # Calculate the RSI indicator
> rsii <- 100 * gainm/(gainm + lossm)
> # Plot dygraph of the RSI indicator
> datav <- cbind(closep, rsii)
> colnames(datav)[2] <- "RSI"
> colv <- colnames(datav)
> dygraphs::dygraph(datav["2020"], main="RSI Indicator for VTI") %>%
+   dyAxis("y", label=colv[1], independentTicks=TRUE) %>%
+   dyAxis("y2", label=colv[2], independentTicks=TRUE) %>%
+   dySeries(name=colv[1], axis="y", strokeWidth=2, col="blue") %>%
+   dySeries(name=colv[2], axis="y2", strokeWidth=2, col="red") %>%
+   dyLegend(show="always", width=300)
```

EMA Price Technical Indicator

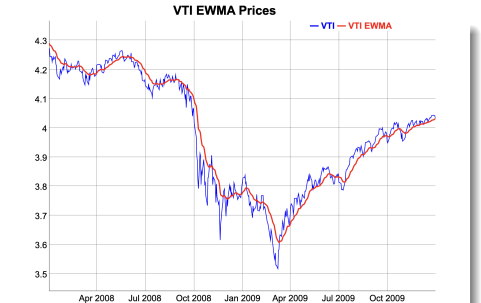
The *Exponential Moving Average Price (EMA)* is defined as the weighted average of prices over a trailing interval:

$$p_t^{EMA} = (1 - \lambda) \sum_{j=0}^{\infty} \lambda^j p_{t-j}$$

The decay factor λ determines the rate of decay of the *EMA* weights, with smaller values of λ producing faster decay, giving more weight to recent prices, and vice versa.

The function `HighFreq::roll_wsum()` calculates the convolution of a time series with a vector of weights.

```
> # Extract log VTI prices
> ohlc <- rutils::etfenv$VTI
> datev <- zoo::index(ohlc)
> closep <- log(quantmod::Cl(ohlc))
> colnames(closep) <- "VTI"
> nrow <- NROW(closep)
> # Calculate EMA weights
> lookb <- 111
> lambdaf <- 0.9
> weightv <- lambdaf^(1:lookb)
> weightv <- weightv/sum(weightv)
> # Calculate EMA prices as the convolution
> pricama <- HighFreq::roll_sumw(closep, weightv=weightv)
> pricev <- cbind(closep, pricama)
> colnames(pricev) <- c("VTI", "VTI EMA")
```



```
> # Dygraphs plot with custom line colors
> colv <- colnames(pricev)
> dygraphs::dygraph(pricev["2008/2009"], main="VTI EMA Prices") %>%
+   dySeries(name=colv[1], strokeWidth=1, col="blue") %>%
+   dySeries(name=colv[2], strokeWidth=2, col="red") %>%
+   dyLegend(show="always", width=300)
> # Standard plot of EMA prices with custom line colors
> x11(width=6, height=5)
> themev <- chart_theme()
> colorv <- c("blue", "red")
> themev$col$line.col <- colors
> quantmod::chart_Series(pricev["2008/2009"], theme=themev,
+   lwd=2, name="VTI EMA Prices")
> legend("topleft", legend=colnames(pricev), y.intersp=0.4,
+   inset=0.1, bg="white", lty=1, lwd=6, cex=0.8,
+   col=themev$col$line.col, bty="n")
```

Recursive EMA Price Indicator

The Exponential Moving Average *EMA* prices can be calculated recursively as follows:

$$p_t^{EMA} = \lambda p_{t-1}^{EMA} + (1 - \lambda) p_t$$

The decay factor λ determines the rate of decay of the *EMA* weights, with smaller values of λ producing faster decay, giving more weight to recent prices, and vice versa.

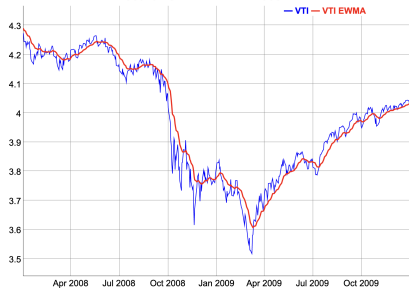
The recursive *EMA* prices are slightly different from those calculated as a convolution, because the convolution uses a fixed look-back interval.

The compiled C++ function `stats::C_rfilter()` calculates the *EMA* prices recursively.

The function `HighFreq::run_mean()` calculates the *EMA* prices recursively using the C++ *Armadillo* numerical library.

```
> # Calculate EMA prices recursively using C++ code
> emar <- .Call(stats::C_rfilter, closep, lambdaf, c(as.numeric(c(
> # Or R code
> # emar <- filter(closep, filter=lambdaf, init=as.numeric(closep[
> emar <- (1-lambdaf)*emar
> # Calculate EMA prices recursively using RcppArmadillo C++
> pricema <- HighFreq::run_mean(closep, lambda=lambdaf)
> all.equal(drop(pricema), emar)
> # Compare the speed of C++ code with RcppArmadillo C++
> library(microbenchmark)
> summary(microbenchmark(
+   filtercpp=HighFreq::run_mean(closep, lambda=lambdaf),
+   rfilter=.Call(stats::C_rfilter, closep, lambdaf, c(as.numeric(
+   times=10)))[, c(1, 4, 5)]
```

Recursive VTI EWMA Prices



```
> # Dygraphs plot with custom line colors
> pricev <- cbind(closep, pricema)
> colnames(pricev) <- c("VTI", "VTI EMA")
> colv <- colnames(pricev)
> dygraphs::dygraph(pricev["2008/2009"], main="Recursive VTI EMA Prices")
+   dySeries(name=colv[1], strokeWidth=1, col="blue") %>%
+   dySeries(name=colv[2], strokeWidth=2, col="red") %>%
+   dyLegend(show="always", width=300)
> # Standard plot of EMA prices with custom line colors
> x11(width=6, height=5)
> themev <- chart_theme()
> colorv <- c("blue", "red")
> themev$col$line.col <- colors
> quantmod::chart_Series(pricev["2008/2009"], theme=themev,
+   lwd=2, name="VTI EMA Prices")
> legend("topleft", legend=colnames(pricev),
+   inset=0.1, bg="white", lty=1, lwd=6, cex=0.8,
+   col=themev$col$line.col, bty="n")
```

Volume-Weighted Average Price Indicator

The Volume-Weighted Average Price (VWAP) is defined as the sum of prices multiplied by trading volumes, divided by the sum of volumes:

$$p_t^{VWAP} = \frac{\sum_{j=0}^n v_{t-j} p_{t-j}}{\sum_{j=0}^n v_{t-j}}$$

The VWAP applies more weight to prices with higher trading volumes, which allows it to react more quickly to recent market volatility.

The drawback of the VWAP indicator is that it applies large weights to prices far in the past.

The VWAP is often used as a technical indicator in trend following strategies.



```
> # Calculate log OHLC prices and volumes
> volumv <- quantmod::Vo(ohlc)
> colnames(volumv) <- "Volume"
> nrow <- NROW(closep)
> # Calculate the VWAP prices
> lookb <- 21
> vwap <- HighFreq::roll_sum(closep, lookb=lookb, weightv=volumv)
> colnames(vwap) <- "VWAP"
> pricev <- cbind(closep, vwap)
```

```
> # Dygraphs plot with custom line colors
> colorv <- c("blue", "red")
> dygraphs::dygraph(pricev["2008/2009"], main="VTI VWAP Prices") %>%
+   dyOptions(colors=colorv, strokeWidth=2) %>%
+   dyLegend(show="always", width=300)
> # Plot VWAP prices with custom line colors
> x11(width=6, height=5)
> themev <- chart_theme()
> themev$col$line.col <- colors
> quantmod::chart_Series(pricev["2008/2009"], theme=themev,
+   lwd=2, name="VTI VWAP Prices")
> legend("bottomright", legend=colnames(pricev),
+   inset=0.1, bg="white", lty=1, lwd=6, cex=0.8,
+   col=themev$col$line.col, bty="n")
```

Recursive VWAP Price Indicator

The *VWAP* prices p^{VWAP} can also be calculated recursively as the ratio of the mean volume weighted prices $\bar{v}p$ divided by the mean trading volumes $\bar{v}$:

$$\bar{v}_t = \lambda \bar{v}_{t-1} + (1 - \lambda) v_t$$

$$\bar{v}p_t = \lambda \bar{v}p_{t-1} + (1 - \lambda) v_t p_t$$

$$p^{VWAP} = \frac{\bar{v}p}{\bar{v}}$$

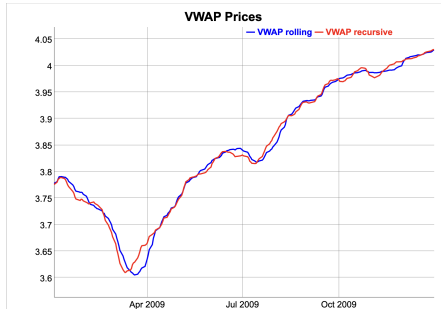
The recursive *VWAP* prices are slightly different from those calculated as a convolution, because the convolution uses a fixed look-back interval.

The advantage of the recursive *VWAP* indicator is that it gradually "forgets" about large trading volumes far in the past.

The recursive formula is also much faster to calculate because it doesn't require a buffer of past data.

The compiled C++ function `stats::C_rfilter()` calculates the trailing weighted values recursively.

The function `HighFreq::run_mean()` also calculates the trailing weighted values recursively.



```
> # Calculate VWAP prices recursively using C++ code
> lambdaf <- 0.9
> volumer <- .Call(stats::C_rfilter, volumv, lambdaf, c(as.numeric(volumv)))
> pricer <- .Call(stats::C_rfilter, volumv*closep, lambdaf, c(as.numeric(closep)))
> vwapr <- pricer/volumer
> # Calculate VWAP prices recursively using RcppArmadillo C++
> vwapc <- HighFreq::run_mean(closep, lambda=lambdaf, weightv=volumv)
> all.equal(vwapr, drop(vwapc))
> # Dygraphs plot the VWAP prices
> pricev <- xts(cbind(vwap, vwapr), zoo::index(ohlc))
> colnames(pricev) <- c("VWAP trailing", "VWAP recursive")
> dygraphs::dygraph(pricev["2008/2009"], main="VWAP Prices") %>%
+   dyOptions(colors=c("blue", "red"), strokeWidth=2) %>%
+   dyLegend(show="always", width=300)
```

Smooth Asset Returns

Asset returns are calculated by filtering prices through a *differencing* filter.

The simplest *differencing* filter is the filter with coefficients $(1, -1)$: $r_t = p_t - p_{t-1}$.

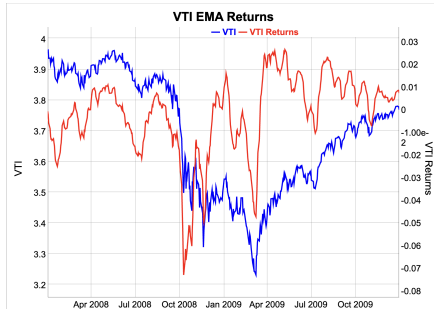
Differencing is a *high-pass filter*, since it eliminates low frequency signals, and it passes through high frequency signals.

An alternative measure of returns is the difference between two moving averages of prices:

$$r_t = p_t^{\text{fast}} - p_t^{\text{slow}}$$

The difference between moving averages is a *mid-pass filter*, since it eliminates both low and high frequency signals, and it passes through medium frequency signals.

```
> # Calculate fast and slow EMA prices
> lambdaf <- 0.8 ## Fast EMA
> emaf <- HighFreq::run_mean(closep, lambda=lambdaf)
> lambdas <- 0.9 ## Slow EMAs
> emas <- HighFreq::run_mean(closep, lambda=lambdas)
```



```
> # Calculate VTI prices
> emad <- (emaf - emas)
> pricev <- cbind(closep, emad)
> symboln <- "VTI"
> colnames(pricev) <- c(symboln, paste(symboln, "Returns"))
> # Plot dygraph of VTI Returns
> colv <- colnames(pricev)
> dygraphs::dygraph(pricev["2008/2009"], main=paste(symboln, "EMA R
+ dyAxis("y", label=colv[1], independentTicks=TRUE) %>%
+ dyAxis("y2", label=colv[2], independentTicks=TRUE) %>%
+ dySeries(name=colv[1], axis="y", strokeWidth=2, col="blue") %>%
+ dySeries(name=colv[2], axis="y2", strokeWidth=2, col="red") %>%
+ dyLegend(show="always", width=300)
```

The MACD Indicator

The Moving average convergence/divergence (*MACD*) indicator consists of three time series: the *MACD series*, the *signal series*, and the *divergence series*.

The *MACD series* is equal to the difference between the fast EMA of the prices minus the slow EMA:

$$MACD = p_{fast} - p_{slow}$$

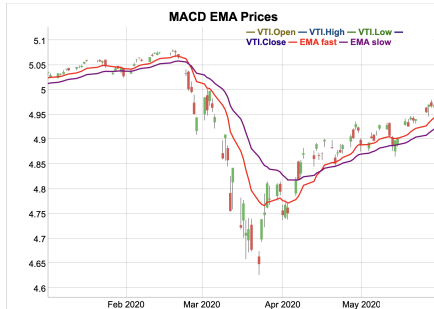
The *MACD* indicator can be used to determine the direction of price movements.

The *MACD series* represents the slope of the prices - the direction of the price movement. A positive *MACD* indicates increasing prices, while a negative *MACD* indicates decreasing prices.

The decay factor λ determines the rate of decay of the *EMA* weights, and it's often expressed in terms of the number of periods (days) n :

$$\lambda = 1 - \frac{2}{n+1} = \frac{n-1}{n+1}$$

A larger number of days means a larger decay factor λ , which gives more weight to the past prices and less weight to the recent prices.



```
> # Calculate fast and slow EMA prices
> lambdaf <- 1 - 2/(1+12) ## 12-day fast EMA
> lambdas <- 1 - 2/(1+26) ## 26-day slow EMA
> emaf <- HighFreq::run_mean(closep, lambda=lambdaf)
> emas <- HighFreq::run_mean(closep, lambda=lambdas)
> # Plot dygraph of the fast and slow EMA prices
> pricev <- cbind(log(ohlc[, 1:4]), emaf, emas)["2020-01/2020-05"]
> colnames(pricev)[5:6] <- c("EMA fast", "EMA slow")
> colv <- colnames(pricev)
> dygraphs::dygraph(pricev, main="MACD EMA Prices") %>%
+   dygraphs::dyCandlestick() %>%
+   dySeries(name=colv[5], strokeWidth=2, col="red") %>%
+   dySeries(name=colv[6], strokeWidth=2, col="purple") %>%
+   dyLegend(show="always", width=300)
```

The Signal and Divergence of the MACD Indicator

The *MACD signal series* is equal to the EMA of the *MACD series*:

$$MACD_{sig} = EMA(MACD)$$

The *signal series* is a smoother version of the slope of the prices. But it also has a larger time lag.

The *MACD divergence series* is equal to the difference between the *MACD series* minus the *signal series*:

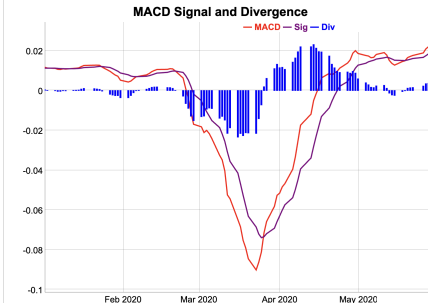
$$MACD_{div} = MACD - MACD_{sig}$$

The sign of the *divergence series* indicates the direction of the price movement.

The *divergence series* is quicker identify the direction of the price movement than the *signal series*.

But it also has more noise than the *signal series*, which produces more false signals.

The number of days for the decay factors λ are often chosen to be 12 days for the fast decay, 26 days for the slow decay, and 9 days for the signal decay,



```
> # Calculate the MACD series
> macds <- emaf - emas
> # Calculate the signal series
> lambdasig <- 1 - 2/(1+9) ## 9-day fast EMA
> macdsig <- HighFreq::run_mean(macds, lambda=lambdasig)
> # Calculate the divergence series
> macddiv <- macds - macdsig
> # Plot dygraph of the signal and divergence series
> pricev <- cbind(macds, macdsig, macddiv)
> pricev <- xts::xts(pricev, datev)
> colnames(pricev) <- c("MACD", "Sig", "Div")
> colv <- colnames(pricev)
> dygraphs::dygraph(pricev["2020-01/2020-05"], main="MACD Signal and
+   dyAxis("y", label=colv[1], independentTicks=TRUE) %>%
+   dyAxis("y2", label=colv[2], independentTicks=TRUE) %>%
+   dySeries(name=colv[1], axis="y", strokeWidth=2, col="red") %>%
+   dySeries(name=colv[2], axis="y", strokeWidth=2, col="purple") %>%
+   dyBarSeries(name=colv[3], axis="y2", col="blue") %>%
```

Fractional Asset Returns

The fractional returns provide a tradeoff between simple returns (which are range-bound but with no memory) and prices (which have memory but are not range-bound).

The lag operator L applies a lag (time shift) to a time series: $L(p_t) = p_{t-1}$.

The simple returns can then be expressed as equal to the returns operator $(1 - L)$ applied to the prices: $r_t = (1 - L)p_t$.

The simple returns can be generalized to the fractional returns by raising the returns operator to some power $\delta < 1$:

$$r_t = (1 - L)^\delta p_t =$$

$$p_t - \delta L p_t + \frac{\delta(\delta-1)}{2!} L^2 p_t - \frac{\delta(\delta-1)(\delta-2)}{3!} L^3 p_t + \dots =$$

$$p_t - \delta p_{t-1} + \frac{\delta(\delta-1)}{2!} p_{t-2} - \frac{\delta(\delta-1)(\delta-2)}{3!} p_{t-3} + \dots$$

```
> # Calculate the fractional weights
> lookb <- 21
> deltav <- 0.1
> weightv <- (deltav - 0:(lookb-2)) / 1:(lookb-1)
> weightv <- (-1)^(1:(lookb-1))*cumprod(weightv)
> weightv <- c(1, weightv)
> weightv <- (weightv - mean(weightv))
```



```
> # Calculate the fractional VTI returns
> retf <- HighFreq::roll_conv(closep, weightv=weightv)
> pricev <- cbind(closep, retf)
> symboln <- "VTI"
> colnames(pricev) <- c(symboln, paste(symboln, "Returns"))
> # Plot dygraph of VTI Returns
> colv <- colnames(pricev)
> dygraphs::dygraph(pricev["2008-08/2009-08"], main=paste(symboln,
+ dyAxis("y", label=colv[1], independentTicks=TRUE) %>%
+ dyAxis("y2", label=colv[2], independentTicks=TRUE) %>%
+ dySeries(name=colv[1], axis="y", strokeWidth=2, col="blue") %>%
+ dySeries(name=colv[2], axis="y2", strokeWidth=2, col="red") %>%
+ dyLegend(show="always", width=300))
```

Augmented Dickey-Fuller Test for Asset Returns

The cumulative sum of a given process is called its *integrated* process.

For example, asset prices follow an *integrated* process with respect to asset returns: $p_t = \sum_{i=1}^t r_i$.

Integrated processes typically have a *unit root* (they have unlimited range), even if their underlying difference process does not have a *unit root* (has limited range).

Asset returns don't have a *unit root* (they have limited range) while prices have a *unit root* (they have unlimited range).

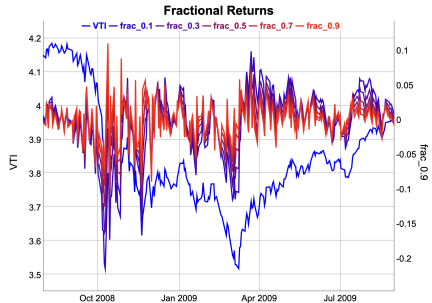
The *Augmented Dickey-Fuller ADF* test is designed to test the *null hypothesis* that a time series has a *unit root*.

```
> # Perform ADF test for prices
> tseries::adf.test(closep)
> # Perform ADF test for returns
> tseries::adf.test(retp)
```

Augmented Dickey-Fuller Test for Fractional Returns

The fractional returns for exponent values close to zero $\delta \approx 0$ resemble the asset price, while for values close to one $\delta \approx 1$ they resemble the standard returns.

```
> # Calculate fractional VTI returns
> deltav <- 0.1*c(1, 3, 5, 7, 9)
> retfrac <- lapply(deltav, function(deltav) {
+   weightv <- (deltav - 0:(lookb-2)) / 1:(lookb-1)
+   weightv <- c(1, (-1)^(1:(lookb-1))*cumprod(weightv))
+   weightv <- (weightv - mean(weightv))
+   HighFreq::roll_conv(closep, weightv=weightv)
+ }) ## end lapply
> retfrac <- do.call(cbind, retfrac)
> retfrac <- cbind(closep, retfrac)
> colnames(retfrac) <- c("VTI", paste0("frac_", deltav))
> # Calculate ADF test statistics
> adfstats <- sapply(retfrac, function(x)
+   suppressWarnings(tseries::adf.test(x)$statistic)
+ ) ## end sapply
> names(adfstats) <- colnames(retfrac)
```



```
> # Plot dygraph of fractional VTI returns
> colorv <- colorRampPalette(c("blue", "red"))(NCOL(retfrac))
> colv <- colnames(retfrac)
> dyplot <- dygraphs::dygraph(retfrac["2008-08/2009-08"], main="Fra
+   dyAxis("y", label=colv[1], independentTicks=TRUE) %>%
+   dySeries(name=colv[1], axis="y", strokeWidth=2, col=colorv[1])
> for (i in 2:NROW(colv))
+   dyplot <- dyplot %>%
+   dyAxis("y2", label=colv[i], independentTicks=TRUE) %>%
+   dySeries(name=colv[i], axis="y2", strokeWidth=2, col=colorv[i])
> dyplot <- dyplot %>% dyLegend(width=300)
> dyplot
```

Trading Volume Z-Scores

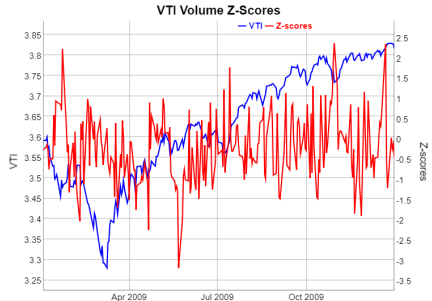
The trailing *volume z-score* is equal to the volume v_t minus the trailing average volumes $\bar{v}_t$ divided by the volatility of the volumes σ_t :

$$z_t = \frac{v_t - \bar{v}_t}{\sigma_t}$$

Trading volumes are typically higher when prices drop and they are also positively correlated with the return volatility.

The volume z-scores represent the first derivative (slope) of the volumes, since the volume level is subtracted.

The volume z-scores are positively skewed because returns are negatively skewed.



```
> # Calculate volume z-scores
> volumv <- quantmod::Vo(ohlc)
> lookb <- 21
> volumean <- HighFreq::roll_mean(volumv, lookb=lookb)
> volumsd <- sqrt(HighFreq::roll_var(rutils::diffit(volumv), lookb=
> volumsd[1] <- 0
> volumz <- ifelse(volumsd > 0, (volumv - volumean)/volumsd, 0)
> # Plot histogram of volume z-scores
> hist(volumz, breaks=1e2)
```

```
> # Plot dygraph of volume z-scores of VTI prices
> pricev <- cbind(closep, volumz)
> colnames(pricev) <- c("VTI", "Z-Scores")
> colv <- colnames(pricev)
> dygraphs::dygraph(pricev["2008/2009"], main="VTI Volume Z-Scores",
+ dyAxis("y", label=colv[1], independentTicks=TRUE) %>%
+ dyAxis("y2", label=colv[2], independentTicks=TRUE) %>%
+ dySeries(name=colv[1], axis="y", strokeWidth=2, col="blue") %>%
+ dySeries(name=colv[2], axis="y2", strokeWidth=2, col="red") %>%
+ dyLegend(show="always", width=300)
```

Volatility Z-Scores

The *true range* is the difference between low and high prices is a proxy for the spot volatility in a bar of data.

The *volatility z-score* is equal to the spot volatility v_t minus the trailing average volatility $\bar{v}_t$ divided by the standard deviation of the volatility σ_t :

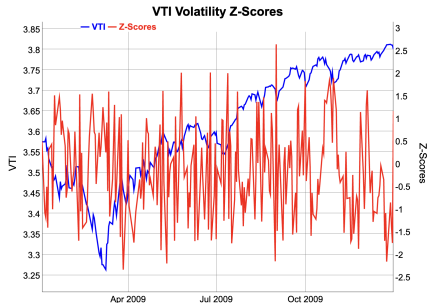
$$z_t = \frac{v_t - \bar{v}_t}{\sigma_t}$$

Volatility is typically higher when prices drop and it's also positively correlated with the trading volumes.

The volatility z-scores represent the first derivative (slope) of the volatilities, since the volatility level is subtracted.

The volatility z-scores are positively skewed because returns are negatively skewed.

```
# Calculate volatility (true range) z-scores
> volv <- log(quantmod::Hi(ohlc) - quantmod::Lo(ohlc))
> lookb <- 21
> volatm <- HighFreq::roll_mean(volv, lookb=lookb)
> volv <- (volv - volatm)
> volatsd <- sqrt(HighFreq::roll_var(rutils::diffit(volv), lookb=1)
> volatsd[1] <- 0
> volatz <- ifelse(volatsd > 0, volv/volatsd, 0)
> # Plot histogram of the volatility z-scores
> hist(volatz, breaks=1e2)
```



```
> # Plot scatterplot of volume and volatility z-scores
> plot(as.numeric(volatz), as.numeric(volumz),
+      xlab="volatility z-score", ylab="volume z-score")
> regmod <- lm(volatz ~ volumz)
> abline(regmod, col="red", lwd=3)
> # Plot dygraph of VTI volatility z-scores
> pricev <- cbind(closep, volatz)
> colnames(pricev) <- c("VTI", "Z-Scores")
> colv <- colnames(pricev)
> dygraphs::dygraph(pricev["2008/2009"], main="VTI Volatility Z-Scores")
+   dyAxis("y", label=colv[1], independentTicks=TRUE) %>%
+   dyAxis("y2", label=colv[2], independentTicks=TRUE) %>%
+   dySeries(name=colv[1], axis="y", strokeWidth=2, col="blue") %>%
+   dySeries(name=colv[2], axis="y2", strokeWidth=2, col="red") %>%
+   dyLegend(show="always", width=300)
```

Trailing Volatility Z-Scores

The *volatility z-score* can also be defined as the difference between the fast v_t^f minus the slow v_t^s trailing volatilities, divided by the standard deviation of the volatility σ_t :

$$z_t = \frac{v_t^f - v_t^s}{\sigma_t}$$

The function `HighFreq::run_var()` calculates the trailing mean and variance of the returns r_t , by recursively weighting the past variance estimates σ_{t-1}^2 , with the squared differences of the returns minus their trailing means $(r_t - \bar{r}_t)^2$, using the decay factor λ :

$$\bar{r}_t = \lambda \bar{r}_{t-1} + (1 - \lambda) r_t$$

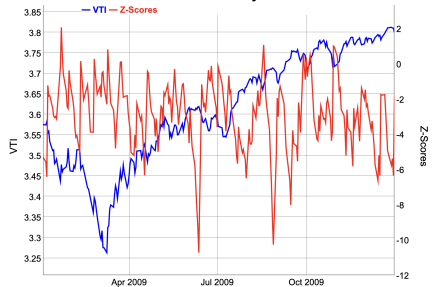
$$\sigma_t^2 = \lambda^2 \sigma_{t-1}^2 + (1 - \lambda^2)(r_t - \bar{r}_t)^2$$

Where $\bar{r}_t$ and σ_t^2 are the trailing mean and variance at time t .

The decay factor λ determines how quickly the mean and variance estimates are updated, with smaller values of λ producing faster updating, giving more weight to recent prices, and vice versa.

```
> # Calculate the recursive trailing VTI volatility
> lambdaf <- 0.8 ## Fast lambda
> lambdas <- 0.81 ## Slow lambda
> volatf <- sqrt(HighFreq::run_var(retp, lambda=lambdaf)[, 2])
> volats <- sqrt(HighFreq::run_var(retp, lambda=lambdas)[, 2])
```

VTI Recursive Volatility Z-Scores



```
> # Plot histogram of the volatility z-scores
> hist(volatz, breaks=1e2)
> # Plot scatterplot of volume and volatility z-scores
> plot(as.numeric(volatz), as.numeric(volumz),
+      xlab="volatility z-score", ylab="volume z-score")
> regmod <- lm(volatz ~ volumz)
> abline(regmod, col="red", lwd=3)
> # Plot dygraph of VTI volatility z-scores
> pricev <- cbind(closep, volatz)
> colnames(pricev) <- c("VTI", "Z-Scores")
> colv <- colnames(pricev)
> dygraphs::dygraph(pricev["2008/2009"], main="VTI Online Volatility
+   dyAxis("y", label=colv[1], independentTicks=TRUE) %>%
+   dyAxis("y2", label=colv[2], independentTicks=TRUE) %>%
+   dySeries(name=colv[1], axis="y", strokeWidth=2, col="blue") %>%
+   dySeries(name=colv[2], axis="y2", strokeWidth=2, col="red") %>%
+   dyLegend(show="always", width=300)
```

Centered Price Z-Scores

An extreme local price is a price which differs significantly from neighboring prices.

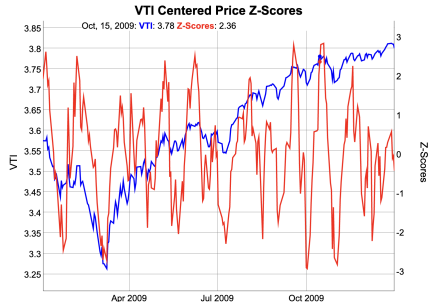
Extreme prices can be identified in-sample using the centered *price* z-score equal to the price difference with neighboring prices divided by the volatility of returns σ_t :

$$z_t = \frac{p_t - 0.5(p_{t-k} - p_{t+k})}{\sigma_t}$$

Where p_{t-k} and p_{t+k} are the lagged and advanced prices.

The lag parameter k is the interval for calculating the volatility of returns σ_t .

```
> # Calculate the centered volatility
> lookb <- 21
> halfb <- lookb %/% 2
> volv <- HighFreq::roll_var(closep, lookb=lookb)
> volv <- sqrt(volv)
> volv <- rutils::lagit(volv, lagg=(-halfb))
> # Calculate the z-scores of prices
> pricez <- (closep -
+ 0.5*(rutils::lagit(closep, halfb, pad_zeros=FALSE) +
+ rutils::lagit(closep, -halfb, pad_zeros=FALSE)))
> pricez <- ifelse(volv > 0, pricez/volv, 0)
```



```
> # Plot dygraph of z-scores of VTI prices
> pricev <- cbind(closep, pricez)
> colnames(pricev) <- c("VTI", "Z-Scores")
> colv <- colnames(pricev)
> dygraphs::dygraph(pricev["2009"], main="VTI Centered Price Z-Scores")
+ dyAxis("y", label=colv[1], independentTicks=TRUE) %>%
+ dyAxis("y2", label=colv[2], independentTicks=TRUE) %>%
+ dySeries(name=colv[1], axis="y", strokeWidth=2, col="blue") %>%
+ dySeries(name=colv[2], axis="y2", strokeWidth=2, col="red") %>%
+ dyLegend(show="always", width=300)
```

Labeling the Tops and Bottoms of Prices

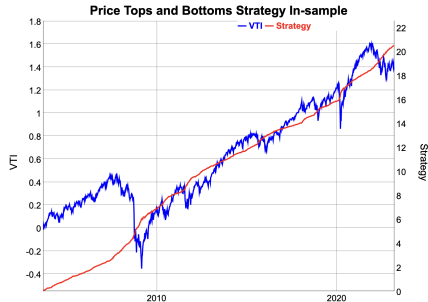
The local tops and bottoms of prices can be labeled approximately in-sample using the z-scores of prices and threshold values.

The local tops of prices represent *overbought* conditions, while the bottoms represent *oversold* conditions.

The labeled data can be used as a response or target variable in machine learning classifier models.

But it's not feasible to classify the prices out-of-sample exactly according to their in-sample labels.

```
> # Calculate the thresholds for labeling tops and bottoms
> confl <- c(0.2, 0.8)
> threshv <- quantile(pricez, confl)
> # Calculate the vectors of tops and bottoms
> topl <- zoo::coredata(pricez > threshv[2])
> bottoml <- zoo::coredata(pricez < threshv[1])
> # Simulate in-sample VTI strategy
> posv <- rep(NA_integer_, nrow)
> posv[1] <- 0
> posv[topl] <- (-1)
> posv[bottoml] <- 1
> posv <- zoo::na.locf(posv)
> posv <- rutils::lagit(posv)
> pnls <- ret*p
```



```
> # Plot dygraph of in-sample VTI strategy
> wealthv <- cbind(ret, pnls)
> colnames(wealthv) <- c("VTI", "Strategy")
> endw <- rutils::calc_endpoints(wealthv, interval="weeks")
> dygraphs::dygraph(cumsum(wealthv)[endw],
+   main="Price Tops and Bottoms Strategy In-sample") %>%
+   dyAxis("y", label="VTI", independentTicks=TRUE) %>%
+   dyAxis("y2", label="Strategy", independentTicks=TRUE) %>%
+   dySeries(name="VTI", axis="y", strokeWidth=2, col="blue") %>%
+   dySeries(name="Strategy", axis="y2", strokeWidth=2, col="red")
```

Trailing Price Z-Scores

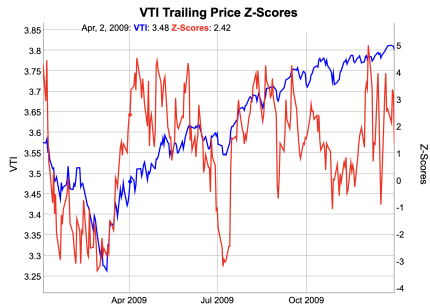
The trailing price z-score is equal to the difference between the current price p_t minus the trailing average price $\bar{p}_{t-k}$, divided by the volatility of the price σ_t :

$$z_t = \frac{p_t - \bar{p}_{t-k}}{\sigma_t}$$

The lag parameter k is the look-back interval for calculating the volatility of returns σ_t .

The trailing price z-scores represent the first derivative (slope) of the prices, since the price level is subtracted.

```
> # Calculate the trailing VTI volatility
> volv <- HighFreq::roll_var(closep, lookb=lookb)
> volv <- sqrt(volv)
> # Calculate the trailing z-scores of VTI prices
> pricez <- (closep - rutils::lagit(closep, lookb, pad_zeros=FALSE))
> pricez <- ifelse(volv > 0, pricez/volv, 0)
```



```
> # Plot dygraph of the trailing z-scores of VTI prices
> pricev <- cbind(closep, pricez)
> colnames(pricev) <- c("VTI", "Z-Score")
> colv <- colnames(pricev)
> dygraphs::dygraph(pricev["2009"],
+   main="VTI Trailing Price Z-Scores") %>%
+   dyAxis("y", label=colv[1], independentTicks=TRUE) %>%
+   dyAxis("y2", label=colv[2], independentTicks=TRUE) %>%
+   dySeries(axis="y", label=colv[1], strokeWidth=2, col="blue") %>%
+   dySeries(axis="y2", label=colv[2], strokeWidth=2, col="red") %>%
+   dyLegend(show="always", width=300)
```

Recursive Trailing Price Z-Scores

The recursive trailing price z-score is equal to the difference between the current price p_t minus the trailing average price $\bar{p}$, divided by the price volatility σ_t :

$$z_t = \frac{p_t - \bar{p}_t}{\sigma_t}$$

The function `HighFreq::run_var()` calculates the trailing mean and variance of the prices p_t , by recursively weighting the past variance estimates σ_{t-1}^2 , with the squared differences of the prices minus their trailing means $(p_t - \bar{p}_t)^2$, using the decay factor λ :

$$\bar{p}_t = \lambda \bar{p}_{t-1} + (1 - \lambda) p_t$$

$$\sigma_t^2 = \lambda^2 \sigma_{t-1}^2 + (1 - \lambda^2) (p_t - \bar{p}_t)^2$$

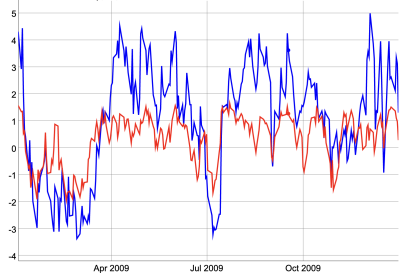
Where $\bar{p}_t$ and σ_t^2 are the trailing mean and variance at time t .

The decay factor λ determines how quickly the mean and variance estimates are updated, with smaller values of λ producing faster updating, giving more weight to recent prices, and vice versa. If λ is close to 1 then the decay is weak and past prices have a greater weight, and the trailing mean values have a stronger dependence on past prices. This is equivalent to a long look-back interval.

And vice versa if λ is close to 0.

VTI Online Trailing Price Z-Scores

Sep, 17, 2009: Z-Scores: 3.63 Recursive: 1.28



```
> # Calculate the EMA returns and volatilities
> lambdaf <- 0.9
> volv <- HighFreq::run_var(closep, lambda=lambdaf)
> # Calculate the recursive trailing z-scores of VTI prices
> pricer <- (closep - volv[, 1])
> pricer <- ifelse(volv > 0, pricer/volv, 0)
> volv <- sqrt(volv[, 2])
> # Plot dygraph of the trailing z-scores of VTI prices
> pricev <- xts::xts(cbind(pricez, pricer), datev)
> colnames(pricev) <- c("Z-Scores", "Recursive")
> colv <- colnames(pricev)
> dygraphs::dygraph(pricev["2009"], main="VTI Online Trailing Price
+   dyOptions(colors=c("blue", "red"), strokeWidth=2) %>%
+   dyLegend(show="always", width=300)
```

Trailing Regression Z-Scores

We can define the trailing *z-score* z_t of the stock price p_t as the *standardized residual* of the linear regression with respect to a predictor variable (for example the time t_i):

$$z_t = \frac{p_t - p_t^{fit}}{\sigma_t}$$

$$p_t^{fit} = \alpha_t + \beta_t t_i$$

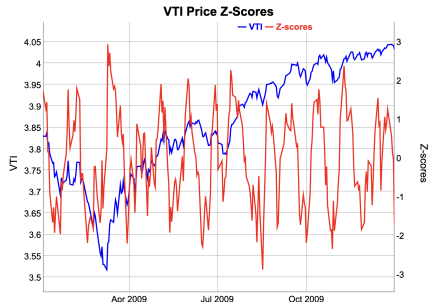
Where p_t^{fit} are the fitted values, α_t and β_t are the *regression coefficients*, and σ_t is the standard deviation of the residuals.

The regression z-scores represent the second derivative (curvature) of the stock prices, since the price level and slope are subtracted.

The regression z-scores can be used as a rich or cheap indicator, either relative to past prices, or relative to prices in a stock pair.

The regression residuals must be calculated in a loop, so it's much faster to calculate them using functions written in C++ code.

The function `HighFreq::roll_reg()` calculates trailing regressions and their residuals.



```
> # Calculate trailing price regression z-scores
> datev <- matrix(zoo::index(closep))
> lookb <- 21
> # Create a default list of regression parameters
> controll <- HighFreq::param_reg()
> regs <- HighFreq::roll_reg(respv=closep, predm=datev,
+   lookb=lookb, controll=controll)
> regs[1:lookb, ] <- 0
> # Plot dygraph of z-scores of VTI prices
> datav <- cbind(closep, regs[, NCOL(regs)])
> colnames(datav) <- c("VTI", "Z-Scores")
> colv <- colnames(datav)
> dygraphs::dygraph(datav["2009"], main="VTI Regression Z-Scores") %>%
+   dyAxis("y", label=colv[1], independentTicks=TRUE) %>%
+   dyAxis("y2", label=colv[2], independentTicks=TRUE) %>%
+   dySeries(name=colv[1], axis="y", strokeWidth=2, col="blue") %>%
+   dySeries(name=colv[2], axis="y2", strokeWidth=2, col="red") %>%
+   dyLegend(show="always", width=300)
```

Recursive Trailing Regression

The trailing regressions of the stock price p_t with respect to the predictor (explanatory) variables X_t are defined by:

$$p_t = \beta_t X_t + \epsilon_t$$

The trailing regression coefficients β_t and the residuals ϵ_t can be calculated as:

$$\beta_t = \text{cov}_{X_t}^{-1} \text{cov}_t$$

$$\epsilon_t = r_t - \beta_t p_t$$

Where cov_t is the covariance matrix between the response p_t and the predictor X_t variables, and cov_{X_t} is the covariance matrix between the predictors.

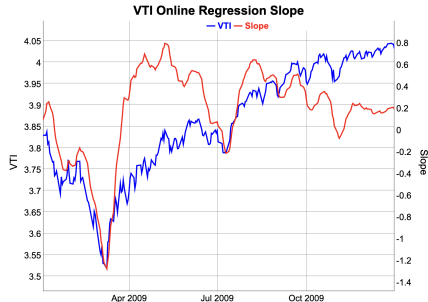
The covariance matrices are updated using the following recursive (online) formulas:

$$\text{cov}_t = \lambda \text{cov}_{t-1} + (1 - \lambda) p_t^T X_t$$

$$\text{cov}_{X_t} = \lambda \text{cov}_{X_{(t-1)}} + (1 - \lambda) X_t^T X_t$$

The function `HighFreq::run_reg()` recursively calculates trailing regressions and their residuals.

```
> # Calculate recursive trailing price regression versus time
> lambdaf <- 0.9
> # Create a list of regression parameters
> controll <- HighFreq::param_reg(residscale="standardize")
> regs <- HighFreq::run_reg(closep, matrix(datev), lambda=lambdaf, controll=controll)
> colnames(regs) <- c("alpha", "beta", "zscores")
> tail(regs)
```



```
> # Plot dygraph of regression betas
> datav <- cbind(closep, 252*regs[, "beta"])
> colnames(datav) <- c("VTI", "Slope")
> colv <- colnames(datav)
> dygraphs::dygraph(datav["2009"], main="VTI Online Regression Slope")
+ dyAxis("y", label=colv[1], independentTicks=TRUE) %>%
+ dyAxis("y2", label=colv[2], independentTicks=TRUE) %>%
+ dySeries(axis="y", label=colv[1], strokeWidth=2, col="blue") %>%
+ dySeries(axis="y2", label=colv[2], strokeWidth=2, col="red") %>%
+ dyLegend(show="always", width=300)
```

Recursive Trailing Regression Z-Scores

The recursive trailing z -score z_t of the stock price p_t is equal to the standardized residual ϵ_t :

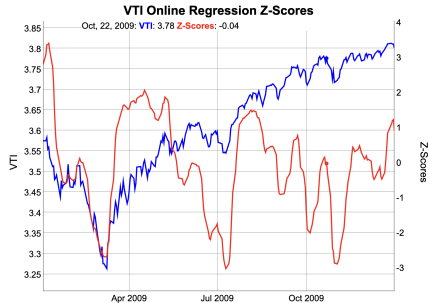
$$\epsilon_t = \lambda \epsilon_{t-1} + (1 - \lambda)(p_t - \beta_t p_t)$$

$$\bar{\epsilon}_t = \lambda \bar{\epsilon}_{t-1} + (1 - \lambda) \epsilon_t$$

$$\varsigma_t^2 = \lambda^2 \varsigma_{t-1}^2 + (1 - \lambda^2)(\epsilon_t - \bar{\epsilon}_t)^2$$

$$z_t = \frac{\epsilon_t}{\varsigma_t}$$

Where ς_t^2 is the variance of the residuals ϵ_t .



```
> # Plot dygraph of z-scores of VTI prices
> datav <- cbind(closep, regs[, "zscores"])
> colnames(datav) <- c("VTI", "Z-Scores")
> colv <- colnames(datav)
> dygraphs::dygraph(datav["2009"], main="VTI Online Regression Z-Scores")
+ dyAxis("y", label=colv[1], independentTicks=TRUE) %>%
+ dyAxis("y2", label=colv[2], independentTicks=TRUE) %>%
+ dySeries(name=colv[1], axis="y", strokeWidth=2, col="blue") %>%
+ dySeries(name=colv[2], axis="y2", strokeWidth=2, col="red") %>%
+ dyLegend(show="always", width=300)
```

The Hampel Filter

The *Median Absolute Deviation* (*MAD*) is a nonparametric measure of dispersion (variability):

$$\text{MAD} = \text{median}(\text{abs}(p_t - \text{median}(\mathbf{p})))$$

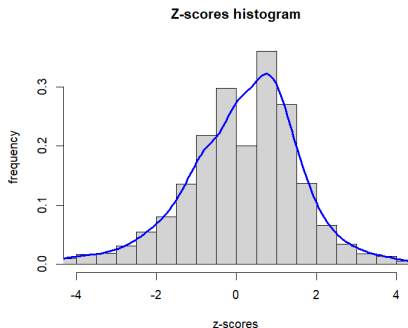
The *Hampel filter* is effective in detecting outliers in the data because it uses the nonparametric *MAD* dispersion measure.

The *Hampel z-score* is equal to the deviation from the median divided by the *MAD*:

$$z_i = \frac{p_t - \text{median}(\mathbf{p})}{\text{MAD}}$$

A time series of *z-scores* over past data can be calculated using a trailing look-back window.

```
> # Extract time series of VTI log prices
> closep <- log(na.omit(rutils::etfenv$prices$VTI))
> # Define look-back window
> lookb <- 11
> # Calculate time series of trailing medians
> medianv <- HighFreq::roll_mean(closep, lookb=lookb, method="nonp")
> # Calculate time series of MAD
> madv <- HighFreq::roll_var(closep, lookb=lookb, method="nonparam")
> # madv <- TTR::runMAD(closep, n=lookb)
> # Calculate time series of z-scores
> zscores <- (closep - medianv)/madv
> zscores[1:lookb, ] <- 0
> tail(zscores, lookb)
> range(zscores)
```



```
> # Plot the prices and medians
> dygraphs::dygraph(cbind(closep, medianv), main="VTI median") %>%
+   dyOptions(colors=c("black", "red")) %>%
+   dyLegend(show="always", width=300)
> # Plot histogram of z-scores
> histp <- hist(zscores, col="lightgrey",
+   xlab="z-scores", breaks=50, xlim=c(-4, 4),
+   ylab="frequency", freq=FALSE, main="Hampel Z-Scores histogram")
> lines(density(zscores, adjust=1.5), lwd=3, col="blue")
```

One-sided and Two-sided Data Filters

Filters calculated over past data are referred to as *one-sided* filters, and they are appropriate for filtering real-time data.

Filters calculated over both past and future data are called *two-sided* (centered) filters, and they are appropriate for filtering historical data.

The function `HighFreq::roll_var()` with parameter `method="nonparametric"` calculates the trailing *MAD* using a look-back interval over past data.

If the trailing medians and *MAD* are advanced (shifted backward) in time, then they are calculated over both past and future data (centered).

The function `rutils::lag_it()` with a negative `lagg` parameter value advances (shifts back) future data points to the present.

```
> # Calculate one-sided Hampel z-scores
> medianv <- HighFreq::roll_mean(closep, lookb=lookb, method="nonparametric")
> madv <- HighFreq::roll_var(closep, lookb=lookb, method="nonparametric")
> zscores <- (closep - medianv)/madv
> zscores[1:lookb, ] <- 0
> tail(zscores, lookb)
> range(zscores)
> # Calculate two-sided Hampel z-scores
> halfb <- lookb %/% 2
> medianv <- rutils::lagit(medianv, lagg=(-halfb))
> madv <- rutils::lagit(madv, lagg=(-halfb))
> zscores <- (closep - medianv)/madv
> zscores[1:lookb, ] <- 0
> tail(zscores, lookb)
> range(zscores)
```

Calculating the Trailing Variance of Asset Returns

The variance of asset returns exhibits *heteroskedasticity*, i.e. it changes over time.

The trailing variance of returns is given by:

$$\sigma_t^2 = \frac{1}{k-1} \sum_{j=0}^{k-1} (r_{t-j} - \bar{r}_t)^2$$

$$\bar{r}_t = \frac{1}{k} \sum_{j=0}^{k-1} r_{t-j}$$

Where k is the *look-back interval* equal to the number of data points for performing aggregations over the past.

It's also possible to calculate the trailing variance in R using vectorized functions, without using an `apply()` loop.

```
> # Calculate VTI percentage returns
> retp <- na.omit(rutils::etfenv$returns$VTI)
> nrow <- NROW(retp)
> # Define end points
> endd <- 1:NROW(retp)
> # Start points are multi-period lag of endd
> lookb <- 11
> startp <- c(rep_len(0, lookb-1), endd[1:(nrow-lookb+1)])
> # Calculate trailing variance in sapply() loop - takes long
> varv <- sapply(1:nrow, function(it) {
+   retp <- retp[startp[it]:endd[it]]
+   sum((retp - mean(retp))^2)/lookb
+ }) ## end sapply
> # Use only vectorized functions
> retc <- cumsum(retp)
> retc <- (retc - c(rep_len(0, lookb), retc[1:(nrow-lookb)]))
> retc2 <- cumsum(retp^2)
> retc2 <- (retc2 - c(rep_len(0, lookb), retc2[1:(nrow-lookb)]))
> var2 <- (retc2 - retc^2/lookb)/lookb
> all.equal(varv[-(1:lookb)], as.numeric(var2)[-(1:lookb)])
> # Or using package rutils
> retc <- rutils::roll_sum(retp, lookb=lookb)
> retc2 <- rutils::roll_sum(retp^2, lookb=lookb)
> var2 <- (retc2 - retc^2/lookb)/lookb
> # Coerce variance into xts
> tail(varv)
> class(varv)
> varv <- xts(varv, order.by=zoo::index(retp))
> colnames(varv) <- "VTI.variance"
> head(varv)
```

Calculating the Trailing Variance Using Package *roll*

The package *roll* contains functions for calculating *weighted* trailing aggregations over *vectors* and *time series* objects:

- `roll_sum()` for the *weighted* trailing sum,
- `roll_var()` for the *weighted* trailing variance,
- `roll_scale()` for the trailing scaling and centering of time series,
- `roll_pcr()` for the trailing principal component regressions of time series.

The *roll* functions are about 1,000 times faster than `apply()` loops!

The *roll* functions are extremely fast because they perform calculations in *parallel* in compiled C++ code, using packages *Rcpp*, *RcppArmadillo*, and *RcppParallel*.

The *roll* functions accept *xts* time series, and they return *xts*.

```
> # Calculate trailing VTI variance using package HighFreq
> varv <- roll::roll_var(retp, width=lookb)
> colnames(varv) <- "Variance"
> head(varv)
> sum(is.na(varv))
> varv[1:(lookb-1)] <- 0
> # Benchmark calculation of trailing variance
> library(microbenchmark)
> summary(microbenchmark(
+   sapply=sapply(1:nrows, function(it) {
+     var(retp[startp[it]:endp[it]])
+   }),
+   roll=roll::roll_var(retp, width=lookb),
+   times=10))[, c(1, 4, 5)]
```

Trailing *EMA* Realized Volatility Estimator

Time-varying volatility can be more accurately estimated using an *Exponential Moving Average (EMA)* variance estimator.

If the *time series* has zero *expected* mean, then the *EMA realized variance estimator* can be written approxiamtely as:

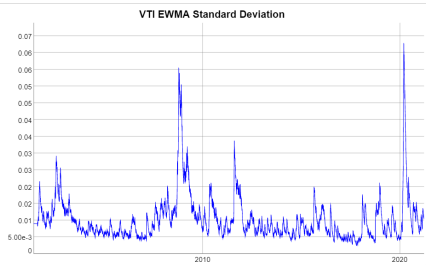
$$\sigma_t^2 = \lambda \sigma_{t-1}^2 + (1 - \lambda) r_t^2 = (1 - \lambda) \sum_{j=0}^{\infty} \lambda^j r_{t-j}^2$$

σ_t^2 is the weighted *realized* variance, equal to the weighted average of the point realized variance for period *i* and the past *realized* variance.

The parameter λ determines the rate of decay of the *EMA* weights, with smaller values of λ producing faster decay, giving more weight to recent realized variance, and vice versa.

The function `stats:::C_cfilter()` calculates the convolution of a vector or a time series with a filter of coefficients (weights).

The function `stats:::C_cfilter()` is very fast because it's compiled C++ code.



```
> # Calculate EMA VTI variance using compiled C++ function
> lookb <- 51
> weightv <- exp(-0.1*1:lookb)
> weightv <- weightv/sum(weightv)
> varv <- .Call(stats:::C_cfilter, retv^2, filter=weightv, sides=1,
> varv[1:(lookb-1)] <- varv[lookb]
> # Plot EMA volatility
> varv <- xts:::xts(sqrt(varv), order.by=zoo::index(retv))
> dygraphs::dygraph(varv, main="VTI EMA Volatility") %>%
+   dyOptions(colors="blue") %>% dyLegend(show="always", width=300)
> quantmod::chart_Series(xtsv, name="VTI EMA Volatility")
```

Estimating Trailing Variance Using Package *roll*

If the *time series* has non-zero *expected* mean, then the trailing *EMA* variance is a vector given by the estimator:

$$\sigma_t^2 = \frac{1}{k-1} \sum_{j=0}^{k-1} w_j (r_{t-j} - \bar{r}_t)^2$$

$$\bar{r}_t = \frac{1}{k} \sum_{j=0}^{k-1} w_j r_{t-j}$$

Where w_j is the vector of exponentially decaying weights:

$$w_j = \frac{\lambda^j}{\sum_{j=0}^{k-1} \lambda^j}$$

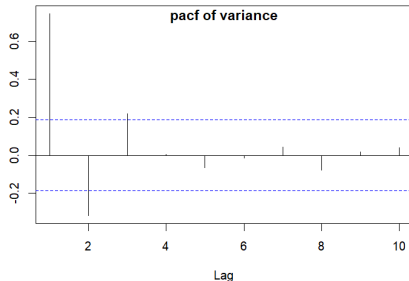
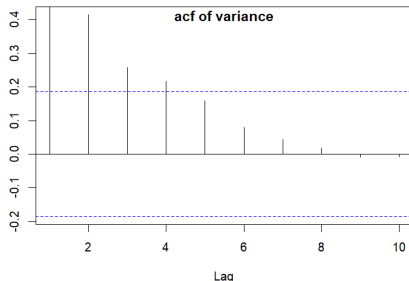
The function `roll_var()` from package *roll* calculates the trailing *EMA* variance.

```
> # Calculate trailing VTI variance using package roll
> library(roll) ## Load roll
> varv <- roll::roll_var(retp, weights=rev(weightv), width=lookb)
> colnames(varv) <- "VTI.variance"
> class(varv)
> head(varv)
> sum(is.na(varv))
> varv[1:(lookb-1)] <- 0
```

Autocorrelation of Volatility

Variance calculated over non-overlapping intervals has very statistically significant autocorrelations.

```
> # Calculate VTI percentage returns
> retp <- na.omit(rutils::etfenv$returns$VTI)
> # Calculate VTI rolling variance
> lookb <- 21
> varv <- HighFreq::roll_var(retp, lookb=lookb)
> colnames(varv) <- "Variance"
> # Number of lookbv that fit over returns
> nrows <- NROW(retp)
> nagg <- nrows %/% lookb
> # Define end points with beginning stub
> endd <- c(0, nrows-lookb*nagg + (0:nagg)*lookb)
> nrows <- NROW(endd)
> # Subset variance to end points
> varv <- varv[endd]
> # Plot autocorrelation function
> rutils::plot_acf(varv, lag=10, main="ACF of Variance")
> # Plot partial autocorrelation
> pacf(varv, lag=10, main="PACF of Variance", ylab=NA)
```



Exponential Moving Average *EMA* Volatility Estimator

The exponential moving average (*EMA*) variance of the returns r_t can be calculated by recursively weighting the past variance estimates σ_{t-1}^2 , with the squared differences of the returns minus their trailing means $(r_t - \bar{r}_t)^2$:

$$\bar{r}_t = \lambda \bar{r}_{t-1} + (1 - \lambda) r_t$$

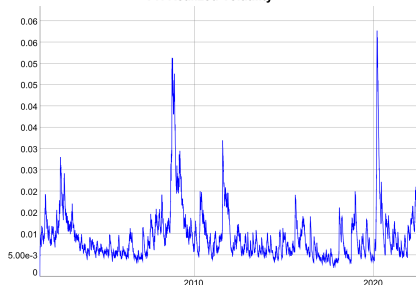
$$\sigma_t^2 = \lambda^2 \sigma_{t-1}^2 + (1 - \lambda^2) (r_t - \bar{r}_t)^2$$

Where $\bar{r}_t$ and σ_t^2 are the *EMA* mean and variance at time t , and λ is the decay factor.

The decay factor λ determines how quickly the mean and variance estimates are updated, with smaller values of λ producing faster updating, giving more weight to recent prices, and vice versa.

The function `HighFreq::run_var()` calculates the *EMA* mean and variance of the returns r_t .

VTI Realized Volatility



```
> # Calculate realized variance recursively
> lambdaf <- 0.9
> volv <- HighFreq::run_var(retp, lambda=lambdaf)
> volv <- sqrt(volv[, 2])
> # Plot EMA volatility
> volv <- xts::xts(volv, order.by=datev)
> dygraphs::dygraph(volv, main="VTI EMA Volatility") %>%
+   dyOptions(colors="blue") %>% dyLegend(show="always", width=300)
```

Estimating Daily Volatility From Intraday Returns

The standard *close-to-close* volatility σ depends on the *Close* prices C_i from *OHLC* data:

$$\sigma^2 = \frac{1}{n-1} \sum_{i=1}^n (r_i - \bar{r})^2$$

$$\bar{r} = \frac{1}{n} \sum_{i=0}^n r_i \quad r_i = \log\left(\frac{C_i}{C_{i-1}}\right)$$

But intraday time series of prices (for example `HighFreq::SPY` prices), can have large overnight jumps which inflate the volatility estimates.

So the overnight returns must be divided by the overnight time interval (in seconds), which produces per second returns.

The per second returns can be multiplied by 60 to scale them back up to per minute returns.

The function `zoo::index()` extracts the time index of a time series.

The function `xts::.index()` extracts the time index expressed in the number of seconds.

```
> library(HighFreq) ## Load HighFreq
> # Minutely SPY returns (unit per minute) single day
> # Minutely SPY volatility (unit per minute)
> retspy <- rutils::diffit(log(SPY["2012-02-13", 4]))
> sd(retspy)
> # SPY returns multiple days (includes overnight jumps)
> retspy <- rutils::diffit(log(SPY[, 4]))
> sd(retspy)
> # Table of time intervals - 60 second is most frequent
> indeks <- rutils::diffit(xts::.index(SPY))
> table(indeks)
> # SPY returns divided by the overnight time intervals (unit per second)
> retspy <- retspy/indeks
> retspy[1] <- 0
> # Minutely SPY volatility scaled to unit per minute
> 60*sd(retspy)
```

Range Volatility Estimators of OHLC Time Series

Range estimators of return volatility utilize the high and low prices, and therefore have lower standard errors than the standard *close-to-close* estimator.

The *Garman-Klass* estimator uses the *low-to-high* price range, but it underestimates volatility because it doesn't account for *close-to-open* price jumps:

$$\sigma^2 = \frac{1}{n} \sum_{i=1}^n (0.5 \log(\frac{H_i}{L_i})^2 - (2 \log 2 - 1) \log(\frac{C_i}{O_i})^2)$$

The *Yang-Zhang* estimator accounts for *close-to-open* price jumps and has the lowest standard error among unbiased estimators:

$$\begin{aligned} \sigma^2 = & \frac{1}{n-1} \sum_{i=1}^n (\log(\frac{O_i}{C_{i-1}}) - \bar{r}_{co})^2 + \\ & 0.134 (\log(\frac{C_i}{O_i}) - \bar{r}_{oc})^2 + \\ & \frac{0.866}{n} \sum_{i=1}^n (\log(\frac{H_i}{O_i}) \log(\frac{H_i}{C_i}) + \log(\frac{L_i}{O_i}) \log(\frac{L_i}{C_i})) \end{aligned}$$

The *Yang-Zhang* (YZ) and *Garman-Klass-Yang-Zhang* (GKYZ) estimators are unbiased and have up to seven times smaller standard errors than the standard *close-to-close* estimator.

But in practice, prices are not observed continuously, so the price range is underestimated, and so is the variance when using the YZ and GKYZ range estimators.

Therefore in practice the YZ and GKYZ range estimators underestimate the volatility, and their standard errors are reduced less than by the theoretical amount, for the same reason.

The *Garman-Klass-Yang-Zhang* estimator is another very efficient and unbiased estimator, and also accounts for *close-to-open* price jumps:

$$\begin{aligned} \sigma^2 = & \frac{1}{n} \sum_{i=1}^n ((\log(\frac{O_i}{C_{i-1}}) - \bar{r})^2 + \\ & 0.5 \log(\frac{H_i}{L_i})^2 - (2 \log 2 - 1) (\log(\frac{C_i}{O_i}))^2) \end{aligned}$$

Calculating the Trailing Range Variance Using *HighFreq*

The function `HighFreq::calc_var_ohlcv()` calculates the *variance* of returns using several different range volatility estimators.

If the logarithms of the *OHLC* prices are passed into `HighFreq::calc_var_ohlcv()` then it calculates the variance of percentage returns, and if simple *OHLC* prices are passed then it calculates the variance of dollar returns.

The function `HighFreq::roll_var_ohlcv()` calculates the *trailing* variance of returns using several different range volatility estimators.

The functions `HighFreq::calc_var_ohlcv()` and `HighFreq::roll_var_ohlcv()` are very fast because they are written in C++ code.

The function `TTR::volatility()` calculates the range volatility, but it's significantly slower than `HighFreq::calc_var_ohlcv()`.

```
> library(HighFreq) ## Load HighFreq
> spy <- HighFreq::SPY["2008/2009"]
> # Calculate daily SPY volatility using package HighFreq
> sqrt(6.5*60*HighFreq::calc_var_ohlcv(log(spy),
+   method="yang_zhang"))
> # Calculate daily SPY volatility from minutely prices using package HighFreq
> sqrt((6.5*60)*mean(na.omit(
+   TTR::volatility(spy, N=1, calc="yang.zhang"))^2))
> # Calculate trailing SPY variance using package HighFreq
> varv <- HighFreq::roll_var_ohlcv(log(spy), method="yang_zhang",
+   lookb=lookb)
> # Plot range volatility
> varv <- xts::xts(sqrt(varv), order.by=zoo::index(spy))
> dygraphs::dygraph(varv["2009-02"], main="SPY Trailing Range Volatility",
+   dyOptions(colors="blue") %>% dyLegend(show="always", width=300))
> # Benchmark the speed of HighFreq vs TTR
> library(microbenchmark)
> summary(microbenchmark(
+   ttr=TTR::volatility(rutils::etfenv$VTI, N=1, calc="yang.zhang"),
+   HighFreq=HighFreq::calc_var_ohlcv(log(rutils::etfenv$VTI), method="yang_zhang",
+   times=2)), c(1, 4, 5))
```

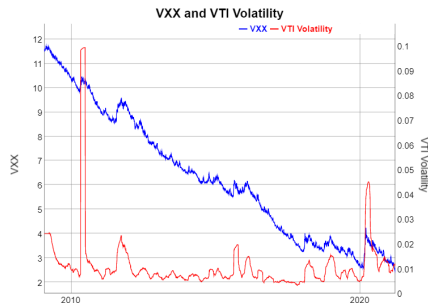
VXX Prices and the Trailing Volatility

The VXX ETF invests in *VIX* futures, so its price is tied to the level of the *VIX* index, with higher VXX prices corresponding to higher levels of the *VIX* index.

The trailing volatility of past returns moves in sympathy with the implied volatility and VXX prices, but with a lag.

But VXX prices exhibit a very strong downward trend which makes them hard to compare with the trailing volatility.

```
> # Calculate VXX log prices
> vxx <- na.omit(rutils::etfenv$prices$VXX)
> datev <- zoo::index(vxx)
> lookb <- 41
> vxx <- log(vxx)
> # Calculate trailing VTI volatility
> closep <- get("VTI", rutils::etfenv)[datev]
> closep <- log(closep)
> volv <- sqrt(HighFreq::roll_var_ohlcv(ohlcv=closep, lookb=lookb, s=
> volv[1:lookb] <- volv[lookb+1]
```



```
> # Plot dygraph of VXX and VTI volatility
> datav <- cbind(vxx, volv)
> colnames(datav)[2] <- "VTI Volatility"
> colv <- colnames(datav)
> caption <- "VXX and VTI Volatility"
> dygraphs::dygraph(datav[, 1:2], main=caption) %>%
+   dyAxis("y", label=colv[1], independentTicks=TRUE) %>%
+   dyAxis("y2", label=colv[2], independentTicks=TRUE) %>%
+   dySeries(name=colv[1], axis="y", strokeWidth=1, col="blue") %>%
+   dySeries(name=colv[2], axis="y2", strokeWidth=1, col="red") %>%
+   dyLegend(show="always", width=300)
```

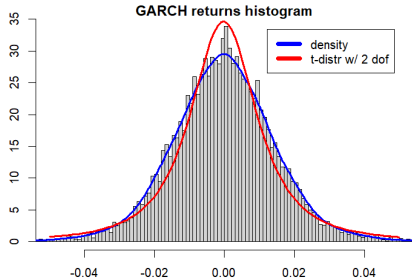
draft: Cointegration of VXX Prices and the trailing Volatility

The trailing volatility of past returns moves in sympathy with the implied volatility and VXX prices, but with a lag.

The parameter α is the weight of the squared realized returns in the variance.

Greater values of α produce a stronger feedback between the realized returns and variance, causing larger variance spikes and higher kurtosis.

```
> # Calculate VXX log prices
> vxx <- na.omit(rutils::etfenv$prices$VXX)
> datev <- zoo::index(vxx)
> lookb <- 41
> vxx <- log(vxx)
> vxx <- (vxx - HighFreq::roll_mean(vxx, lookb=lookb))
> vxx[1:lookb] <- vxx[lookb+1]
> # Calculate trailing VTI volatility
> closep <- get("VTI", rutils::etfenv)[datev]
> closep <- log(closep)
> volv <- sqrt(HighFreq::roll_var_ohlc(ohlc=closep, lookb=lookb, s=
> volv[1:lookb] <- volv[lookb+1]
> # Calculate regression coefficients of XLB ~ XLE
> betac <- drop(cov(vxx, volv)/var(volv))
> alphac <- drop(mean(vxx) - betac*mean(volv))
> # Calculate regression residuals
> fitv <- (alphac + betac*volv)
> residuals <- (vxx - fitv)
> # Perform ADF test on residuals
> tseries::adf.test(residuals, k=1)
```



```
> # Plot dygraph of VXX and VTI volatility
> datav <- cbind(vxx, volv)
> colv <- colnames(datav)
> caption <- "VXX and VTI Volatility"
> dygraphs::dygraph(datav[, 1:2], main=caption) %>%
+   dyAxis("y", label=colv[1], independentTicks=TRUE) %>%
+   dyAxis("y2", label=colv[2], independentTicks=TRUE) %>%
+   dySeries(name=colv[1], axis="y", strokeWidth=1, col="blue") %>%
+   dySeries(name=colv[2], axis="y2", strokeWidth=1, col="red") %>%
+   dyLegend(show="always", width=300)
```

draft: The ARCH Volatility Model

The $ARCH(1,1)$ is a volatility model defined by two coupled equations:

$$\sigma_t^2 = \omega + \beta\sigma_{t-1}^2 + \xi_t^2$$

Where σ_t^2 is the time-dependent variance, equal to the weighted average of the point *realized* variance $(r_t - \bar{r}_t)^2$ and the past variance σ_{t-1}^2 , and ξ_t are standard normal innovations.

The return process r_t follows a normal distribution with a time-dependent variance σ_t^2 .

The parameter α is the weight associated with recent realized variance updates, and β is the weight associated with the past variance.

The long-term expected value of the variance is proportional to the parameter ω :

$$\sigma^2 = \frac{\omega}{1 - \alpha - \beta}$$

So the sum of α plus β should be less than 1, otherwise the volatility is explosive.

```
> # Define GARCH parameters
> alphac <- 0.3; betac <- 0.5;
> omega <- 1e-4*(1 - alphac - betac)
> nrows <- 1000
> # Calculate matrix of standard normal innovations
> set.seed(1121, "Mersenne-Twister", sample.kind="Rejection") ##
> innov <- rnorm(nrows)
> retp <- numeric(nrows)
> varv <- numeric(nrows)
> varv[1] <- omega/(1 - alphac - betac)
> retp[1] <- sqrt(varv[1])*innov[1]
> # Simulate GARCH model
> for (i in 2:nrows) {
+   retp[i] <- sqrt(varv[i-1])*innov[i]
+   varv[i] <- omega + alphac*retp[i]^2 + betac*varv[i-1]
+ } ## end for
> # Simulate the GARCH process using Rcpp
> garchsim <- HighFreq::sim_garch(omega=omega, alpha=alphac,
+   beta=betac, innov=matrix(innov))
> all.equal(garchsim, cbind(retp, varv), check.attributes=FALSE)
```

The *GARCH* process must be simulated using an explicit loop, so it's better to perform it in C++ instead of R.

The GARCH Volatility Model

The $GARCH(1,1)$ volatility model is defined by two coupled equations:

$$r_t = \sigma_{t-1} \xi_t$$

$$\sigma_t^2 = \omega + \beta \sigma_{t-1}^2 + \alpha r_t^2$$

The time-dependent variance σ_t^2 , is equal to the weighted average of the *realized* variance r_t^2 and the past variance σ_{t-1}^2 .

The source of uncertainty are the returns r_t , which are proportional to the standard normal innovations ξ_t .

The parameter α is the weight associated with recent realized variance updates, and β is the weight associated with the past variance.

The long-term equi value of the variance is proportional to the parameter ω :

$$\sigma_{eq}^2 = \frac{\omega}{1 - \alpha - \beta}$$

So the sum of α plus β should be less than 1, otherwise the volatility is explosive.

The difference between the $GARCH(1,1)$ estimator and the *EMA* estimator is that the $GARCH(1,1)$ decays to a non-zero equilibrium value σ_{eq}^2 , while the *EMA* estimator decays to zero.

```
> # Define GARCH parameters
> alphac <- 0.3; betac <- 0.5;
> omega <- 1e-4*(1 - alphac - betac)
> nrows <- 1000
> # Calculate matrix of standard normal innovations
> set.seed(1121, "Mersenne-Twister", sample.kind="Rejection") ##
> innov <- rnorm(nrows)
> retp <- numeric(nrows)
> varv <- numeric(nrows)
> varv[1] <- omega/(1 - alphac - betac)
> retp[1] <- sqrt(varv[1])*innov[1]
> # Simulate GARCH model
> for (i in 2:nrows) {
+   retp[i] <- sqrt(varv[i-1])*innov[i]
+   varv[i] <- omega + alphac*retp[i]^2 + betac*varv[i-1]
+ } ## end for
> # Simulate the GARCH process using Rcpp
> garchsim <- HighFreq::sim_garch(omega=omega, alpha=alphac,
+   beta=betac, innov=matrix(innov))
> all.equal(garchsim, cbind(retp, varv), check.attributes=FALSE)
```

The $GARCH$ process is path dependent, so it must be simulated using an explicit loop, and it's better to perform it in C++ instead of R.

NYU Professor [Robert Engle](#) was awarded the Nobel Prize in Economics for developing the $GARCH$ volatility model.

GARCH Volatility Time Series

The simulated *GARCH* volatility exhibits spikes of volatility followed by an exponential decay.

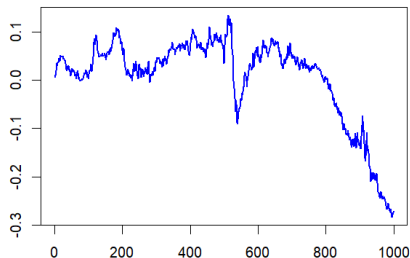
Larger values of α produce a stronger feedback between the simulated returns and variance, which produce larger variance spikes, which produce larger kurtosis.

The parameter α is the weight of the squared realized returns in the variance.

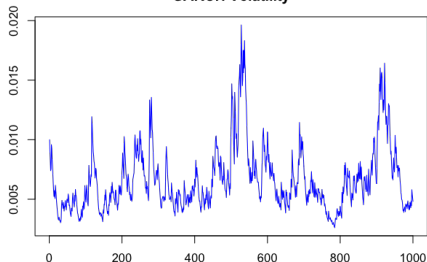
But the decay of the volatility in the *GARCH* model is faster than what is observed in practice.

```
> # Open plot window on Mac
> dev.new(width=6, height=5, noRStudioGD=TRUE)
> # Set plot parameters to reduce whitespace around plot
> par(mar=c(2, 2, 3, 1), oma=c(0, 0, 0, 0))
> # Plot GARCH cumulative returns
> plot(cumsum(retp), t="l", col="blue", xlab="", ylab="",
+      main="GARCH Cumulative Returns")
> quartz.save("figure/garch_returns.png", type="png",
+             width=6, height=5)
> # Plot GARCH volatility
> plot(sqrt(varv), t="l", col="blue", xlab="", ylab="",
+      main="GARCH Volatility")
> quartz.save("figure/garch_volat.png", type="png",
+             width=6, height=5)
```

GARCH cumulative returns



GARCH Volatility



GARCH Returns Distribution

The return process r_t follows a normal distribution, *conditional* on the variance in the previous period σ_{t-1}^2 .

$$r_t = \sigma_{t-1} \xi_t$$

$$\sigma_t^2 = \omega + \beta \sigma_{t-1}^2 + \alpha r_t^2$$

But the *unconditional* distribution of returns is *not* normal, since their standard deviation is time-dependent, so they are *leptokurtic* (fat tailed).

The GARCH volatility model produces *leptokurtic* return distribution, with fat tails.

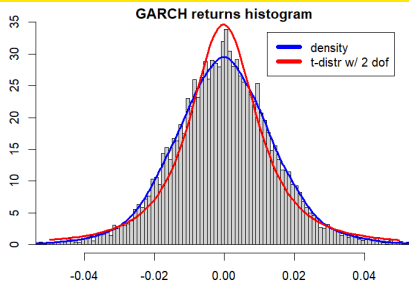
Student's *t-distribution* has fat tails, so it fits asset returns much better than the normal distribution.

Student's *t-distribution* with 3 degrees of freedom is often used to represent asset returns.

The function `fitdistr()` from package *MASS* fits a univariate distribution into a sample of data, by performing *maximum likelihood* optimization.

The function `hist()` calculates and plots a histogram, and returns its data *invisibly*.

```
> # Calculate kurtosis of GARCH returns
> mean(((retp-mean(retp))/sd(retp))^4)
> # Perform Jarque-Bera test of normality
> tseries::jarque.bera.test(retp)
```



```
> # Fit t-distribution into GARCH returns
> fitobj <- MASS::fitdistr(retp, densfun="t", df=2)
> locv <- fitobj$estimate[1]
> scalev <- fitobj$estimate[2]
> # Plot histogram of GARCH returns
> histp <- hist(retp, col="lightgrey",
+   xlab="returns", breaks=200, xlim=c(-0.03, 0.03),
+   ylab="frequency", freq=FALSE, main="GARCH Returns Histogram")
> lines(density(retp, adjust=1.5), lwd=2, col="blue")
> curve(expr=dt((x-locv)/scalev, df=2)/scalev,
+   type="l", xlab="", ylab="", lwd=2,
+   col="red", add=TRUE)
> legend("topright", inset=-0, bty="n", y.intersp=0.4,
+   leg=c("density", "t-distr w/ 2 dof"),
+   lwd=6, lty=1, col=c("blue", "red"))
> quartz.save("figure/garch_hist.png", type="png", width=6, height=6)
```

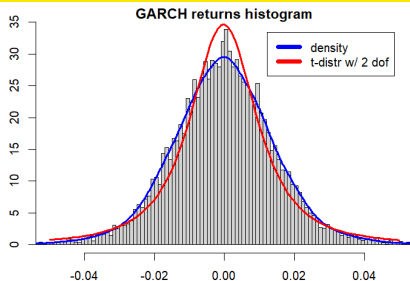
GARCH Model Simulation

The package *fGarch* contains functions for applying *GARCH* models.

The function `fGarch::garchSpec()` specifies a *GARCH* model.

The function `fGarch::garchSim()` simulates a *GARCH* model, but it uses its own random innovations, so its output is not reproducible.

```
> # Specify GARCH model
> garch_spec <- fGarch::garchSpec(model=list(ar=c(0, 0), omega=omeg
+   alpha=alphac, beta=betac))
> # Simulate GARCH model
> garch_sim <- fGarch::garchSim(spec=garch_spec, n=nrows)
> retp <- as.numeric(garch_sim)
> # Calculate kurtosis of GARCH returns
> moments::moment(retp, order=4) /
+   moments::moment(retp, order=2)^2
> # Perform Jarque-Bera test of normality
> tseries::jarque.bera.test(retp)
> # Plot histogram of GARCH returns
> histp <- hist(retp, col="lightgrey",
+   xlab="returns", breaks=200, xlim=c(-0.05, 0.05),
+   ylab="frequency", freq=FALSE,
+   main="GARCH Returns Histogram")
> lines(density(retp, adjust=1.5), lwd=3, col="blue")
```



```
> # Fit t-distribution into GARCH returns
> fitobj <- MASS::fitdistr(retp, densfun="t", df=2, lower=c(-1, 1e-7))
> locv <- fitobj$estimate[1]
> scalev <- fitobj$estimate[2]
> curve(expr=dt((x-locv)/scalev, df=2)/scalev,
+   type="l", xlab="", ylab="", lwd=3,
+   col="red", add=TRUE)
> legend("topright", inset=0.05, bty="n", y.intersp=0.4,
+   leg=c("density", "t-distr w/ 2 dof"),
+   lwd=6, lty=1, col=c("blue", "red"))
```

GARCH Returns Kurtosis

The expected value of the variance σ^2 of GARCH returns is proportional to the parameter ω :

$$\sigma^2 = \frac{\omega}{1 - \alpha - \beta}$$

The expected value of the kurtosis κ of GARCH returns is equal to:

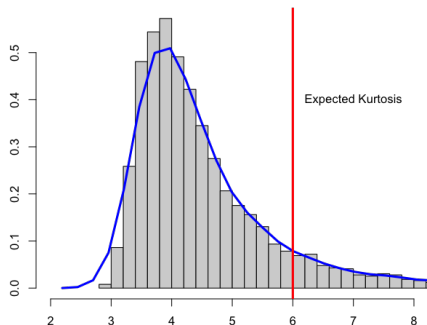
$$\kappa = 3 + \frac{6\alpha^2}{1 - 2\alpha^2 - (\alpha + \beta)^2}$$

The excess kurtosis $\kappa - 3$ is proportional to α^2 because larger values of the parameter α produce larger variance spikes which produce larger kurtosis.

The distribution of kurtosis is highly positively skewed, especially for short returns samples, so most kurtosis values will be significantly below their expected value.

```
> # Calculate variance of GARCH returns
> var(retp)
> # Calculate expected value of variance
> omega/(1 - alphac - betac)
> # Calculate kurtosis of GARCH returns
> mean(((retp-mean(retp))/sd(retp))^4)
> # Calculate expected value of kurtosis
> 3 + 6*alpha^2/(1-2*alpha^2-(alphac+betac)^2)
```

Distribution of GARCH Kurtosis



```
> # Calculate the distribution of GARCH kurtosis
> kurt <- sapply(1:1e4, function(x) {
+   garchsim <- HighFreq::sim_garch(omega=omega, alpha=alphac,
+     beta=betac, innov=matrix(rnorm(nrows)))
+   retp <- garchsim[, 1]
+   c(var(retp), mean(((retp-mean(retp))/sd(retp))^4))
+ }) ## end sapply
> kurt <- t(kurt)
> apply(kurt, 2, mean)
> # Plot the distribution of GARCH kurtosis
> dev.new(width=6, height=5, noRStudioGD=TRUE)
> par(mar=c(2, 2, 3, 1), oma=c(0, 0, 0, 0))
> histp <- hist(kurt[, 2], breaks=500, col="lightgrey",
+   xlim=c(2, 8), xlab="returns", ylab="frequency", freq=FALSE,
+   main="Distribution of GARCH Kurtosis")
```

GARCH Variance Estimation

The *GARCH* model can be used to estimate the trailing variance of empirical (historical) returns.

If the time series of returns r_t is given, then it can be used in the *GARCH(1,1)* formula to estimate the trailing variance σ_t^2 :

$$\sigma_t^2 = \omega + \beta\sigma_{t-1}^2 + \alpha r_t^2$$

If the simulated returns from the *GARCH(1,1)* model are used in the above formula, then it produces the simulated *GARCH(1,1)* variance.

But to estimate the trailing variance of historical returns, the parameters ω , α , and β must be first estimated through model calibration.

```
> # Simulate the GARCH process using Rcpp
> garchsim <- HighFreq::sim_garch(omega=omega, alpha=alphac,
+   beta=betac, innov=matrix(innov))
> # Extract the returns
> retp <- garchsim[, 1]
> # Estimate the trailing variance from the returns
> varv <- numeric(nrows)
> varv[1] <- omega/(1 - alphac - betac)
> for (i in 2:nrows) {
+   varv[i] <- omega + alphac*retp[i]^2 +
+     betac*varv[i-1]
+ } ## end for
> all.equal(garchsim[, 2], varv, check.attributes=FALSE)
```

GARCH Model Calibration

GARCH models can be calibrated from the returns using the *maximum-likelihood* method.

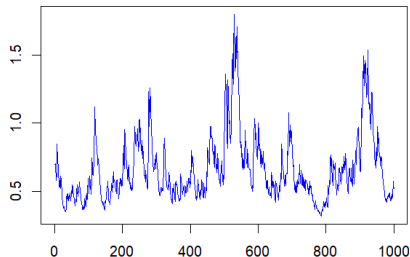
But it's a complex optimization procedure which requires a large amount of data for accurate results.

The function `fGarch::garchFit()` calibrates a *GARCH* model on a time series of returns.

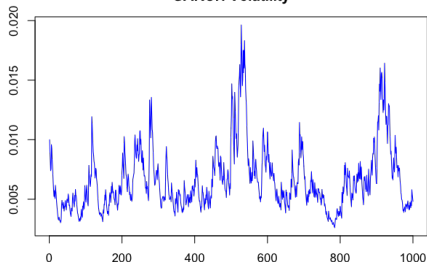
The function `garchFit()` returns an S4 object of class *fGARCH*, with multiple slots containing the *GARCH* model outputs and diagnostic information.

```
> library(fGarch)
> # Fit returns into GARCH
> garchfit <- fGarch::garchFit(data=retp)
> # Fitted GARCH parameters
> garchfit@fit$coef
> # Actual GARCH parameters
> c(mu=mean(retp), omega=omega,alpha=alphac, beta=betac)
> # Plot GARCH fitted volatility
> plot(sqrt(garchfit@fit$series$h), t="l",
+      col="blue", xlab="", ylab="",
+      main="GARCH Fitted Volatility")
> quartz.save("figure/garch_fGarch_fitted.png",
+      type="png", width=6, height=5)
```

GARCH fitted standard deviation



GARCH Volatility



GARCH Likelihood Function

Under the $GARCH(1,1)$ volatility model, the returns follow the process: $r_t = \sigma_{t-1}\xi_t$. (We can assume that the returns have been centered.)

So the *conditional* distribution of returns is normal with standard deviation equal to σ_{t-1} :

$$\phi(r_t, \sigma_{t-1}) = \frac{e^{-r_t^2/2\sigma_{t-1}^2}}{\sqrt{2\pi}\sigma_{t-1}}$$

The *log-likelihood* function $\mathcal{L}(\omega, \alpha, \beta | r_t)$ for the normally distributed returns is therefore equal to:

$$\mathcal{L}(\omega, \alpha, \beta | r_t) = - \sum_{t=1}^n \left(\frac{r_t^2}{\sigma_{t-1}^2} + \log(\sigma_{t-1}^2) \right)$$

The *log-likelihood* depends on the $GARCH(1,1)$ parameters ω , α , and β because the trailing variance σ_t^2 depends on the $GARCH(1,1)$ parameters:

$$\sigma_t^2 = \omega + \beta\sigma_{t-1}^2 + \alpha r_t^2$$

The $GARCH$ process must be simulated using an explicit loop, so it's better to perform it in C++ instead of R.

```
> # Define likelihood function
> likefun <- function(omega, alphac, betac) {
+   ## Estimate the trailing variance from the returns
+   varv <- numeric(nrows)
+   varv[1] <- omega/(1 - alphac - betac)
+   for (i in 2:nrows) {
+     varv[i] <- omega + alphac*ret[p[i]]^2 + betac*varv[i-1]
+   } ## end for
+   varv <- ifelse(varv > 0, varv, 0.000001)
+   ## Lag the variance
+   varv <- rutils::lagit(varv, pad_zeros=FALSE)
+   ## Calculate the likelihood
+   -sum(ret[p]^2/varv + log(varv))
+ } ## end likefun
> # Calculate the likelihood in R
> likefun(omega, alphac, betac)
> # Calculate the likelihood in Rcpp
> HighFreq::lik_garch(omega=omega, alpha=alphac,
+   beta=betac, returns=matrix(ret[p]))
> # Benchmark speed of likelihood calculations
> library(microbenchmark)
> summary(microbenchmark(
+   Rcode=likefun(omega, alphac, betac),
+   Rcpp=HighFreq::lik_garch(omega=omega, alpha=alphac, beta=betac,
+   ), times=10)[, c(1, 4, 5)])
```

GARCH Likelihood Function Matrix

The $GARCH(1,1)$ *log-likelihood* function depends on three parameters $\mathcal{L}(\omega, \alpha, \beta | r_t)$.

The more parameters the harder it is to find their optimal values using optimization.

We can simplify the optimization task by assuming that the expected variance is equal to the realized variance:

$$\sigma^2 = \frac{\omega}{1 - \alpha - \beta} = \frac{1}{n-1} \sum_{t=1}^n (r_t - \bar{r})^2$$

This way the *log-likelihood* becomes a function of only two parameters, say α and β .

```
> # Calculate the variance of returns
> retp <- garchsim[, 1, drop=FALSE]
> varv <- var(retp)
> retp <- (retp - mean(retp))
> # Calculate likelihood as function of alpha and betac parameters
> likefun <- function(alphac, betac) {
+   omega <- variance*(1 - alpha - betac)
+   -HighFreq::lik_garch(omega=omega, alpha=alphac, beta=betac, retp=retp)
+ } ## end likefun
> # Calculate matrix of likelihood values
> alphas <- seq(from=0.15, to=0.35, len=50)
> betac <- seq(from=0.35, to=0.5, len=50)
> likmat <- sapply(alphas, function(alphac) sapply(betac,
+   function(betac) likefun(alphac, betac)))
```

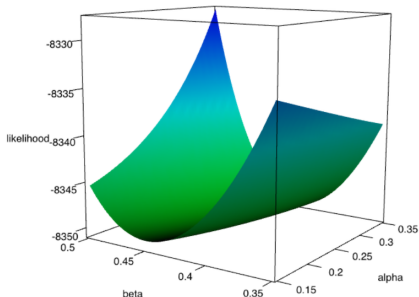
GARCH Likelihood Perspective Plot

The perspective plot shows that the *log-likelihood* is much more sensitive to the β parameter than to α .

The function `rgl::persp3d()` plots an *interactive* 3d surface plot of a *vectorized* function or a matrix.

The optimal values of α and β can be found approximately using a grid search on the *log-likelihood* matrix.

```
> # Set rgl options and load package rgl
> options(rgl.useNULL=TRUE); library(rgl)
> # Draw and render 3d surface plot of likelihood function
> ncols <- 100
> color <- rainbow(ncols, start=2/6, end=4/6)
> zcols <- cut(likmat, ncols)
> rgl::persp3d(alphaacs, betac, likmat, col=color[zcols],
+   xlab="alpha", ylab="beta", zlab="likelihood")
> rgl::rglwidget(elementId="plot3drgl", width=700, height=700)
> # Perform grid search
> coord <- which(likmat == min(likmat), arr.ind=TRUE)
> c(alphaacs[coord[2]], betac[coord[1]])
> likmat[coord]
> likefun(alphaacs[coord[2]], betac[coord[1]])
> # Optimal and actual parameters
> options(scipen=2) ## Use fixed not scientific notation
> cbind(actual=c(alpha=alphac, beta=betac, omega=omega),
+   optimal=c(alphaacs[coord[2]], betac[coord[1]], variance*(1 - sum(alphaacs[coord[2]], betac[coord[1]]))))
```



GARCH Likelihood Function Optimization

The flat shape of the *GARCH* likelihood function makes it difficult for steepest descent optimizers to find the best parameters.

The function `DEoptim()` from package *DEoptim* performs *global* optimization using the *Differential Evolution* algorithm.

Differential Evolution is a genetic algorithm which evolves a population of solutions over several generations:

<https://link.springer.com/content/pdf/10.1023/A:1008202821328.pdf>

The first generation of solutions is selected randomly.

Each new generation is obtained by combining the best solutions from the previous generation.

The *Differential Evolution* algorithm is well suited for very large multi-dimensional optimization problems, such as portfolio optimization.

Gradient optimization methods are more efficient than *Differential Evolution* for smooth objective functions with no local minima.

```
> # Define vectorized likelihood function
> likefun <- function(x, retp) {
+   alphac <- x[1]; betac <- x[2]; omega <- x[3]
+   -HighFreq::lik_garch(omega=omega, alpha=alphac, beta=betac, retp=retp)
+ } ## end likefun
> # Initial parameters
> initp <- c(alphac=0.2, beta=0.4, omega=varv/0.2)
> # Find max likelihood parameters using steepest descent optimizer
> fitobj <- optim(par=initp,
+   fn=likefun, ## Log-likelihood function
+   method="L-BFGS-B", ## Quasi-Newton method
+   returns=retp,
+   upper=c(0.35, 0.55, varv), ## Upper constraint
+   lower=c(0.15, 0.35, varv/100)) ## Lower constraint
> # Optimal and actual parameters
> cbind(actual=c(alphac=alphac, beta=betac, omega=omega),
+   optimal=c(fitobj$par["alpha"], fitobj$par["beta"], fitobj$par["omega"]))
> # Find max likelihood parameters using DEoptim
> optim1 <- DEoptim::DEoptim(fn=likefun,
+   upper=c(0.35, 0.55, varv), ## Upper constraint
+   lower=c(0.15, 0.35, varv/100), ## Lower constraint
+   returns=retp,
+   control=list(trace=FALSE, itermax=1000, parallelType=1))
> # Optimal and actual parameters
> cbind(actual=c(alphac=alphac, beta=betac, omega=omega),
+   optimal=c(optim1$optim$bestmem[1], optim1$optim$bestmem[2], optim1$optim$bestmem[3]))
```

GARCH Variance of Stock Returns

The *GARCH* model can be used to estimate the trailing variance of empirical (historical) returns.

If the time series of returns r_t is given, then it can be used in the *GARCH(1,1)* formula to estimate the trailing variance σ_t^2 :

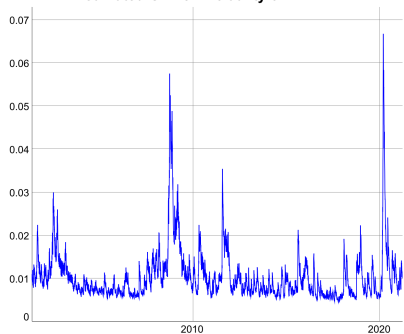
$$\sigma_t^2 = \omega + \beta\sigma_{t-1}^2 + \alpha r_t^2$$

The *GARCH* estimator of the trailing variance is a generalization of the exponential moving average (*EMA*) variance estimator:

$$\sigma_t^2 = \lambda\sigma_{t-1}^2 + (1 - \lambda)r_t^2$$

The main difference is that the *GARCH* model has a non-zero equilibrium value of the variance σ_{eq}^2 , while the *EMA* estimator decays to zero.

Estimated GARCH Volatility of VTI



```
> # Calculate VTI returns
> retp <- na.omit(rutils::etfenv$returns$VTI)
> # Find max likelihood parameters using DEoptim
> optim1 <- DEoptim::DEoptim(fn=likefun,
+   upper=c(0.4, 0.9, varv), ## Upper constraint
+   lower=c(0.1, 0.5, varv/100), ## Lower constraint
+   returns=retp,
+   control=list(trace=FALSE, itermax=1000, parallelType=1))
> # Optimal parameters
> paramv <- unname(optim1$optim$bestmem)
> alphac <- paramv[1]; betac <- paramv[2]; omega <- paramv[3]
> c(alphac, betac, omega)
> # Equilibrium GARCH variance
> omega/(1 - alphac - betac)
> drop(var(retp))
```

```
> # Estimate the GARCH volatility of VTI returns
> nrow <- NROW(retp)
> varv <- numeric(nrow)
> varv[1] <- omega/(1 - alphac - betac)
> for (i in 2:nrow) {
+   varv[i] <- omega + alphac*retp[i]^2 + betac*varv[i-1]
+ } ## end for
> # Estimate the GARCH volatility using Rcpp
> garchsim <- HighFreq::sim_garch(omega=omega, alpha=alphac,
+   beta=betac, innov=retp, is_random=FALSE)
> all.equal(garchsim[, 2], varv, check.attributes=FALSE)
> # Plot dygraph of the estimated GARCH volatility
> dygraphs::dygraph(xts::xts(sqrt(varv), zoo::index(retp)),
+   main="Estimated GARCH Volatility of VTI") %>%
+   dyOptions(colors="blue") %>% dvLegend(show="always". width=300)
```

GARCH Variance Forecasts

The *GARCH* model can't forecast the volatility spikes. It only forecasts the exponential decay of the volatility after a spike.

The one-step-ahead forecast of the squared returns is equal to their expected value: $r_{t+1}^2 = \mathbb{E}[(\sigma_t \xi_t)^2] = \sigma_t^2$.

The variance forecasts depend on the previous variance: $\sigma_{t+1}^2 = \mathbb{E}[\omega + \alpha r_{t+1}^2 + \beta \sigma_t^2] = \omega + (\alpha + \beta) \sigma_t^2$

The variance forecasts gradually decay to the equilibrium value σ_{eq}^2 , such that the forecast is equal to itself: $\sigma_{eq}^2 = \omega + (\alpha + \beta) \sigma_{eq}^2$.

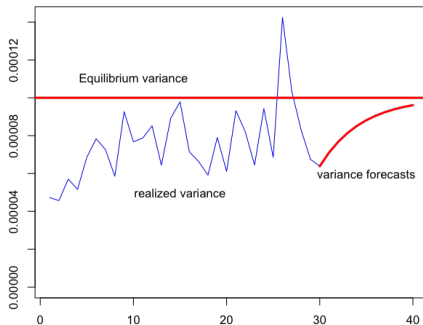
This gives: $\sigma_{eq}^2 = \frac{\omega}{1 - \alpha - \beta}$, which is the long-term expected value of the variance.

So the variance forecasts decay exponentially to their equilibrium value σ^2 at the decay rate equal to $(\alpha + \beta)$:

$$\sigma_{t+1}^2 - \sigma_{eq}^2 = (\alpha + \beta)(\sigma_t^2 - \sigma_{eq}^2)$$

```
> # Simulate GARCH model
> garchsim <- HighFreq::sim_garch(omega=omega, alpha=alphac,
+   beta=betac, innov=matrix(innov))
> varv <- garchsim[, 2]
> # Calculate the equilibrium variance
> vareq <- omega/(1 - alphac - betac)
> # Calculate the variance forecasts
> varf <- numeric(10)
> varf[1] <- vareq + (alphac + betac)*(xts::last(varv) - vareq)
> for (i in 2:10) {
+   varf[i] <- vareq + (alphac + betac)*(varf[i-1] - vareq)
+ } ## end for
```

GARCH Variance Forecasts



```
> # Open plot window on Mac
> dev.new(width=6, height=5, noRStudioGD=TRUE)
> par(mar=c(2, 2, 3, 1), oma=c(0, 0, 0, 0))
> # Plot GARCH variance forecasts
> plot(tail(varv, 30), t="l", col="blue", xlab="", ylab="",
+   xlim=c(1, 40), ylim=c(0, max(tail(varv, 30))),
+   main="GARCH Variance Forecasts")
> text(x=15, y=0.5*vareq, "realized variance")
> lines(x=30:40, y=c(xts::last(varv), varf), col="red", lwd=3)
> text(x=35, y=0.6*vareq, "variance forecasts")
> abline(h=vareq, lwd=3, col="red")
> text(x=10, y=1.1*vareq, "Equilibrium variance")
> quartz.save("figure/garch_forecast.png", type="png", width=6, height=5)
```

depr: old stuff about Estimating Volatility of Intraday Time Series

The *close-to-close* estimator depends on *Close* prices specified over the aggregation intervals:

$$\hat{\sigma}^2 = \frac{1}{n-1} \sum_{i=1}^n \left(\log\left(\frac{C_i}{C_{i-1}}\right) - \bar{r} \right)^2$$

$$\bar{r} = \frac{1}{n} \sum_{i=1}^n \log\left(\frac{C_i}{C_{i-1}}\right)$$

Volatility estimates for intraday time series depend both on the units of returns (per second, minute, day, etc.), and on the aggregation interval (secondly, minutely, daily, etc.)

A minutely time interval is equal to 60 seconds, a daily time interval is equal to $24 \cdot 60 \cdot 60 = 86,400$ seconds.

For example, it's possible to measure returns in minutely intervals in units per second.

The estimated volatility is directly proportional to the measurement units.

For example, the volatility estimated from per minute returns is 60 times the volatility estimated from per second returns.

```
> library(HighFreq) ## Load HighFreq
> # Minutely SPY returns (unit per minute) single day
> retspy <- rutils::diffit(log(SPY["2012-02-13", 4]))
> # Minutely SPY volatility (unit per minute)
> sd(retspy)
> # Divide minutely SPY returns by time intervals (unit per second)
> retspy <- retspy/rutils::diffit(xts::index(SPY["2012-02-13"]))
> retspy[1] <- 0
> # Minutely SPY volatility scaled to unit per minute
> 60*sd(retspy)
> # SPY returns multiple days
> retspy <- rutils::diffit(log(SPY[, 4]))
> # Minutely SPY volatility (includes overnight jumps)
> sd(retspy)
> # Table of intervals - 60 second is most frequent
> indeks <- rutils::diffit(xts::index(SPY))
> table(indeks)
> # hist(indeks)
> # SPY returns with overnight scaling (unit per second)
> retspy <- retspy/indeks
> retspy[1] <- 0
> # Minutely SPY volatility scaled to unit per minute
> 60*sd(retspy)
```

draft: Comparing Range Volatility

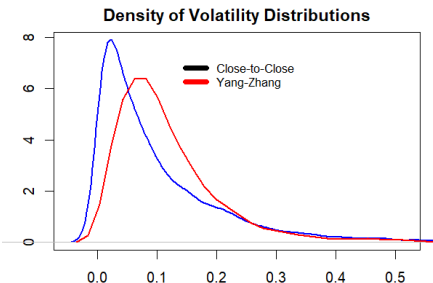
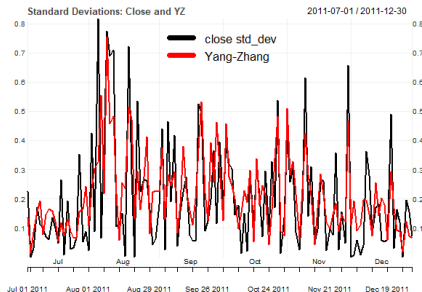
The range volatility estimators have much lower variability (standard errors) than the standard *Close-to-Close* estimator.

Is the above correct? Because the plot shows otherwise.

The range volatility estimators follow the standard *Close-to-Close* estimator, except in intervals of high intra-period volatility.

During the May 6, 2010 *flash crash*, range volatility spiked more than the *Close-to-Close* volatility.

```
> library(HighFreq) ## Load HighFreq
> ohlc <- log(rutils::etfenv$VTI)
> # Calculate variance
> varcl <- HighFreq::run_variance(ohlc=ohlc,
+   method="close")
> var_yang_zhang <- HighFreq::run_variance(ohlc=ohlc)
> stdev <- 24*60*60*sqrt(252*cbind(varcl, var_yang_zhang))
> colnames(stdev) <- c("close stdev", "Yang-Zhang")
> # Plot the time series of volatility
> themev <- chart_theme()
> themev$col$line.col <- c("black", "red")
> quantmod::chart_Series(stdev["2011-07/2011-12"],
+   theme=themev, name="Standard Deviations: Close and YZ")
> legend("top", legend=colnames(stdev), y.intersp=0.4,
+   bg="white", lty=1, lwd=6, inset=0.1, cex=0.8,
+   col=themev$col$line.col, bty="n")
> # Plot volatility around 2010 flash crash
> quantmod::chart_Series(stdev["2010-04/2010-06"],
+   theme=themev, name="Volatility Around 2010 Flash Crash")
> legend("top", legend=colnames(stdev), y.intersp=0.4,
+   bg="white", lty=1, lwd=6, inset=0.1, cex=0.8,
+   col=themev$col$line.col, bty="n")
> # Plot density of volatility distributions
```



draft: Log-range Volatility Proxies

To-do: plot time series of *intra-day range* volatility estimator and standard close-to-close volatility estimator. Emphasize flash-crash of 2010.

An alternative range volatility estimator can be created by calculating the logarithm of the range, (as opposed to the range percentage, or the logarithm of the price ratios).

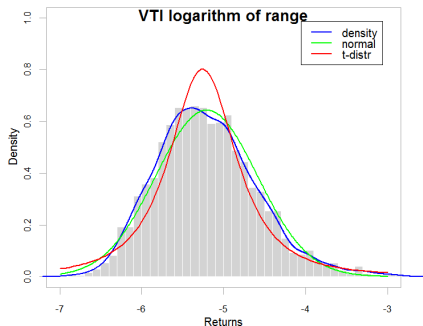
To-do: plot scatterplot of *intra-day range* volatility estimator and standard close-to-close volatility estimator.

Emphasize the two are different: the intra-day range volatility estimator captures volatility events which aren't captured by close-to-close volatility estimator, and vice versa.

$$\hat{\sigma}^2 = \frac{1}{n} \sum_{i=1}^n \log\left(\frac{H_i - L_i}{H_i + L_i}\right)^2$$

The range logarithm fits better into the normal distribution than the range percentage.

```
> ohlc <- rutils::etfenv$VTI
> retp <- log((ohlc[, 2] - ohlc[, 3]) / (ohlc[, 2] + ohlc[, 3]))
> foo <- rutils::diffit(log(ohlc[, 4]))
> plot(as.numeric(foo)^2, as.numeric(retp)^2)
> bar <- lm(retp ~ foo)
> summary(bar)
>
> # Perform normality tests
```



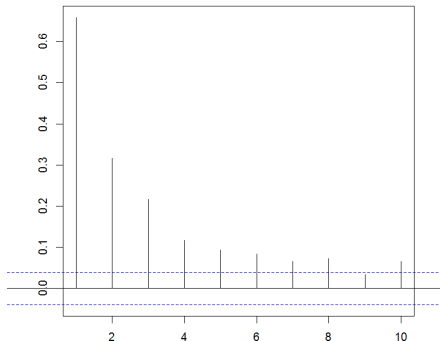
```
> # Plot histogram of VTI returns
> colorv <- c("lightgray", "blue", "green", "red")
> PerformanceAnalytics::chart.Histogram(retp,
+   main="", xlim=c(-7, -3), col=colorv[1:3],
+   methods = c("add.density", "add.normal"))
> curve(expr=dt((x-fitobj$estimate[1])/
+   fitobj$estimate[2], df=2)/fitobj$estimate[2],
+   type="l", xlab="", ylab="", lwd=2,
+   col=colorv[4], add=TRUE)
> # Add title and legend
> title(main="VTI logarithm of range",
+   cex.main=1.3, line=-1)
> legend("topright", inset=0.05, y.intersp=0.4,
+   legend=c("density", "normal", "t-distr"),
+   lwd=6, lty=1, col=colorv[2:4], bty="n")
```

draft: Autocorrelations of Alternative *Range* Estimators

The logarithm of the range exhibits very significant autocorrelations, unlike the range percentage.

```
> # Calculate VTI range variance partial autocorrelations  
> pacf(retp^2, lag=10, xlab=NA, ylab=NA,  
+     main="PACF of VTI log range")  
> quantmod::chart_Series(retp^2, name="VTI log of range squared")
```

PACF of VTI log range



depr: Standard Errors of Volatility Estimators Using Bootstrap

The standard errors of estimators can be calculated using a *bootstrap* simulation.

The *bootstrap* procedure generates new data by randomly sampling with replacement from the observed data set.

The *bootstrapped* data is then used to recalculate the estimator many times, producing a vector of values.

The *bootstrapped* estimator values can then be used to calculate the probability distribution of the estimator and its standard error.

Bootstrapping doesn't provide accurate estimates for estimators that are sensitive to the ordering and correlations in the data.

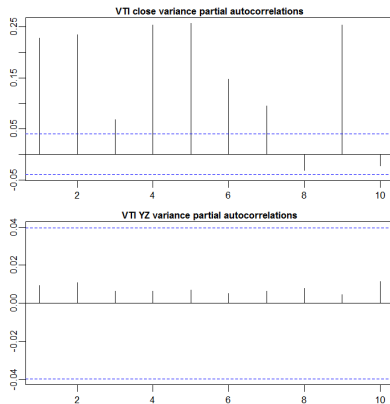
```
> # Standard errors of variance estimators using bootstrap
> bootd <- sapply(1:1e2, function(x) {
+   ## Create random OHLC
+   ohlc <- HighFreq::random_ohlc()
+   ## Calculate variance estimate
+   c(var=var(ohlc[, 4]),
+     yang_zhang=HighFreq::calcvariance(
+       ohlc, method="yang_zhang", scalev=FALSE))
+ }) ## end sapply
> # Analyze bootstrapped variance
> bootd <- t(bootd)
> head(bootd)
> colMeans(bootd)
> apply(bootd, MARGIN=2, sd) /
+   colMeans(bootd)
```

draft: Autocorrelations of *Close-to-Close* and *Range* Variances

The standard *Close-to-Close* estimator exhibits very significant autocorrelations, but the *range* estimators are not autocorrelated.

That is because the time series of squared intra-period ranges is not autocorrelated.

```
> # Close variance estimator partial autocorrelations
> pacf(varcl, lag=10, xlab=NA, ylab=NA)
> title(main="VTI close variance partial autocorrelations")
>
> # Range variance estimator partial autocorrelations
> pacf(var_yang_zhang, lag=10, xlab=NA, ylab=NA)
> title(main="VTI YZ variance partial autocorrelations")
>
> # Squared range partial autocorrelations
> retp <- log(rutils::etfenv$VTI[,2] /
+           rutils::etfenv$VTI[,3])
> pacf(retp^2, lag=10, xlab=NA, ylab=NA)
> title(main="VTI squared range partial autocorrelations")
```



Single Period Binary Gamble

Consider a single investment (gamble) with a binary outcome:

The investor makes no up-front payments, and either wins an amount a (with probability p), or loses an amount b (with probability $q = 1 - p$).

	win	lose
probability	p	$q = 1 - p$
payout	a	$-b$
terminal wealth	$1 + a$	$1 - b$

The initial wealth is equal to 1 dollar, and the terminal wealth after the gamble is either $1 + a$ (with probability p), or $1 - b$ (with probability $q = 1 - p$).

The amounts a and b are expressed as percentages of the wealth risked in the gamble, and the ratio a/b is called the *betting odds*.

The expected return on the gamble is called the *edge* and is equal to: $\mu = p a - q b$, and the variance of returns is equal to: $\sigma^2 = p q (a + b)^2$.

If the investor chooses to risk only a fraction k_f of wealth, then the return on the gamble is either $k_f a$ (with probability p), or $-k_f b$ (with probability $q = 1 - p$).

The fraction k_f can be greater than 1 (leveraged investing), or it can be negative (shorting).

And the expected return on the gamble is equal to: $p k_f a - q k_f b = k_f \mu$.

If an investor makes decisions exclusively based on the expected return μ , then they would either invest all their wealth ($k_f = 1$) on the gamble if $\mu > 0$, or choose not to invest at all ($k_f = 0$) if $\mu < 0$.

Without loss of generality we can assume that $p = q = \frac{1}{2}$.

And then $\mu = 0.5(a - b)$, and $\sigma^2 = 0.25(a + b)^2$.

The *Sharpe ratio* of the gamble is then equal to:

$$S_r = \frac{\mu}{\sigma} = \frac{(a - b)}{(a + b)}$$

Investor Utility and Fractional Betting

The *expected utility* hypothesis states that investors try to maximize the expected *utility* of wealth, not the expected wealth.

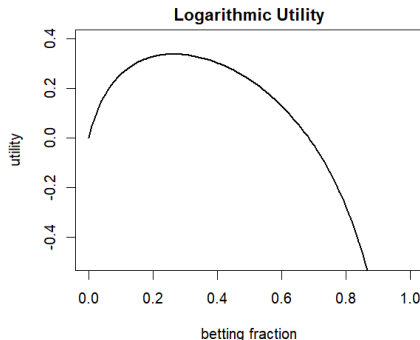
In 1738 Daniel Bernoulli introduced the concept of *logarithmic utility* in his work "*Specimen Theoriae Novae de Mensura Sortis*" (New Theory of the Measurement of Risk).

The *logarithmic utility* function is defined as the logarithm of wealth: $u(w) = \log(w)$.

Under *logarithmic utility* investor preferences depend on the percentage change of wealth, instead of the absolute change of wealth: $du(w) = \frac{dw}{w}$.

An investor with *logarithmic utility* invests only a fraction k_f of their wealth in a gamble, depending on the risk-return of the gamble.

If the initial wealth is equal to 1, then the expected value of *logarithmic utility* for the binary gamble is equal to: $u(k_f) = p \log(1 + k_f a) + q \log(1 - k_f b)$.



```
> # Define logarithmic utility
> utilfun <- function(frac, p=0.3, a=20, b=1) {
+   p*log(1+frac*a) + (1-p)*log(1-frac*b)
+ } ## end utilfun
> # Plot utility
> curve(expr=utilfun, xlim=c(0, 1),
+   ylim=c(-0.5, 0.4), xlab="betting fraction",
+   ylab="utility", main="", lwd=2)
> title(main="Logarithmic Utility", line=0.5)
```

Optimal Fractional Betting Under Logarithmic Utility

The betting fraction that maximizes the *utility* can be found by equating the derivative of *utility* to zero:

$$\frac{du(k_f)}{dk_f} = \frac{p a}{1 + k_f a} - \frac{q b}{1 - k_f b} = 0$$

$$k_f = \frac{p}{b} - \frac{q}{a} = \frac{p a - q b}{b a} = \frac{\mu}{b a}$$

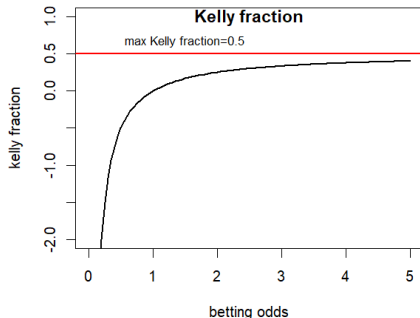
The optimal k_f is called the *Kelly fraction*, and it depends on the parameters of the gamble.

The *Kelly fraction* can be greater than 1 (leveraged investing), or it can be negative (shorting).

If we assume that $b = 1$, then the betting odds are equal to a and the *Kelly fraction* is: $k_f = \frac{p(a+1)-1}{a}$

The *Kelly fraction* is then equal to the expected payout divided by the betting odds.

If the expected payout of the gamble is not positive, then an investor with logarithmic utility should not allocate any capital to the gamble.



```
> # Define and plot Kelly fraction
> kelly_frac <- function(a, p=0.5, b=1) {
+   p/b - (1-p)/a
+ } ## end kelly_frac
> curve(expr=kelly_frac, xlim=c(0, 5),
+       ylim=c(-2, 1), xlab="betting odds",
+       ylab="kelly fraction", main="", lwd=2)
> abline(h=0.5, lwd=2, col="red")
> text(x=1.5, y=0.5, pos=3, cex=0.8, labels="max Kelly fraction=0.5")
> title(main="Kelly fraction", line=-0.8)
```

The Kelly Criterion

The *Kelly criterion* states that investors should bet the optimal *Kelly fraction* of their capital in a gamble.

Investors with concave utility functions (for example logarithmic utility) are sensitive to the risk of ruin (losing all their capital).

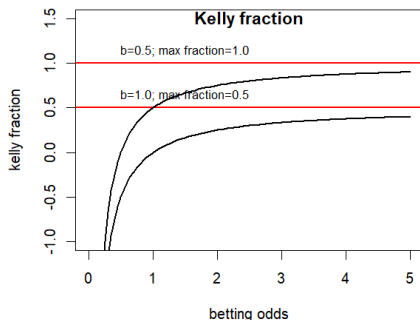
Applying the *Kelly criterion* and betting only a fraction of their capital reduces the risk of ruin (but it doesn't eliminate the risk if prices drop suddenly).

The loss amount b determines the risk of ruin, with larger values of b increasing the risk of ruin.

Therefore investors will choose a smaller betting fraction k_f for larger values of b .

This means that even for huge odds in their favor, investors may not choose to invest all their capital, because of the risk of ruin.

For example, if the betting odds are very large $a \rightarrow \infty$, then the *Kelly fraction*: $k_f = \frac{p}{b}$.



```
> # Plot several Kelly curves
> curve(expr=kelly_frac(x, b=1), xlim=c(0, 5),
+       ylim=c(-1, 1.5), xlab="betting odds",
+       ylab="kelly fraction", main="", lwd=2)
> abline(h=0.5, lwd=2, col="red")
> text(x=1.5, y=0.5, pos=3, cex=0.8, labels="b=1.0; max fraction=0.5")
> curve(expr=kelly_frac(x, b=0.5), add=TRUE, main="", lwd=2)
> abline(h=1.0, lwd=2, col="red")
> text(x=1.5, y=1.0, pos=3, cex=0.8, labels="b=0.5; max fraction=1.0")
> title(main="Kelly fraction", line=-0.8)
```

Utility of Multiperiod Betting

Let r_t be the random return on the gamble in period i , and let $w_i = (1 + k_f r_t)$ be the random wealth increment.

Then the terminal wealth after n rounds is equal to the compounded wealth increments:

$$w_n = \prod_{i=1}^n w_i = \prod_{i=1}^n (1 + k_f r_t).$$

And the utility is equal to the sum of the individual utilities:

$$u_n = \log(w_n) = \sum_{i=1}^n \log(w_i) = \sum_{i=1}^n \log(1 + k_f r_t) = \sum_{i=1}^n u_i$$

The individual utilities are all maximized by the same *Kelly fraction* k_f , so the *Kelly fraction* for multiperiod betting is the same as for single period betting:

$$k_f = \frac{p}{b} - \frac{q}{a}$$

Wealth of Multiperiod Betting

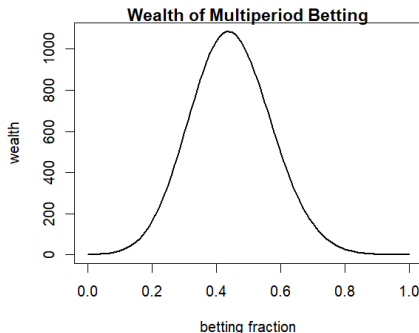
In multiperiod betting the investor participates in n rounds of gambles, and in each round they risk a fixed fraction k_f of their current outstanding wealth.

In each round the wealth is multiplied by either $(1 + k_f a)$ (win) or $(1 - k_f b)$ (loss), so that the current outstanding wealth changes over time.

The terminal wealth after n rounds with m wins is equal to: $w(k_f) = (1 + k_f a)^m (1 - k_f b)^{n-m}$.

If the number of rounds n is very large, then the number of wins is almost always equal to $m = np$, and the terminal wealth is equal to:

$$w(k_f) = (1 + k_f a)^{np} (1 - k_f b)^{nq}.$$



```
> # Wealth of multiperiod binary betting
> wealthv <- function(f, a=0.8, b=0.1, n=1e3, i=150) {
+   (1+f*a)^i * (1-f*b)^(n-i)
+ } ## end wealth
> curve(expr=wealthv, xlim=c(0, 1),
+ xlab="betting fraction",
+ ylab="wealth", main="", lwd=2)
> title(main="Wealth of Multiperiod Betting", line=0.1)
```

Optimal Multiperiod Betting

The betting fraction k_f that maximizes the terminal wealth is found by setting the derivative of $w(k_f)$ to zero:

$$\begin{aligned}\frac{dw(k_f)}{dk_f} &= npa(1 + k_f a)^{np-1}(1 - k_f b)^{nq} - nqb(1 + k_f a)^{np}(1 - k_f b)^{nq-1} \\ &= \left(\frac{npa}{1 + k_f a} - \frac{nqb}{1 - k_f b} \right) (1 + k_f a)^{np-1} (1 - k_f b)^{nq} = 0\end{aligned}$$

We can then solve for the optimal betting fraction k_f :

$$\begin{aligned}\frac{pa}{1 + k_f a} - \frac{qb}{1 - k_f b} &= 0 \\ pa(1 - k_f b) - qb(1 + k_f a) &= 0 \\ pa - qb - k_f ab &= 0 \\ k_f &= \frac{pa - qb}{ab} = \frac{p}{b} - \frac{q}{a}\end{aligned}$$

The above is just the *Kelly fraction* k_f that maximizes the utility.

So the *Kelly fraction* k_f that maximizes the utility also maximizes the terminal wealth.

depr: Multiperiod Binary Gambles

The terminal wealth after n repeated gambles with m wins is equal to: $(1 + k_f a)^m (1 - k_f b)^{n-m}$.

And the expected value of the wealth is equal to:

$$w(k_f) = \sum_{m=0}^n \binom{n}{m} p^m q^{n-m} (1 + k_f a)^m (1 - k_f b)^{n-m}$$

We can then find the fraction k_f which maximizes the expected wealth $w(k_f)$:

$$\begin{aligned} \frac{dw(k_f)}{dk_f} &= \sum_{m=0}^n \binom{n}{m} p^m q^{n-m} (1 + k_f a)^m (1 - k_f b)^{n-m} \left(\frac{am}{1 + k_f a} - \frac{b(n-m)}{1 - k_f b} \right) = \\ &\quad \frac{a}{1 + k_f a} \sum_{m=0}^n \binom{n}{m} p^m q^{n-m} m - \\ &\quad \sum_{m=0}^n \binom{n}{m} p^m q^{n-m} (1 + k_f a)^m (1 - k_f b)^{n-m} \left(\frac{am}{1 + k_f a} - \frac{b(n-m)}{1 - k_f b} \right) \end{aligned}$$

If the investor chooses to risk only a fraction k_f of wealth, then the wealth after the gamble is either $1 + k_f a$ (with probability p), or $1 - k_f b$ (with probability $q = 1 - p$).

(with probability p), or $1 - b$ (with probability $q = 1 - p$). initial wealth is equal to 1, and the The *Kelly fraction* for multiperiod betting can be found by maximizing the expected *utility* of the final wealth distribution:

$$u(k_f) = \sum_{m=0}^n \binom{n}{m} p^m q^{n-m} \log((1 + k_f a)^m (1 - k_f b)^{n-m})$$

depr: Utility of Multiperiod Binary Gambles

The *Kelly fraction* for multiperiod betting can be found by maximizing the expected *utility* of the final wealth distribution:

$$\begin{aligned}
 u(k_f) &= \sum_{m=0}^n \binom{n}{m} p^m q^{n-m} \log((1 + k_f a)^m (1 - k_f b)^{n-m}) \\
 &= \log(1 + k_f a) \sum_{m=0}^n \binom{n}{m} p^m q^{n-m} m + \\
 &\quad \log(1 - k_f b) \sum_{m=0}^n \binom{n}{m} p^m q^{n-m} (n - m) \\
 &= n p \log(1 + k_f a) + n q \log(1 - k_f b)
 \end{aligned}$$

The above is just the single period *utility* multiplied by the number of rounds of betting n .

The *Kelly fraction* k_f for multiperiod betting is the same as for single period betting:

$$k_f = \frac{p}{b} - \frac{q}{a}$$

Investing With Fixed Margin

Let r_t be the percentage returns on a *risky asset*, so that the asset price p_t at time t is given by:

$$p_t = p_0 \prod_{i=1}^t (1 + r_i)$$

The initial investor wealth at time $t = 0$ is equal to 1 dollar, and they also borrow on margin m dollars to invest in the *risky asset*.

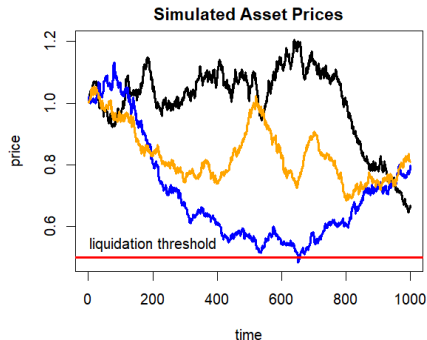
The investor's *wealth* at time t is equal to (the margin borrowing rate is assumed to be zero):

$$w_t = 1 + m \frac{p_t - p_0}{p_0}$$

The *leverage* k_f is equal to the *margin debt* m divided by the total wealth w_t : $k_f = m/w_t$.

If the asset price drops then the *leverage* increases, because the *margin debt* is fixed while the wealth drops.

If the asset price drops enough so that the wealth reaches zero, then the investment is liquidated and the investor is ruined.



```
> set.seed(1121, "Mersenne-Twister", sample.kind="Rejection")
> # Simulate asset prices
> calc_pricev <- function(x) cumprod(1 + rnorm(1e3, sd=0.01))
> price_paths <- sapply(1:3, calc_pricev)
> plot(price_paths[, 1], type="l", lwd=3,
+      main="Simulated Asset Prices",
+      ylim=range(price_paths),
+      lty="solid", xlab="time", ylab="price")
> lines(price_paths[, 2], col="blue", lwd=3)
> lines(price_paths[, 3], col="orange", lwd=3)
> abline(h=0.5, col="red", lwd=3)
> text(x=200, y=0.5, pos=3, labels="liquidation threshold")
```

Investing With Fixed Leverage

In order to avoid ruin, the investor may choose to maintain a fixed *leverage ratio* equal to k_f , so that the amount invested in the *risky asset* is proportional to the *wealth*: $k_f w_t$.

This requires buying the *risky asset* when its price increases, and selling it when it drops.

The return on the *risky asset* in a single period is equal to: $k_f w_t r_t$, so the *terminal wealth* at time t is equal to the compounded returns:

$$w_t = (1 + k_f r_1) \dots (1 + k_f r_t) = \prod_{i=1}^t (1 + k_f r_i)$$

The utility of the *terminal wealth* is equal to the sum of the utilities of single periods:

$$\begin{aligned} \mathbb{E}[\log w_t] &= \mathbb{E}[\log((1 + k_f r_1) \dots (1 + k_f r_t))] \\ &= \sum_{i=1}^t \mathbb{E}[\log(1 + k_f r_i)] = t \mathbb{E}[\log(1 + k_f r)] \end{aligned}$$

The last equality holds because all the utilities of single periods are the same.

Let the returns over a short time period be equal to r , with probability distribution $p(r)$.

The mean return $\bar{r}$, and variance σ^2 are:

$$\bar{r} = \int r p(r) dr ; \quad \sigma^2 = \int (r - \bar{r})^2 p(r) dr$$

Since the returns are over a short time period, we have: $r \ll 1$ and $\bar{r} \ll \sigma$, so that we can replace $r - \bar{r}$ with r as follows:

$$\int (r - \bar{r})^2 p(r) dr \approx \int r^2 p(r) dr$$

Utility of Leveraged Asset Returns

So the utility of the *terminal wealth* u_t is equal to the utility of a single period times the number of periods:

$$u_t = \mathbb{E}[\log w_t] = t \mathbb{E}[\log (1 + k_f r)] = t u_r$$

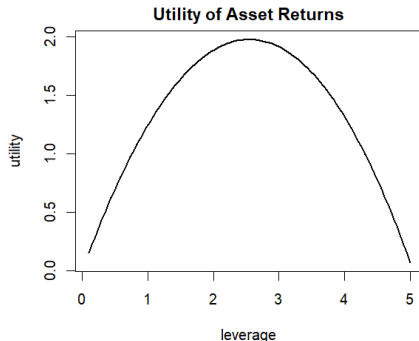
The utility of the asset returns u_r is equal to:

$$u_r = \mathbb{E}[\log (1 + k_f r)] = \int \log(1 + k_f r) p(r) dr$$

The leverage k_f is limited so that $(1 + k_f r) > 0$ for all return values r .

If the mean returns are positive, then at first the utility increases with leverage, but only up to a point.

With higher leverage, the negative utility of the time periods with negative returns becomes significant, forcing the aggregate utility to drop.



```
> # Calculate the VTI returns
> retp <- rutils::etfenv$returns$VTI
> retp <- na.omit(retp)
> c(mean=mean(retp), stdev=sd(retp))
> range(retp)
```

```
> # Define vectorized logarithmic utility function
> utilfun <- function(kellyfrac, retp) {
+   sapply(kellyfrac, function(x) sum(log(1 + x*retp)))
+ } ## end utilfun
> utilfun(1, retp)
> utilfun(c(1, 4), retp)
> # Plot the logarithmic utility
> curve(expr=utilfun(x, retp=retp),
+ xlim=c(0.1, 5), xlab="leverage", ylab="utility",
+ main="Utility of Asset Returns", lwd=2)
```

Kelly Criterion for Optimal Leverage of Asset Returns

The *logarithmic utility* u_r can be expanded in the moments of the return distribution:

$$\begin{aligned} u_r &= \mathbb{E}[\log(1 + k_f r)] = \int \log(1 + k_f r) p(r) dr \\ &= \int \left(k_f r - \frac{(k_f r)^2}{2} + \frac{(k_f r)^3}{3} - \frac{(k_f r)^4}{4} \right) p(r) dr \\ &= k_f \bar{r} - \frac{k_f^2 \sigma^2}{2} + \frac{k_f^3 \sigma^3 \varsigma}{3} - \frac{k_f^4 \sigma^4 \kappa}{4} \end{aligned}$$

Where $\varsigma = \int \frac{r^3}{\sigma^3} p(r) dr$ is the *skewness*, and
 $\kappa = \int \frac{r^4}{\sigma^4} p(r) dr$ is the *kurtosis*.

The *Kelly leverage* which maximizes the *utility* is found by equating the derivative of *utility* to zero:

$$\frac{du_r}{dk_f} = \bar{r} - k_f \sigma^2 + k_f^2 \sigma^3 \varsigma - k_f^3 \sigma^4 \kappa = 0$$

This shows that the logarithmic utility has positive odd derivatives and negative even derivatives.

Assuming that the third and fourth moments $\sigma^4 \varsigma$ and $\sigma^4 \kappa$ are small and can be neglected, we get:

$$k_f = \frac{\bar{r}}{\sigma^2} = \frac{S_r}{\sigma} ; \quad u_r = \frac{1}{2} \frac{\bar{r}^2}{\sigma^2} = \frac{1}{2} S_r^2$$

The *Kelly leverage* is *approximately* equal to the *Sharpe ratio* divided by the *standard deviation*.

The optimal utility u_r is *approximately* equal to half the *Sharpe ratio* S_r squared.

The *standard deviation* and *Sharpe ratio* are calculated over the same time interval as the returns (not annualized).

```
> # Approximate Kelly leverage
> mean(retp)/var(retp)
> PerformanceAnalytics::KellyRatio(R=retp, method="full")
> # Kelly leverage
> unlist(optimize(
+   f=function(x) ~utilfun(x, retp),
+   interval=c(1, 4)))
```

Kelly Strategy Wealth Path

The wealth of a Kelly Strategy with a fixed leverage ratio k_f is equal to:

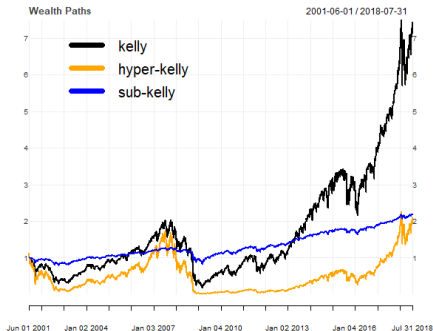
$$w_t = \prod_{i=1}^t (1 + k_f r_t)$$

The *Kelly fraction* k_f provides the optimal leverage to maximize the utility of wealth, by balancing the benefit of leveraging higher positive returns, with the risk of ruin due to excessive leverage.

If the mean asset returns are positive, then a higher leverage ratio provides higher returns.

But if the leverage is too high, then the losses in periods with negative returns wipe out most of the wealth, so then it's slow to recover.

```
> retp <- na.omit(retp)
> # Calculate the wealth paths
> kelly_ratio <- drop(mean(retp)/var(retp))
> kelly_wealthv <- cumprod(1 + kelly_ratio*retp)
> hyper_kelly <- cumprod(1 + (kelly_ratio+2)*retp)
> sub_kelly <- cumprod(1 + (kelly_ratio-2)*retp)
> kelly_paths <- cbind(kelly_wealth, hyper_kelly, sub_kelly)
> colnames(kelly_paths) <- c("kelly", "hyper-kelly", "sub-kelly")
```



```
> # Plot wealth paths
> themev <- chart_theme()
> themev$col$line.col <- c("black", "orange", "blue")
> quantmod::chart_Series(kelly_paths, theme=themev, name="Wealth Paths")
> legend("topleft", legend=colnames(kelly_paths),
+ inset=0.1, bg="white", lty=1, lwd=6, y.intersp=0.5,
+ col=themev$col$line.col, bty="n")
```

Kelly Strategy With Margin Account

The *margin debt* m_t is equal to the dollar amount borrowed to purchase the *risky asset*.

The wealth w_t at time t is equal to the initial wealth $w_0 = 1$ plus the dollar amount of the *risky asset* a_t , minus the *margin debt* m_t : $w_t = 1 + a_t - m_t$.

The dollar amount of the *risky asset* a_t is equal to the wealth w_t times the *leverage* k_f : $a_t = k_f w_t$.

So the *margin debt* m_t is proportional to the wealth w_t : $m_t = (k_f - 1)w_t + 1$.

The wealth changes from w_{t-1} to:

$w_t = w_{t-1}(1 + k_f r_t)$, while the dollar amount of the *risky asset* changes from $a_{t-1} = k_f w_{t-1}$ to:

$a_t = k_f w_{t-1}(1 + r_t)$, so that the leverage changes from k_f to:

$$\frac{k_f w_{t-1}(1 + r_t)}{w_{t-1}(1 + k_f r_t)} = \frac{k_f(1 + r_t)}{1 + k_f r_t}$$

In order to maintain a fixed *leverage ratio* equal to k_f , the investor must actively trade the *risky asset*, and the *margin debt* m_t changes over time.

The change in margin in a single time period is equal to:

$$\Delta m_t = (k_f - 1)\Delta w_t = k_f(k_f - 1)w_{t-1}r_t$$

The dollar amount of the *risky asset* traded is equal to the change in *margin*.

Therefore the investor must borrow on margin and buy the *risky asset* when its price increases, and sell it when it drops.

Kelly Strategy With Transaction Costs of Trading

The *bid-ask spread* is the percentage difference between the *offer* minus the *bid* price, divided by the *mid* price.

The *bid-ask spread* for liquid stocks can be assumed to be about 10 basis points (bps).

The *transaction costs* c^r due to the *bid-ask spread* are equal to half the *bid-ask spread* δ times the absolute value of the traded dollar amount of the *risky asset*:

$$c^r = \frac{\delta}{2} |\Delta m_t|$$

If the transaction costs are much less than the change in wealth $c^r \ll |\Delta w_t|$, then we can write approximately:

$$c^r = \frac{\delta}{2} k_f (k_f - 1) w_{t-1} |r_t|$$

The wealth of the Kelly Strategy after accounting for the *bid-ask spread* is then equal to:

$$w_t = \prod_{i=1}^t (1 + k_f r_t - \frac{\delta}{2} k_f (k_f - 1) |r_t|)$$

The effect of the *bid-ask spread* is to reduce the effective asset returns by an amount proportional to the *bid-ask spread*.

```
> # bidask equal to 1 bp for liquid ETFs
> bidask <- 0.001
> # Calculate the wealth paths
> kelly_ratio <- drop(mean(retp)/var(retp))
> wealthv <- cumprod(1 + kelly_ratio*retp)
> wealth_trans <- cumprod(1 + kelly_ratio*retp -
+ 0.5*bidask*kelly_ratio*(kelly_ratio-1)*abs(retp))
> # Calculate the compounded wealth from returns
> wealthv <- cbind(wealthv, wealth_trans)
> colnames(wealthv) <- c("Kelly", "Including bid-ask")
> # Plot compounded wealth
> dygraphs::dygraph(wealthv, main="Kelly Strategy With Transaction Costs")
+ dyOptions(colors=c("green", "blue"), strokeWidth=2) %>%
+ dyLegend(show="always", width=300)
```

draft: The Half-Kelly Criterion

In reality investors don't know the probability of winning or the odds of the gamble, so they can't accurately Calculate the optimal *Kelly fraction*.

The *Kelly fraction*: $k_f = \frac{\bar{r}}{\sigma^2}$ is especially sensitive to the uncertainty of the expected returns $\bar{r}$.

If the expected returns are over-estimated, then it can produce an inflated value of the *Kelly fraction*, leading to ruin.

The risk of applying too much leverage (over-betting) is much greater than the risk of applying too little leverage (under-betting).

Too much leverage (over-betting) not only reduces returns, but it increases the risk of ruin.

So in practice many investors apply only half the theoretical *Kelly fraction* (the Half-Kelly), to reduce the risk of ruin.

Perform bootstrap simulation to obtain the standard error of the *Kelly fraction*.

```
> # Plot several Kelly curves
> curve(expr=kelly_frac(x, b=1), xlim=c(0, 5),
+ ylim=c(-1, 1.5), xlab="betting odds",
+ ylab="kelly fraction", main="", lwd=2)
> abline(h=0.5, lwd=2, col="red")
> text(x=1.5, y=0.5, pos=3, cex=0.8, labels="b=1.0; max fraction=0.5")
> curve(expr=kelly_frac(x, b=0.5), add=TRUE, main="", lwd=2)
> abline(h=1.0, lwd=2, col="red")
> text(x=1.5, y=1.0, pos=3, cex=0.8, labels="b=0.5; max fraction=1.0")
> title(main="Kelly fraction", line=-0.8)
```

Risk Aversion

Risk aversion is the investor preference to avoid losses more than to seek similar percentage gains in wealth.

For example, for a risk averse investor, a 10% loss of wealth is more important than a 10% gain.

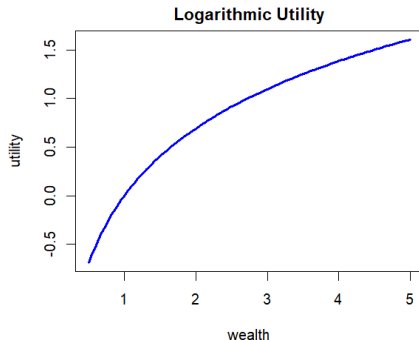
Risk aversion is associated with the *diminishing marginal utility* of the percentage change in wealth Δw .

This manifests itself as a concave utility function, with a negative second derivative $u''(w) < 0$.

For example, the *logarithmic utility* function is concave.

The Arrow-Pratt coefficient of relative risk aversion is proportional to the convexity $u''(w)$ of the utility, and is defined as: $\eta = -\frac{w u''(w)}{u'(w)}$.

The relative risk aversion of *logarithmic utility* is equal to one: $\eta = 1$.



```
> # Plot logarithmic utility function
> curve(expr=log, lwd=3, col="blue", xlim=c(0.5, 5),
+ xlab="wealth", ylab="utility",
+ main="Logarithmic Utility")
```

Constant Relative Risk Aversion

It's not a given that all investors have a risk aversion coefficient equal to 1, and other *utility functions* are possible.

The Constant Relative Risk Aversion (*CRRA*) utility function is a generalization of logarithmic utility:

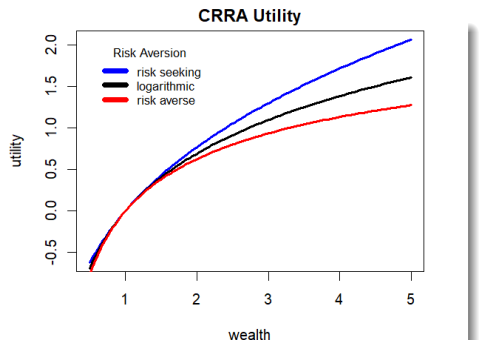
$$u(w) = \frac{w^{1-\eta} - 1}{1-\eta}$$

Where η is the risk aversion parameter.

The relative risk aversion of the *CRRA* utility function is constant and equal to η .

When the risk aversion parameter is equal to one $\eta = 1$, then the *CRRA* utility function is equal to the logarithmic utility.

In practice, the risk aversion parameter η is not known, and must be estimated through empirical studies.



```
> # Define CRRA utility
> cr_ra <- function(w, ra) {
+   (w^(1-ra) - 1)/(1-ra)
+ } ## end cr_ra
> # Plot utility functions
> curve(expr=cr_ra(x, ra=0.7), xlim=c(0.5, 5), lwd=3,
+ xlab="wealth", ylab="utility", main="", col="blue")
> curve(expr=log, add=TRUE, lwd=3)
> curve(expr=cr_ra(x, ra=1.3), add=TRUE, lwd=3, col="red")
> # Add title and legend
> title(main="CRRA Utility", line=0.5)
> legend(x="topleft", legend=c("risk seeking", "logarithmic", "risk
+ title="Risk Aversion", inset=0.05, cex=0.8, bg="white", y.intersp
+ lwd=6, lty=1, bty="n", col=c("blue", "black", "red"))
```

draft: CRRA Optimal Leverage

It's not a given that all investors have a risk aversion coefficient equal to 1, and other *utility functions* are possible.

The Constant Relative Risk Aversion (*CRRA*) utility function is a generalization of logarithmic utility:

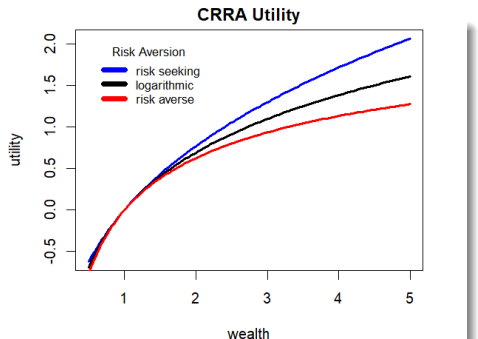
$$u(w) = \frac{w^{1-\eta} - 1}{1-\eta}$$

Where η is the risk aversion parameter.

The relative risk aversion of the *CRRA* utility function is constant and equal to η .

When the risk aversion parameter is equal to one $\eta = 1$, then the *CRRA* utility function is equal to the logarithmic utility.

In practice, the risk aversion parameter η is not known, and must be estimated through empirical studies.



```
> # Define CRRA utility
> cr_ra <- function(w, ra) {
+   (w^(1-ra) - 1)/(1-ra)
+ } ## end cr_ra
> # Plot utility functions
> curve(expr=cr_ra(x, ra=0.7), xlim=c(0.5, 5), lwd=3,
+ xlab="wealth", ylab="utility", main="", col="blue")
> curve(expr=log, add=TRUE, lwd=3)
> curve(expr=cr_ra(x, ra=1.3), add=TRUE, lwd=3, col="red")
> # Add title and legend
> title(main="CRRA Utility", line=0.5)
> legend(x="topleft", legend=c("risk seeking", "logarithmic", "risk
+ title="Risk Aversion", inset=0.05, cex=0.8, bg="white", y.intersp
+ lwd=6, lty=1, bty="n", col=c("blue", "black", "red"))
```

draft: CRRA Strategy Wealth Path

The wealth of a Kelly Strategy with a fixed leverage ratio k_f is equal to:

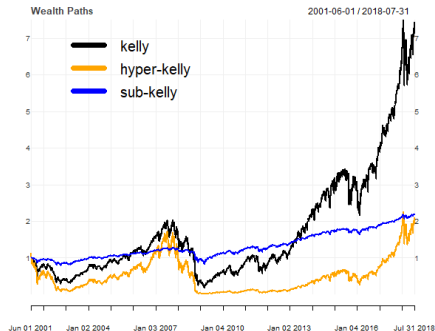
$$w_t = \prod_{i=1}^t (1 + k_f r_t)$$

The *Kelly fraction* k_f provides the optimal leverage to maximize the utility of wealth, by balancing the benefit of leveraging higher positive returns, with the risk of ruin due to excessive leverage.

If the mean asset returns are positive, then a higher leverage ratio provides higher returns.

But if the leverage is too high, then the losses in periods with negative returns wipe out most of the wealth, so then it's slow to recover.

```
> retp <- na.omit(retp)
> # Calculate the wealth paths
> kelly_ratio <- drop(mean(retp)/var(retp))
> kelly_wealthv <- cumprod(1 + kelly_ratio*retp)
> hyper_kelly <- cumprod(1 + (kelly_ratio+2)*retp)
> sub_kelly <- cumprod(1 + (kelly_ratio-2)*retp)
> kelly_paths <- cbind(kelly_wealth, hyper_kelly, sub_kelly)
> colnames(kelly_paths) <- c("kelly", "hyper-kelly", "sub-kelly")
```



```
> # Plot wealth paths
> themev <- chart_theme()
> themev$col$line.col <- c("black", "orange", "blue")
> quantmod::chart_Series(kelly_paths, theme=themev,
+   name="Wealth Paths")
> legend("topleft", legend=colnames(kelly_paths),
+   inset=0.1, bg="white", lty=1, lwd=6, y.intersp=0.5,
+   col=themev$col$line.col, bty="n")
```

The Utility of Lottery Tickets

Lottery tickets are equivalent to binary gambles with a very small probability of winning p , but a very large winning amount a , and a small loss amount b equal to the ticket price.

The expected payout $\mu = p a - q b$ of most lottery tickets is negative.

So under *logarithmic utility*, the Kelly fraction k_f for most lottery tickets is also negative, meaning that investors should not be expected to buy these lottery tickets.

But in reality many people do buy lottery tickets with negative expected payouts, which means that their utility functions are not logarithmic.

The demand for lottery tickets can be explained by assuming a strong demand for positive *skewness*, which exceeds the demand for a positive payout.

People buy lottery tickets because they want a small chance of a very large payout, even if the average payout is negative.

Without loss of generality we can assume that the lottery ticket price is one dollar $b = 1$, that it pays out a dollars, and that the expected payout is equal to zero: $\mu = p a - q b = 0$.

Then the probabilities of winning and losing are equal to: $p = \frac{1}{a+1}$ and $q = \frac{a}{a+1}$.

The variance is equal to: $\sigma^2 = p q (a + 1)^2 = a$.

And the *skewness* is equal to:

$$\varsigma = \frac{1}{\sigma^3} \left(\frac{a^3}{a+1} - \frac{a}{a+1} \right) = \frac{a-1}{\sqrt{a}}.$$

So the positive *skewness* of a lottery ticket increases as the square root of the *betting odds* a , and it can become very large for large *betting odds*.

Investor Risk Aversion, Prudence and Temperance

Investor risk and return preferences depend on the signs of the derivatives of their *utility* function.

Investors with *logarithmic utility* have positive *odd* derivatives ($u'(w) > 0$ and $u'''(w) > 0$) and negative *even* derivatives ($u''(w) < 0$ and $u''''(w) < 0$), which is typical for most other investors as well.

Risk averse investors have a negative second derivative of utility $u''(w) < 0$.

The demand for lottery tickets shows that investors' utility typically has a positive third derivative $u'''(w) > 0$.

Positive *odd* derivatives imply a preference for larger *odd moments* of the change in the wealth distribution (mean, skewness).

Negative *even* derivatives imply a preference for smaller *even moments* (variance, kurtosis).

The preference for smaller *variance* is called *risk aversion*, for larger *skewness* is called *prudence*, and for smaller *kurtosis* is called *temperance*.

The expected change of the *utility* of wealth $\mathbb{E}[\Delta u(w)]$ can be expanded in the moments of the wealth distribution Δw :

$$\begin{aligned}\mathbb{E}[\Delta u(w)] &= u'(w)\mathbb{E}[\Delta w] + \frac{u''(w)}{2}\sigma^2 \\ &\quad + \frac{u'''(w)}{3!}\mu_3 + \frac{u''''(w)}{4!}\mu_4\end{aligned}$$

Where $\mathbb{E}[\Delta w]$ is the expected change of wealth, $\sigma^2 = \int \Delta w^2 p(w) dw$ is the *variance* of The change in wealth, and $\mu_3 = \int \Delta w^3 p(w) dw = \sigma^3 \zeta$ and $\mu_4 = \int \Delta w^4 p(w) dw = \sigma^4 \kappa$ are the third and fourth moments, proportional to the *skewness* ζ and the *kurtosis* κ .

Investor Preferences and Empirical Return Distributions

The investor preference for higher *returns* and for lower *volatility* is expressed by maximizing the *Sharpe ratio*.

The third and fourth moments of asset returns are usually much smaller than the *variance*, so they typically have a smaller effect on the investor risk and return preferences.

Nevertheless, there is evidence that investors also have significant preferences for positive *skewness* and lower *kurtosis*.

But stock returns typically have negative *skewness* and excess *kurtosis*, the opposite of what investors prefer.

Many investors may prefer positive *skewness*, even at the expense of lower *returns*, similar to the buyers of lottery tickets.

A paper by Amaya asks if the *Realized Skewness Predicts the Cross-Section of Equity Returns?*

But higher moments are hard to estimate accurately from low frequency (daily) returns, which makes empirical investigations more difficult.

```
> # Calculate the VTI returns
> retp <- rutils::etfenv$returns$VTI
> retp <- na.omit(retp)
> # Calculate the higher moments of VTI returns
> c(mean=sum(retp),
+   variance=sum(retp^2),
+   mom3=sum(retp^3),
+   mom4=sum(retp^4))/NROW(retp)
> # Calculate the higher moments of minutely SPY returns
> spy <- HighFreq::SPY[, 4]
> spy <- na.omit(spy)
> spy <- rutils::diffit(log(spy))
> c(mean=sum(spy),
+   variance=sum(spy^2),
+   mom3=sum(spy^3),
+   mom4=sum(spy^4))/NROW(spy)
```

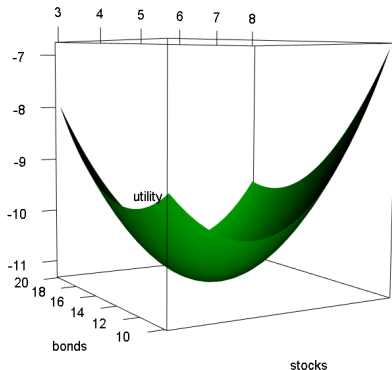
Utility of Stock and Bond Portfolio

The utility u of the stock and bond portfolio with weights $stocku$, $bondu$ is equal to:

$$u = \sum_{i=1}^n \log(1 + stocku r_i^s + bondu r_i^b)$$

Where r_i^s , r_i^b are the stock and bond returns.

```
> retp <- na.omit(rutils::etfenv$returns[, c("VTI", "IEF")])
> # Logarithmic utility of stock and bond portfolio
> utilfun <- function(stocku, bondu) {
+   -sum(log(1 + stocku*retp$VTI + bondu*retp$IEF))
+ } ## end utilfun
> # Create matrix of utility values
> stocku <- seq(from=3, to=7, by=0.2)
> bondu <- seq(from=12, to=20, by=0.2)
> utilm <- sapply(bondu, function(y) sapply(stocku,
+   function(x) utilfun(x, y)))
> # Set rgl options and load package rgl
> options(rgl.useNULL=TRUE)
> library(rgl)
> # Draw 3d surface plot of utility
> rgl::persp3d(stocku, bondu, utilm, col="green",
+   xlab="stocks", ylab="bonds", zlab="utility")
> # Render the surface plot
> rgl::rglwidget(elementId="plot3drgl1")
> # Save the surface plot to png file
> rgl::rgl.snapshot("utility_surface.png")
```



Kelly Optimal Weights

The Kelly optimal stock and bond portfolio weights $stock_u$, $bond_u$ can be calculated by maximizing the utility u .

```
> # Approximate Kelly weights
> weightv <- sapply(retp, function(x) mean(x)/var(x))
> # Kelly weight for stocks
> unlist(optimize(f=function(x) utilfun(x, bondu=0), interval=c(1,
> # Kelly weight for bonds
> unlist(optimize(f=function(x) utilfun(x, stocku=0), interval=c(1
> # Vectorized utility of stock and bond portfolio
> utility_vec <- function(weightv) {
+   utilfun(weightv[1], weightv[2])
+ } ## end utility_vec
> # Optimize with respect to vector argument
> optiml <- optim(fn=utility_vec, par=c(3, 10),
+   method="L-BFGS-B",
+   upper=c(8, 20), lower=c(2, 5))
> # Exact Kelly weights
> optiml$par
```

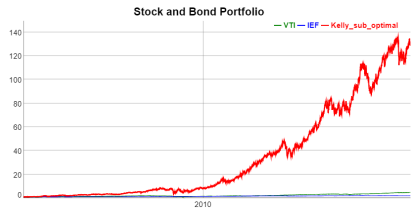
The Kelly optimal weights can be calculated approximately by first calculating the individual stock and bond weights, and then multiplying them by the Kelly weight of the combined portfolio.

```
> # Approximate Kelly weights
> retsport <- (retp %*% weightv)
> drop(mean(retsport)/var(retsport))*weightv
> # Exact Kelly weights
> optiml$par
```

Kelly Optimal Stock and Bond Portfolio

In practice, the Kelly optimal weights under logarithmic utility are too aggressive and they require very active trading, so half-Kelly or even quarter-Kelly weights are used instead.

```
> # Quarter-Kelly sub-optimal weights
> weightv <- optiml$par/4
> # Plot Kelly optimal portfolio
> retp <- cbind(retp, weightv[1]*retp$VTI + weightv[2]*retp$IEF)
> colnames(retp)[3] <- "Kelly_sub_optimal"
> # Calculate the compounded wealth from returns
> wealthv <- cumprod(1 + retp)
> # Plot compounded wealth
> dygraphs::dygraph(wealthv, main="Stock and Bond Portfolio") %>%
+   dyOptions(colors=c("green", "blue", "green")) %>%
+   dySeries("Kelly_sub_optimal", color="red", strokeWidth=2) %>%
+   dyLegend(show="always", width=300)
```



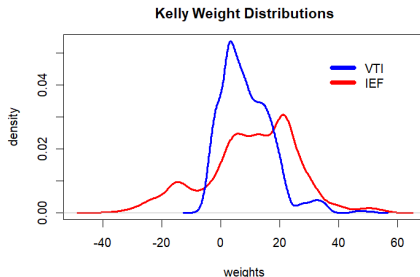
Rolling Kelly Weights

The Kelly weights k_f are calculated daily over a rolling look-back interval:

$$k_f = \frac{\bar{r}_t}{\sigma_t^2}$$

The distribution of the Kelly weights depends on the rolling returns $\bar{r}_t$ and variance σ_t^2 .

```
> retp <- na.omit(rutils::etfenv$returns[, c("VTI", "IEF")])
> # Calculate the rolling returns and variance
> lookb <- 200
> var_rolling <- HighFreq::roll_var(retp, lookb)
> weightv <- HighFreq::roll_sum(retp, lookb)/lookb
> weightv <- weightv/var_rolling
> weightv[1, ] <- 1/NCOL(weightv)
> weightv <- zoo::na.locf(weightv)
> sum(is.na(weightv))
> range(weightv)
```



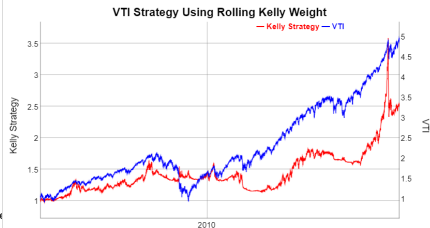
```
> # Plot the weights
> x11(width=6, height=5)
> par(mar=c(4, 4, 3, 1), oma=c(0, 0, 0, 0))
> plot(density(retp$IEF), t="l", lwd=3, col="red",
+       xlab="weights", ylab="density",
+       ylim=c(0, max(density(retp$VTI)$y)),
+       main="Kelly Weight Distributions")
> lines(density(retp$VTI), t="l", col="blue", lwd=3)
> legend("topright", legend=c("VTI", "IEF"),
+       inset=0.1, bg="white", lty=1, lwd=6, y.intersp=0.5,
+       col=c("blue", "red"), bty="n")
```

Rolling Kelly Strategy For Stocks

In the rolling Kelly strategy, the leverage of the risky asset k_f changes over time.

The leverage is equal to the updated weight from the previous period.

```
> # Scale and lag the Kelly weights
> weightv <- lapply(weightv, function(x) 10*x/sum(abs(range(x))))
> weightv <- do.call(cbind, weightv)
> weightv <- rutils::lagit(weightv)
> # Calculate the compounded Kelly wealth and VTI
> wealthv <- cbind(cumprod(1 + weightv$VTI*retp$VTI), cumprod(1 + retp$VTI))
> colnames(wealthv) <- c("Kelly Strategy", "VTI")
> dygraphs::dygraph(wealthv, main="VTI Strategy Using Rolling Kelly Weight") %>%
+   dyAxis("y", label="Kelly Strategy", independentTicks=TRUE) %>%
+   dyAxis("y2", label="VTI", independentTicks=TRUE) %>%
+   dySeries(name="Kelly Strategy", axis="y", strokeWidth=1, col="red") %>%
+   dySeries(name="VTI", axis="y2", strokeWidth=1, col="blue")
```



Rolling Kelly Strategy With Transaction Costs

The *margin debt* m_t is proportional to the wealth w_t :
 $m_t = (k_f - 1)w_t + 1$.

The dollar amount of the *risky asset* traded is equal to the change in *margin*, equal to: $\Delta m_t = \Delta[(k_f - 1)w_t]$.

If the transaction costs are large, then they will reduce the wealth and reduce the dollar amount of the *risky asset* held by the investor.

The transaction costs depend on the change in wealth, and the wealth is decreased by the transaction costs.

So the transaction costs in each time period must be calculated recursively in a loop from the wealth in the past period.

If the transaction costs are much less than the change in wealth $c^r \ll |\Delta w_t|$, then they can be calculated approximately as the absolute value of the change in *margin* m_t^{nc} for a wealth path with no transaction costs:

$$c^r = \frac{\delta}{2} |\Delta m_t^{nc}|$$

The transaction costs as a percentage of wealth are equal to: c_t/w_t^{nc} , where w_t^{nc} is the wealth assuming no transaction costs.

The wealth of the Kelly Strategy after accounting for the *bid-ask spread* is then equal to:

$$w_t = \prod_{i=1}^t \left(1 + k_f r_t - \frac{\delta}{2} \frac{|\Delta m_i^{nc}|}{w_i^{nc}} \right)$$

The effect of the *bid-ask spread* is to reduce the effective asset returns by an amount proportional to the *bid-ask spread*.

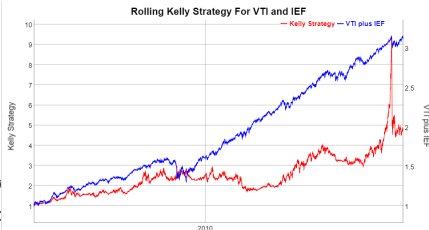
```
> # bidask equal to 1 bp for liquid ETFs
> bidask <- 0.001
> # Calculate the compounded Kelly wealth and margin
> wealthv <- cumprod(1 + weightv$VTI*retp$VTI)
> marginv <- (retp$VTI - 1)*wealthv + 1
> # Calculate the transaction costs
> costs <- bidask*drop(rutils::diffit(marginv))/2
> wealth_diff <- drop(rutils::diffit(wealthv))
> costs_rel <- ifelse(wealth_diff>0, costs/wealth_diff, 0)
> range(costs_rel)
> hist(costs_rel, breaks=10000, xlim=c(-0.02, 0.02))
> # Scale and lag the transaction costs
> costs <- rutils::lagit(abs(costs)/wealthv)
> # ReCalculate the compounded Kelly wealth
> wealth_trans <- cumprod(1 + retp$VTI*retp$VTI - costs)
> # Plot compounded wealth
> wealthv <- cbind(wealthv, wealth_trans)
> colnames(wealthv) <- c("Kelly", "Including bid-ask")
> dygraphs::dygraph(wealthv, main="Kelly Strategy With Transaction Costs")
+ dyOptions(colors=c("green", "blue"), strokeWidth=2) %>%
```

Rolling Kelly Strategy For Stocks and Bonds

In the rolling Kelly strategy, the leverage of the risky asset k_f changes over time.

The leverage is equal to the updated weight from the previous period.

```
> # Calculate the compounded wealth from returns
> wealthv <- cumprod(1 + rowSums(weightv*retp))
> wealthv <- xts::xts(wealthv, zoo::index(retp))
> quantmod::chart_Series(wealthv, name="Rolling Kelly Strategy For VTI and IEF")
> # Calculate the compounded Kelly wealth and VTI
> wealthv <- cbind(wealthv, cumprod(1 + 0.6*retp$IEF + 0.4*retp$VTI))
> colnames(wealthv) <- c("Kelly Strategy", "VTI plus IEF")
> dygraphs::dygraph(wealthv, main="Rolling Kelly Strategy For VTI and IEF") %>%
+   dyAxis("y", label="Kelly Strategy", independentTicks=TRUE) %>%
+   dyAxis("y2", label="VTI plus IEF", independentTicks=TRUE) %>%
+   dySeries(name="Kelly Strategy", axis="y", strokeWidth=1, col="red") %>%
+   dySeries(name="VTI plus IEF", axis="y2", strokeWidth=1, col="blue")
```



Tests for Market Timing Skill

Market timing skill a trading strategy is the ability to switch market positions, from long risk to short and vice versa, in anticipation of future market returns.

If a trading strategy has timing skill, then its returns have a positive convexity with respect to the market returns. The beta-adjusted strategy returns are positive, both when the market returns are positive and when they are negative.

The *market timing* skill can be measured by performing a *linear regression* of a strategy's returns against a strategy with perfect *market timing* skill.

The *Merton-Henriksson* market timing test uses a linear *market timing* term:

$$R - R_f = \alpha + \beta(R_m - R_f) + \gamma \max(R_m - R_f, 0) + \varepsilon$$

Where R are the strategy returns, R_m are the market returns, and R_f are the risk-free rates.

If the coefficient γ is statistically significant, then it's very likely due to *market timing* skill.

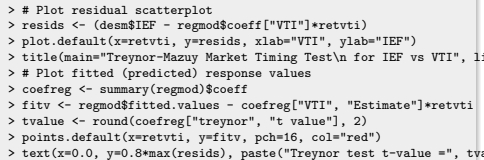
The *market timing* regression is a generalization of the *Capital Asset Pricing Model*.

The *Treynor-Mazuy* test uses a quadratic term, which makes it more sensitive to the magnitude of returns:

$$R - R_f = \alpha + \beta(R_m - R_f) + \gamma(R_m - R_f)^2 + \varepsilon$$

```
> # Create a design matrix of IEF and VTI returns
> desm <- na.omit(rutils::etfenv$returns[, c("IEF", "VTI")])
> retvti <- desm$VTI
> # Add returns with perfect timing skill
> desm <- cbind(desm, 0.5*(retvti+abs(retvti)), retvti^2)
> colnames(desm)[3:4] <- c("merton", "treynor")
> # Perform Merton-Henriksson test regression
> regmod <- lm(IEF ~ VTI + merton, data=desm); summary(regmod)
> # Perform Treynor-Mazuy test regression
> regmod <- lm(IEF ~ VTI + treynor, data=desm); summary(regmod)
```

Contrarian strategies have negative timing skill. Because they buy stocks when their prices are dropping and sell them when the prices are rising, expecting that the trend will reverse.



draft: Identifying Managers With Skill

Consider a binary investment (gamble) with the probability of winning equal to p , the winning amount (gain) equal to a , and the loss equal to b .

The investor makes no up-front payments, and either wins an amount a , or loses an amount b .

Assuming that an investor makes decisions exclusively on the basis of the expected value of future wealth, then they would choose to invest all their wealth on the gamble if its expected value is positive, and choose not to invest at all if its expected value is negative.

	win	lose
probability	p	$q = 1 - p$
payout	a	$-b$

The expected value of the gamble is equal to:
 $m = p a - q b$.

The variance of the gamble is equal to:
 $var = p q (a + b)^2$.

Without loss of generality we can assume that
 $p = q = \frac{1}{2}$,
 $m = 0.5 (b - a)$,
 $var = 0.25 (a + b)^2$.

The *Sharpe ratio* of the gamble is then equal to:

$$S_r = \frac{m}{\sqrt{var}} = \frac{(b - a)}{(a + b)}$$